

SARA (Smart Assistive Reading Aid): An Offline Smart Glass for Visually Impaired People that Reads Bangla and English Text, Detects Objects, and Recognises and Checks Bangladeshi Taka Notes

Spring 2026

Prepared by:

MD Abdun Fattah Lam
ID: 2220014

Mukut Protim Memo
ID: 2020086

Tanjila Hossain Tapti
ID: 2130561

Department of Computer Science and Engineering
Independent University, Bangladesh

Supervised by

Mohammad Shidujaman, PhD
Assistant Professor
Department of Computer Science and Engineering
School of Engineering, Technology and Sciences
Independent University, Bangladesh

Attestation

We are aware of the fact that plagiarism is strictly prohibited and it conflicts with our university's rules and regulations. We guarantee the authenticity of our work. We have used copyrighted material, public datasets and pre-trained models from others in our project, which have been properly cited in accordance with international standards and guidelines.

Name: MD Abdun Fattah Lam (ID: 2220014)

. . . . . .
Signature with Date

Name: Mukut Protim Memo (ID: 2020086)

. . . . . .
Signature with Date

Name: Tanjila Hossain Tapti (ID: 2130561)

. . . . . .
Signature with Date

Evaluation Committee

Supervisor

Name: _____ Signature: _____

Internal Examiner 1

Name: _____ Signature: _____

Internal Examiner 2

Name: _____ Signature: _____

External Examiner

Name: _____ Signature: _____

Acknowledgement

We commence by offering our heartfelt gratitude to the Almighty Allah for endowing us with the fortitude, perseverance, and capability to diligently embark on and successfully complete our senior project. Our profound appreciation extends to Mohammad Shidujaman, PhD, our esteemed Supervisor, Assistant Professor of Independent University, Bangladesh. The invaluable guidance, patience, and time our supervisor invested have been instrumental in steering us through the technical and practical challenges of building an assistive device, and in shaping the formulation of this report.

We are grateful to the creators of the public datasets that made this work possible, in particular the BanglaTaka currency dataset, the JaalTaka genuine and counterfeit note collection, Microsoft COCO, RAF-DB, and the Ekush Bangla handwriting dataset, as well as the open-source communities behind Ultralytics YOLO, PyTorch, OpenCV, EasyOCR, Piper and espeak-ng.

Moreover, we express our sincere gratitude to the committee members for their valuable contributions during the defence session. Their feedback and recommendations served to augment the depth and quality of our project significantly.

In addition to our academic mentors, we are deeply indebted to our family and friends for their unwavering support and encouragement throughout our tenure in the B.Sc. program, particularly during the more challenging phases of our senior project.

Furthermore, our heartfelt appreciation extends to Independent University, Bangladesh for fostering an enriching senior project program that has played an integral role in honing our skills and preparing us for research and industry.

Abstract

A blind person in Bangladesh who wants to read a label, or to know which banknote is in the hand and whether it is real, usually has to ask someone. The assistive devices that could help are expensive, built for English, and often need a phone or the internet. This thesis presents *SARA (Smart Assistive Reading Aid)*, an assistive smart glass that speaks Bangla and reads Bangla and English, built on a Raspberry Pi 5 with a camera, five push-buttons and audio output. It has five modes: Bangla and English text reading, object announcement, Bangladeshi Taka recognition with a cautious counterfeit check, facial-expression announcement, and an optional online scene description. Everything except the last runs on the device.

To recognise notes, we pasted 5,073 background-free note images onto real photographs and let the program write the labels, which gave 22,333 training images without any manual annotation. A YOLOv8s detector trained on them reaches 0.995 mAP@0.5 and 0.849 mAP@0.5:0.95 on 2,282 held-out composites. On 1,536 photographs taken by other people it names the right denomination 91.5% of the time.

For counterfeit notes we worked with the JaalTaka dataset (1,390 notes, six views each). A model trained on all six views breaks down when a user captures fewer; training on random prefixes of the views fixes this and raises one-view accuracy from 71.2% to 98.2% over three seeds. We also found that the usual split of this dataset lets the same counterfeit print appear in both training and test data, and built a stricter split with prints the model has never seen. On it, checking the banknote watermark, found by a learned 4.3 MB localizer, raises accuracy from about 90% to 95.0%. We further propose a denomination-matched watermark residual (DMWR): it removes the lamp from the back-lit photograph, compares the watermark with a "genuine portrait" of the note value built from genuine notes only, and classifies what is left over. With 47,818 parameters (0.21 MB) it reaches 91.5% on unseen prints, statistically level with a fine-tuned MobileNetV2 that is 46 times larger. Because a wrong verdict can hurt a blind user, the glass never says "counterfeit". It says "likely genuine" only above a threshold fixed on validation data and otherwise asks for a check by hand; none of 88 test counterfeits was passed as genuine.

For text reading we built a benchmark with correctly shaped Bangla and with separate phrases and fonts for validation and test. Keeping only the main block of text, tuning the detector and cleaning the output lowers the character error rate from 9.3% to 2.4% overall and from 11.9% to 3.2% for Bangla. On real street photographs of single words the error was 39% for Bangla; a recogniser fine-tuned on real words and blended with the released weights lowers it to 29% (Latin script 62% to 51%) without losing accuracy on printed text, but reading real signs remains the hardest part.

On the Raspberry Pi 5 the currency check takes a median of 327 ms, and the system ran for 30 minutes within 1.5 GB of memory and without thermal throttling. Trial sessions with a visually impaired participant have started, supported by a voice-driven questionnaire. Every number in this thesis comes from a saved result file.

Keywords: Assistive Technology, Smart Glass, Visually Impaired, Raspberry Pi, Bangla OCR, YOLOv8, Bangladeshi Currency, Counterfeit Detection, Multi-View Learning, Watermark, Quotient Image, Selective Prediction, Edge AI

Contents

1 Introduction **14**
- 1.1 Background . . . 14
- 1.2 Problem Statement . . . 15
- 1.3 Objectives . . . 17
- 1.4 Scope of the Study . . . 17
- 1.5 Research Questions . . . 18
- 1.6 Contributions . . . 18
- 1.7 Thesis Structure . . . 19

2 Literature Review **20**
- 2.1 Assistive Technology for the Visually Impaired . . . 20
- 2.2 Historical Background and Evolution . . . 20
- 2.3 Key Technologies in Smart Assistive Glasses . . . 21
  - 2.3.1 Embedded Computing Platforms . . . 21
  - 2.3.2 Optical Character Recognition . . . 21
  - 2.3.3 Object Detection . . . 21
  - 2.3.4 Speech Output and Feedback . . . 21
- 2.4 Currency Recognition and Authentication . . . 22
- 2.5 Multi-View Learning with Missing Views . . . 22
- 2.6 Evaluation Leakage and Shortcuts . . . 22
- 2.7 Prediction with a Reject Option . . . 23
- 2.8 Evaluating Assistive Devices with Users . . . 23
- 2.9 Real-World Applications . . . 23
  - 2.9.1 Reading Assistants . . . 23
  - 2.9.2 Multi-Function Wearables . . . 23
  - 2.9.3 Mobile Applications . . . 23
- 2.10 Summary of Reviewed Assistive Reading Systems . . . 24
- 2.11 Comparative Evaluation of Assistive Vision Systems . . . 26
- 2.12 Key Research Areas Relevant to SARA . . . 28
- 2.13 Research Gap . . . 30

3 System Design and Methodology **32**
- 3.1 Framework of SARA . . . 32
- 3.2 Overview of SARA System Design . . . 32
  - 3.2.1 General Overview of SARA . . . 33
  - 3.2.2 Structural Characteristics of SARA . . . 33
  - 3.2.3 Interactive Features and Interface Design . . . 34
- 3.3 Hardware Components . . . 34
- 3.4 Software Design . . . 37
  - 3.4.1 Text Reading Module . . . 37

3.4.2 Object Detection Module . . . 39
3.4.3 Currency Detection Module . . . 39
3.4.4 Currency Authentication Module . . . 39
3.4.5 Safe Verdict Policy on the Glass . . . 40
3.4.6 Watermark Check with Guided Capture . . . 40
3.4.7 Denomination-Matched Watermark Residual (DMWR) . . . 42
3.4.8 UV Security-Fibre Check . . . 44
3.4.9 Emotion Mode . . . 45
3.4.10 Field Log . . . 45
3.4.11 Voice Interaction and Feedback . . . 45
3.4.12 Overall System Flow and Logic . . . 46
3.5 Methodology: Building SARA . . . 46
3.5.1 Requirement Analysis . . . 47
3.5.2 Data Engineering and Model Development . . . 47
3.5.3 System Integration . . . 48
3.5.4 Evaluation Protocol . . . 48

4 Implementation Process 50
4.1 Design Overview . . . 50
4.1.1 User-Centred Design . . . 50
4.1.2 Prototype Development Environment . . . 50
4.2 Hardware Implementation . . . 50
4.2.1 Component Assembly . . . 50
4.2.2 Connectivity and Wiring . . . 51
4.3 Software Deployment and Functional Integration . . . 52
4.3.1 Environment Setup and Libraries . . . 52
4.3.2 Windows Development Harness . . . 58
4.3.3 Error Handling and Functional Validation . . . 58
4.4 Testing and Calibration . . . 59
4.5 Deployment Problems on the Raspberry Pi 5 . . . 59

5 Dataset and Data Engineering 61
5.1 Dataset . . . 61
5.1.1 Implications of These Datasets on Our Research . . . 63
5.2 Currency Detection Data Engineering . . . 63
5.2.1 Source Currency Dataset (Foreground) . . . 63
5.2.2 Background Dataset . . . 63
5.2.3 The Domain-Gap Problem . . . 63
5.2.4 Copy-Paste Compositing Process . . . 63
5.2.5 Augmentation Operations . . . 64
5.2.6 Final Detection Dataset . . . 65
5.2.7 Raw Photograph Test Set . . . 65
5.2.8 Independent Photograph Collections . . . 66
5.3 Currency Authentication Data . . . 66
5.3.1 Serial-Number Audit . . . 67
5.3.2 Print-Disjoint Split . . . 67
5.3.3 Back-Lit Views and Watermark Labels . . . 68
5.3.4 Whole-Note Counterfeit Photographs . . . 68
5.3.5 A Tool for a Larger Print-Disjoint Dataset . . . 68
5.4 Emotion Recognition Data . . . 68

5.5 Text Recognition Data . . . 68
5.5.1 OCR Benchmark . . . 68
5.5.2 Ekush Handwritten Characters . . . 69
5.6 Object Detection Data . . . 69
5.7 Note on User Survey . . . 69

6 Result and Discussion **70**
6.1 Performance Evaluation Overview . . . 70
6.2 Taka Detection Performance . . . 70
6.2.1 Training Behaviour . . . 70
6.2.2 Results on Held-Out Composites . . . 71
6.2.3 Results on Raw Phone Photographs (Source Domain, Not Held Out) . . . 73
6.2.4 Results on Independent Photographs . . . 74
6.2.5 Unknown Notes . . . 75
6.2.6 Live Check with a Laptop Webcam . . . 75
6.2.7 Several Notes in One Picture . . . 75
6.2.8 Fall-Back Classifier . . . 76
6.2.9 Comparison with Related Work . . . 76
6.3 Taka Authentication Performance . . . 76
6.3.1 Accuracy for One to Six Views . . . 76
6.3.2 Controlled Comparison: What Causes the Gain . . . 77
6.3.3 Comparison with Standard Backbones . . . 78
6.3.4 Repeatability over Seeds . . . 78
6.3.5 Other Metrics and Calibration . . . 78
6.3.6 Robustness to Image Corruptions . . . 79
6.4 Counterfeit Detection on Unseen Prints . . . 80
6.4.1 Learned Watermark Localizer . . . 81
6.4.2 Watermark Classifiers Compared, Including DMWR . . . 82
6.4.3 UV Security-Fibre Check . . . 83
6.4.4 Capture Guide . . . 85
6.5 The Verdict on the Glass . . . 85
6.5.1 Authentication on Whole-Note Photographs . . . 85
6.5.2 Safety of the Deployed Policy . . . 86
6.5.3 Other Training Variants and Negative Results . . . 87
6.5.4 Speed . . . 87
6.6 Emotion Recognition Performance . . . 87
6.7 Text Recognition Performance . . . 89
6.7.1 The Seven Steps . . . 89
6.7.2 Results on the Test Set . . . 90
6.7.3 Text Reading on Real Photographs . . . 90
6.7.4 Handwritten Bangla Character Recognition . . . 92
6.8 Object Detection Performance . . . 92
6.9 Processing Speed . . . 93
6.9.1 Laptop . . . 93
6.9.2 Raspberry Pi 5 . . . 93
6.9.3 Voice Output and Voice Commands (14 October 2026) . . . 95
6.9.4 Raspberry Pi 5, Second Run (13 October 2026) . . . 95
6.10 First Field Test with the Glass . . . 97
6.11 Corrections to Earlier Figures . . . 98
6.12 Discussion . . . 100

6.13 Scope of the Results . . . 100

7 Conclusion and Future Work **102**
7.1 Conclusion . . . 102
7.2 Future Work . . . 103
7.2.1 Tuning the Glass Camera . . . 103
7.2.2 User Study with Visually Impaired Participants . . . 103
7.2.3 A Print-Disjoint Counterfeit Dataset . . . 103
7.2.4 Faster and Better Text Reading . . . 103
7.2.5 Power, Speech and Remaining Measurements . . . 103
7.2.6 Navigation and Voice Control . . . 103
7.2.7 Mechanical Design . . . 104
7.3 Impact and Sustainability . . . 104
7.4 Social and Ethical Considerations . . . 104
7.5 Contributions and Final Thoughts . . . 104
7.6 Analysis of Social Effects . . . 104
7.6.1 Reducing Fraud and Financial Risk . . . 105
7.6.2 Supporting Education and Employment . . . 105
7.6.3 Changing Perceptions of Assistive Technology . . . 105
7.6.4 Possible Social Risks . . . 105
7.6.5 Summary of Social Impact . . . 105

A Appendix **106**
A.1 Additional Plots . . . 107
A.1.1 Taka Detector Precision–Recall Curve . . . 107
A.1.2 Sample Detections . . . 107
A.1.3 Authentication Calibration . . . 108
A.1.4 Authentication over Seeds . . . 108
A.2 Configuration Parameters . . . 109
A.3 Reproducing the Results . . . 110
A.4 Spoken Messages . . . 111
A.5 User Study Questionnaire . . . 111

Bibliography **112**

List of Figures

1 Left: a 10 Taka note from the local currency dataset. Right: a 100 Taka note placed on a real street background, the kind of cluttered scene the currency detector of SARA is trained to handle. 15

2 Input–process–output framework of SARA. The emotion mode and the optional online mode use the same path and are omitted for clarity. 33

3 First prototype of SARA: the camera module above the bridge of a 3D-printed frame (left), the Raspberry Pi 5 with heat sink and fan tied to the right temple (centre), and the ribbon cable between camera and board (right) 34

4 YOLOv8s detection architecture used for Taka detection: CSPDarknet backbone, PAN-FPN neck and decoupled anchor-free head predicting 9 classes and box coordinates 39

5 Multi-view currency authenticator. During prefix-robust training, a random prefix of 1 to 6 views is kept so the model learns to decide with any number of views. 40

6 Watermark check with guided capture 42

7 Genuine 200 and 1,000 Taka notes under a UV lamp (left of each pair) and the security fibres found by the check (green, right of each pair) 45

8 Overall control flow of SARA 46

9 Currency detection methodology: source notes, COCO backgrounds and COCO-pretrained YOLOv8s weights feed the compositing engine, which produces the labelled dataset used for training, evaluation and deployment . . . 47

10 The first prototype worn during a trial session. A 200 Taka and a 100 Taka note are held in front of the camera for the currency mode. The photographs are used with the wearer's consent. 51

11 Live detection application on a laptop webcam: a 1,000 Taka and a 500 Taka note held by hand are detected with confidence 0.82 and 0.63 56

12 Grad-CAM explanation for a 100 Taka note: (a) input, (b) heat map, (c) overlay. The model focuses on the portrait and numeral regions. 56

13 Copy-paste compositing applied to a 100 Taka note: (a) background-free source note, (b) real COCO background, (c) note after perspective warp, rotation, scaling and colour jitter, (d) final composite with the automatically generated bounding box and label 64

14 The six ordered views of one genuine note in the JaalTaka collection. Each view shows a different region, such as the portrait with the hologram strip and the serial number. 66

15 Training curves of the YOLOv8s Taka detector: box, class and DFL losses (top) and mAP@0.5, mAP@0.5:0.95, precision and recall (bottom) against epoch . . 71

16 Normalised confusion matrix of the Taka detector on the synthetic test set . . . 72

17 Example predictions on held-out composites with predicted class and confidence. The note is localised across different scales, rotations and backgrounds. 73
18 Live detection of real notes with a laptop webcam (photographs of the screen). Top: 50, 200 and 100 Taka. Bottom: a worn 10 Taka note, 1,000 and 500 Taka together, and three notes in one frame. 75
19 Authentication accuracy against the number of views for the CNN+ViT baseline, PRMVT and the other training variants tried in this project 77
20 Normalised confusion matrices of the baseline and PRMVT at six views (0 = counterfeit, 1 = genuine) 79
21 Accuracy drop (clean minus corrupted accuracy) at six views under 12 corruptions for PRMVT, the NDAL variant and the baseline 80
22 DMWR on two 500 Taka test notes with unseen prints (seed 42). Left to right: the back-lit watermark window; the quotient image Q with the lamp removed; the genuine portrait T_{500} built from training notes only; the residual R (red above, blue below the portrait). On the counterfeit note the printed portrait stands out in the residual. 83
23 Text reading on the Raspberry Pi 5 on 13 October 2026: response time of each capture and where it goes (from the benchmark report) 96

24 Precision–recall curves of the YOLOv8s Taka detector for all nine classes on the held-out synthetic test set 107
25 Sample detections of the Taka detector on composited scenes 107
26 Reliability diagrams of the authenticator at six views for five confidence estimates (raw predictive confidence, temperature scaling, entropy mapping, HER and MC dropout); calibration maps fitted on the validation split and evaluated on the test split 108
27 Spread of test accuracy over seeds 42, 43 and 44 at one view (1v) and six views (6v) for the main training variants 108

List of Tables

2.10.0.1 Summary of reviewed assistive reading systems for the visually impaired 25
2.11.0.1 Overview of assistive vision systems for visually impaired users, with key technologies, strengths and limitations . . . 27
2.11.0.2 Comparison of currency recognition methods related to Bangladeshi Taka 28
2.12.0.1 Key research categories in assistive vision technology . . . 29
2.13.0.1 Research gaps and how this thesis responds to them . . . 31

3.3.0.1 Hardware components of SARA . . . 35
3.3.0.2 Button map of SARA . . . 36
3.4.0.1 Software modules of SARA . . . 37

4.3.1.1 Main libraries and tools used in SARA . . . 53
4.3.1.2 Voice commands of SARA . . . 58
4.5.0.1 Problems met when deploying on the Raspberry Pi 5, and their fixes . . . 60

5.1.0.1 Datasets used in this project . . . 62
5.2.1.1 Per-denomination image counts in the BanglaTaka source dataset . . . 63
5.2.5.1 Domain-randomisation operations used in compositing . . . 65
5.2.6.1 Composition of the synthetic Taka detection dataset . . . 65
5.3.0.1 JaalTaka split used for authentication (split seed 42) . . . 67
5.3.1.1 Serial numbers read from JaalTaka notes . . . 67
5.4.0.1 Class distribution of the RAF-DB official test set . . . 68

6.2.2.1 Taka detection performance on the synthetic test set (2,282 images) . . . 71
6.2.2.2 Per-denomination mAP@0.5:0.95 on the synthetic test set . . . 72
6.2.3.1 Taka detection on 450 raw phone photographs (50 per denomination). Source-domain check: these photographs come from the detector's training source and are not held out. Independent results: Table 6.2.4.1. . 74
6.2.4.1 Taka detection on independent photograph collections . . . 74
6.3.1.1 Authentication accuracy on 208 test notes with the first k views (seed 42) 76
6.3.2.1 Same network, four ways of training it: test accuracy (%) on 208 notes, average ± standard deviation over three training runs (seeds 42–44). "Normalisation" is batch normalisation (BN). . . . 77
6.3.4.1 PRMVT accuracy over three seeds (mean ± standard deviation) . . . 78
6.4.0.1 Accuracy on unseen counterfeit prints: 222 test notes, 101 counterfeit, mean ± standard deviation over three seeds . . . 81
6.4.1.1 Learned watermark localizer on the print-disjoint test split, three seeds . . 82
6.4.2.1 Watermark classifiers on the 197 registered test notes with unseen counterfeit prints . . . 82
6.4.3.1 UV fibre check on photographs of genuine notes (no counterfeit note was tested) . . . 84

6.4.3.2 UV fibre check before and after the colour condition. Genuine side: real photographs. Fibre-free side: simulated printed sheets; no counterfeit note was photographed. 84
6.4.4.1 Capture guide on 222 back-lit test photographs 85
6.5.1.1 Deployed genuine/counterfeit verdict on whole-note photographs (one crop, threshold 0.5) 85
6.5.1.2 Genuine whole-note photographs called counterfeit at $p = 0.5$, and ranking quality (AUC) on the counterfeit set 86
6.5.2.1 Deployed verdict policy: "likely genuine" above τ, otherwise "check by hand" 86
6.6.0.1 Emotion recognition models compared during development 88
6.6.0.2 Results for each expression of the EfficientNet-V2-S emotion model on RAF-DB (3,068 test faces). Precision: of the faces given this label, the share that were right. Recall: of the faces with this expression, the share recognised. F1: a balance of the two. 88
6.6.0.3 Confusion matrix of the EfficientNet-V2-S emotion model on the RAF-DB official test set. Each row is the real expression and each column the model's answer; the bold diagonal counts the correct answers. 88
6.7.1.1 How the text-reading pipeline was built, step by step, on the validation set. Error is the character error rate (CER): the share of letters read wrongly, lower is better. "Kept" marks what went into the glass. 89
6.7.2.1 Text reading on the held-out test set (432 images, three seeds), measured once per method. All numbers are error rates: lower is better. 90
6.7.3.1 Text reading on real photographs: word images of the ICDAR 2019 MLT collection, read once with the pipeline of the glass 91
6.8.0.1 Object detection on all 5,000 COCO val2017 photographs (mAP@0.5, predictions with confidence of at least 0.15) 92
6.9.1.1 Measured processing speed on the development laptop 93
6.9.2.1 Latency on the Raspberry Pi 5 (100 calls per module) 94
6.9.3.1 Voice commands with synthetic speech (117 spoken commands, 90 English and 10 Bangla non-command sentences per noise level) 95
6.9.4.1 Second run on the Raspberry Pi 5 (13 October 2026), on a hot and throttled board 96

A.2.0.1 Main configuration parameters of SARA (`config.py`) 109
A.2.0.2 Training hyper-parameters of the YOLOv8s Taka detector 110
A.4.0.1 Main spoken messages of SARA (translated from Bangla) 111

List of Algorithms

3.1 Text reading with the main text block (pipeline v2) 38
3.2 Currency answer with the safe verdict policy 41
3.3 Guided back-lit capture and watermark check 42
3.4 Denomination-matched watermark residual (DMWR) 44
3.5 Prefix-robust multi-view training (PRMVT) 48
5.1 Copy-paste compositing with automatic labels 64

1 Introduction

1.1 Background

Vision is the sense through which people obtain most of the information about their surroundings. According to the World Health Organization, at least 2.2 billion people worldwide live with some form of near or distance vision impairment, and a large share of them live in low- and middle-income countries [1, 2]. For a person who is blind or has low vision, everyday actions that sighted people perform without thinking become difficult or impossible: reading a medicine label, a shop sign or a bus number, knowing what object is on the table, or checking which banknote is in the hand and whether it is genuine. Each of these tasks usually requires help from another person, which reduces independence, privacy and dignity.

Bangladesh faces this problem in a particular form. Most printed information in public places is written in Bangla, often mixed with English. Cash is still the main payment method in daily life, and the country has nine circulating denominations of Taka notes (2, 5, 10, 20, 50, 100, 200, 500 and 1000). Counterfeit notes are a recurring problem in markets. At the same time, internet connectivity and the purchasing power of many visually impaired users are limited. A visually impaired person in Bangladesh therefore needs an assistive device that (i) speaks Bangla, (ii) recognises Bangladeshi currency, (iii) works without the internet, and (iv) is affordable and simple to operate.

Commercial assistive products such as OrCam MyEye [3], Envision Glasses [4] and Microsoft Seeing AI [5] show that camera-based assistance is technically possible. However, these products are expensive, are designed mostly for English and other major languages, rely in many functions on a smartphone or a cloud service, and do not support Bangladeshi Taka. Research prototypes built on Raspberry Pi boards [6, 7, 8] are cheaper, but most of them read only English text with Tesseract and send speech through online services such as Google Text-to-Speech.

This thesis presents **SARA (Smart Assistive Reading Aid)**, a wearable, offline smart glass that speaks Bangla and reads Bangla and English, for visually impaired users. SARA is designed around a Raspberry Pi 5 [9], a camera module mounted near the eyes, five physical push-buttons and an earphone or speaker. It offers five modes that a user cycles with a single button: *text reading* (Bangla and English OCR), *object detection* (spoken names of everyday objects in Bangla), *currency detection* (denomination of Bangladeshi Taka notes, with a safe counterfeit check for 500 and 1,000 Taka notes), an optional *online scene description*, and *emotion* (the facial expression of the person in front of the user). All modes except the online one run on the device itself without internet access once the models are installed.

The work is also a data-engineering study. Public Bangladeshi currency datasets consist mainly of clean, close-up scans of notes. A model trained only on them learns that "the whole frame is the note" and fails on a live camera. We therefore built a copy-paste compositing pipeline that places notes on real photographs and generates exact bounding boxes automatically. For counterfeit detection we worked with a six-view genuine/counterfeit note dataset and found that a model trained only on six views collapses when it sees one view, which led to a prefix-robust training protocol. We then found that the dataset's usual split is easier than it looks:



Figure 1: Left: a 10 Taka note from the local currency dataset. Right: a 100 Taka note placed on a real street background, the kind of cluttered scene the currency detector of SARA is trained to handle.

most counterfeit notes are copies of a few prints, and the same print appears in training and test. This led to a second split with unseen prints, to a check of the banknote watermark, and to a deliberately cautious spoken verdict.

For the watermark we also designed a new, very small classifier from the physics of a back-lit note: it removes the lamp from the photograph, compares the watermark with a genuine "portrait" of each note value, and classifies what is left over. Throughout the thesis, every choice is made on validation data and every number is taken from a saved evaluation file.

1.2 Problem Statement

Think of an ordinary errand. A blind person in Dhaka buys medicine, hands over a note and receives change. To know what the label on the packet says, which note came back, and whether that note is real, this person has to ask someone: a family member, the shopkeeper, a stranger. Most of the time the answer is honest. But the person has no way to check it, and has to ask again for the next label and the next note.

This dependence creates three concrete risks: loss of independence in daily activities, exposure to fraud when receiving change or counterfeit notes, and loss of privacy when others must read personal documents. Existing assistive technologies do not solve these problems for Bangladeshi users.

Common limitations in current solutions include:

- High cost of commercial smart glasses and dependence on a smartphone or subscription;
- Dependence on cloud services for OCR, recognition or speech, which fails without internet;
- English-centric design, with little or no support for Bangla text and Bangla speech;
- No support for recognising Bangladeshi Taka, and almost no support for detecting counterfeit notes;
- Touch-screen or app-based interfaces that are hard to use without sight.

On the technical side, five specific problems had to be addressed:

1. **Domain gap in currency recognition.** Models trained on clean note images do not transfer to real camera frames in which the note is small, rotated, partly lit and surrounded by clutter.
2. **Unreliable number of views.** Counterfeit authentication needs several close views of a note, but a blind user cannot guarantee that all views are captured; a model that is accurate only with all six views is not usable.
3. **Misleading evaluation.** Counterfeits printed from the same plate share one serial number. If such notes appear in both training and test data, a high score measures recognition of known reprints, not detection of new counterfeits.
4. **The cost of a wrong verdict.** A wrong "genuine" costs the user money; a wrong "counterfeit" can lead to a public accusation. The device has to be safe before it is decisive.
5. **Fragile Bangla reading on small hardware.** Bangla has conjuncts and vowel signs that move around the consonant, open OCR engines are weaker on it than on English, and everything has to run on a Raspberry Pi 5 without overheating.

The problem in one statement

Put together, the problem this thesis works on is the following.

> *A blind or low-vision person in Bangladesh has no affordable wearable device that reads Bangla text aloud, tells which Taka note is in the hand, and gives trustworthy advice on whether that note may be counterfeit, without a phone, a screen or an internet connection. The public data and test protocols on which such functions are built also make them look more reliable than they are in real use.*

The statement has two halves on purpose. The first is an engineering problem: build the device. The second is a measurement problem: find out how good the device really is. A sighted user notices when a device is wrong and stops trusting it. A blind user often cannot notice, so an error that goes unmeasured in the laboratory becomes an error that goes unnoticed in the street. For this user group, honest evaluation is part of the product, not an afterthought.

Who is affected and what success would look like

The people affected are blind and low-vision adults who handle cash and printed Bangla every day, and, indirectly, the family members they now depend on. For them the work would count as successful if a user wearing the glass can, alone and with buttons only, (i) hear a short Bangla sign or label read correctly, (ii) hear the right denomination of a note, (iii) be told honestly when the device is not sure, and (iv) never be told that a genuine note is counterfeit. Chapter 6 reports how far the present prototype is from each of these four points.

This project addresses these gaps and develops SARA, an assistive smart glass that brings together:

- Bangla and English text reading with a capture-then-read interaction;
- Everyday object announcement with Bangla labels and repetition control;
- Detection of all nine Bangladeshi Taka denominations in cluttered scenes;
- Multi-view genuine/counterfeit authentication that stays accurate with fewer views;
- A watermark check with spoken capture guidance, and a verdict policy that never says "counterfeit";
- Offline Bangla/English speech, haptic feedback and physical buttons for control.

1.3 Objectives

The primary objective of this project is to design, implement and evaluate SARA, an affordable, offline assistive smart glass for visually impaired people. The specific goals are:

1. To build a wearable prototype on Raspberry Pi 5 that is operated entirely through physical buttons and audio feedback.
2. To provide Bangla and English text reading using on-device OCR and offline text-to-speech.
3. To announce everyday objects in front of the user in Bangla using a real-time object detector.
4. To detect and name all nine denominations of Bangladeshi Taka in real-world scenes, by creating a large synthetic detection dataset without manual annotation.
5. To develop a counterfeit-note authenticator that remains reliable when only one or a few views of the note are available.
6. To measure counterfeit detection on prints never seen in training, and to design spoken feedback that is safe for a blind user.
7. To improve Bangla text reading with a benchmark in which all choices are made on validation data.
8. To add supporting features such as emotion announcement, haptic confirmation, field logging and a desktop test harness.
9. To measure latency, memory and temperature of every mode on the Raspberry Pi 5.
10. To evaluate every module with standard metrics on held-out data and to report the results and gaps honestly.

Ultimately, the objective is to create a human-centred, low-cost and privacy-preserving assistive device that increases the independence of visually impaired people in Bangladesh.

1.4 Scope of the Study

This research is concerned with the design, development and technical evaluation of a camera-based wearable assistant. The main functions of the system are the following:

- **Text Reading (OCR):** Recognition of printed Bangla and English text from the camera image and reading it aloud on request.
- **Object Detection:** Periodic detection of the 80 COCO object classes (person, chair, bottle, bus and so on) and announcement of the most relevant ones in Bangla.
- **Currency Detection:** Localisation and classification of the nine Bangladeshi Taka denominations, spoken in Bangla, with vibration confirmation.
- **Currency Authentication:** Classification of a 500 or 1,000 Taka note as genuine or counterfeit from one to six close-up views, a watermark check of a back-lit note, and a cautious spoken verdict on the glass.
- **Emotion:** Recognition of the facial expression of the person in front of the user, announced on request.
- **Online Scene Description:** An optional mode that sends one image to a cloud vision–language model; it needs the internet and an API key.

The study does not cover navigation, obstacle distance measurement, GPS, or voice-command control, although parts of these exist as experimental code. Model accuracy is evaluated on public datasets and on test sets built for this project; latency, memory and temperature are measured on the Raspberry Pi 5. A formal study with visually impaired participants, battery life and accuracy on photographs taken through the glass camera have not been measured and are presented as future work. The glass is an aid; it does not replace a white cane, a guide or a bank's counterfeit-detection equipment.

1.5 Research Questions

This research addresses the following questions:

1. **RQ1 (system).** How can a multi-function assistive smart glass with Bangla support be designed and run on a Raspberry Pi 5, and what latency, memory and temperature does it reach?

2. **RQ2 (synthetic data).** Can a detector trained on synthetically composited images, generated without manual annotation, recognise Bangladeshi Taka notes in independent real photographs?

3. **RQ3 (number of views).** Why does a multi-view counterfeit authenticator trained on a fixed number of views fail when fewer views are available, and can a training protocol fix this?

4. **RQ4 (unseen prints).** How much does counterfeit-detection accuracy fall when every test counterfeit comes from a print never seen in training, and can a physical security feature, the watermark, recover the loss?

5. **RQ5 (safety).** Can the glass give counterfeit feedback that never accuses a genuine note and has a bounded chance of passing a counterfeit, and what does this safety cost in usefulness?

6. **RQ6 (Bangla OCR).** Which parts of a Bangla OCR pipeline matter most for text read by a wearable camera: region selection, pre-processing, the engine, its parameters, post-processing or fine-tuning?

7. **RQ7 (interaction).** How can the interaction (buttons, capture/read separation, spoken capture guidance, Bangla speech) be made suitable for users without sight? This question is answered only at the level of design; the user study that would test it has not been carried out.

1.6 Contributions

The main contributions of this thesis are:

1. A complete five-mode assistive smart-glass system (SARA) with Bangla speech and physical button control, measured on a Raspberry Pi 5: a median of 327 ms for the currency check, 1.48 GB peak memory and no thermal throttling in a 30-minute run.
2. A copy-paste compositing pipeline that turns 5,073 clean note images into a 22,333-image detection dataset with automatic bounding boxes, and a YOLOv8s Taka detector trained on it that names 91.5% of 1,536 independent photographs correctly.

3. A prefix-robust multi-view training protocol that removes the train/test mismatch in the number of views: one-view accuracy 98.2% against 71.2% for fixed six-view training, over three seeds on a note-disjoint test set.
4. An audit showing that most JaalTaka counterfeits share a few printed serial numbers, a print-disjoint split built from it, and the finding that accuracy drops from about 98% to about 90% on unseen prints.
5. A denomination-matched watermark residual (DMWR), derived from a model of back-lit paper, that reaches 91.5% on unseen prints with 47,818 parameters, statistically level with a 46 times larger MobileNetV2;
6. A watermark check with a learned localizer and spoken capture guidance that raises unseen-print accuracy to 95.0%, and a verdict policy that never says "counterfeit" and passed 0 of 88 test counterfeits.
7. A Bangla and English OCR benchmark with correctly shaped text and separate validation and test phrases and fonts, and an OCR pipeline that lowers the character error rate from 9.3% to 2.4%.
8. Tools for the work that remains: a field logger, a print-locked data-collection tool, a pre-registered study protocol, a voice-driven questionnaire in Bangla, and an append-only database of all experiments.
9. A file-backed evaluation of every module, including negative results, corrections of earlier figures and a list of measurements that are still missing.

1.7 Thesis Structure

This thesis is organized into seven chapters, each addressing a specific aspect of the project. The chapters are outlined as follows:

- **Chapter 1: Introduction**
  This chapter provides the background, problem statement, objectives, scope, research questions, contributions and thesis structure.

- **Chapter 2: Literature Review**
  This chapter reviews assistive technologies for visually impaired people, existing reading devices, smart glasses, OCR and text-to-speech systems, object detection, Bangladeshi currency recognition, multi-view learning, evaluation leakage and safe prediction.

- **Chapter 3: System Design and Methodology**
  This chapter describes the framework, hardware components, software design and the methodology of SARA.

- **Chapter 4: Implementation Process**
  This chapter explains how each module was implemented, integrated and deployed.

- **Chapter 5: Dataset and Data Engineering**
  This chapter presents the datasets used, the synthetic compositing pipeline, the serial-number audit, the data splits and the OCR benchmark.

- **Chapter 6: Results and Discussion**
  This chapter reports the measured performance of each module on laptops and on the Raspberry Pi 5, the first field test, the negative results, and the limitations.

- **Chapter 7: Conclusion and Future Work**
  This chapter summarises the work, its social impact and the directions for future research.

2 Literature Review

2.1 Assistive Technology for the Visually Impaired

Assistive technology (AT) for blind and visually impaired (BVI) people aims to replace or supplement visual information with another channel, usually sound or touch. Traditional aids such as the white cane, guide dogs and Braille are reliable but provide limited information: a cane detects an obstacle but cannot say what it is, and Braille is available only for a small fraction of printed material. Over the last two decades, the combination of low-cost cameras, embedded processors and deep learning has made it possible to build devices that "see" on behalf of the user and describe the scene through speech [10, 11].

A recent bibliometric review of 533 articles on wearable mobility aids published between 2014 and 2024 shows a clear shift from single-sensor prototypes towards user-centred, multi-function systems that combine computer vision, audio and haptic feedback [11]. Reviews of image-to-voice systems describe a common two-stage pipeline: a vision model converts the image into text or labels, and a text-to-speech (TTS) engine converts that text into audio [12]. SARA follows the same general pipeline, but specialises each stage for Bangla and for offline use.

2.2 Historical Background and Evolution

The first electronic reading aid, the Optacon, did not recognise letters at all: it turned the image of each printed letter into a vibrating pattern that the reader felt with a fingertip [13]. Early reading machines then combined a scanner, a hand-crafted OCR engine and concatenative speech synthesis. Assistive reading systems of that period localised text with connected-component labelling and histogram analysis on binarised images and produced speech through recorded phoneme units [14, 15]. These systems worked on clean printed pages but not on natural scenes.

The next generation moved to portable hardware. FingerReader used a finger-worn camera to read printed text on the go [16], and product-label readers used motion to isolate a held object in front of a Pi camera [17]. With the arrival of the Raspberry Pi, many low-cost readers were built from a Pi board, a USB or Pi camera, Tesseract OCR and a speaker [6, 7, 8, 18, 19]. Most of these systems read English only and use Google Text-to-Speech, which requires an internet connection.

In parallel, smart glasses appeared. Student-GLASS built a camera glass with voice requests and auditory feedback on a microcontroller [20]. A Google Glass system used Microsoft Azure Custom Vision in the cloud for scene recognition and bone-conduction audio [21]. A deep learning smart glass published in *Electronics* combined low-light enhancement, object recognition and salient object detection for night-time assistance [22]. More recently, AIris placed a camera on eyewear and sent images to a remote server for face recognition, scene description and text reading [23]. The trend is towards richer functions, but also towards heavier dependence on remote computation.

2.3 Key Technologies in Smart Assistive Glasses

2.3.1 Embedded Computing Platforms

Raspberry Pi boards dominate low-cost assistive prototypes because they run a full Linux system, support Python, OpenCV and PyTorch, and provide GPIO pins for buttons and motors. Earlier projects used the Raspberry Pi 3 or 4 [6, 7, 24, 25]. The Raspberry Pi 5 used in this work has a quad-core Cortex-A76 CPU at 2.4 GHz, which is a large step in CPU performance over the Pi 4 [9]. Arduino-based glasses [26] are lighter and cheaper but cannot run deep neural networks.

2.3.2 Optical Character Recognition

OCR converts an image of text into machine-readable characters. Tesseract [27] is the most common engine in assistive readers [6, 8, 18, 28]. It works well on clean, horizontal, printed text but struggles with scene text and non-Latin scripts. Modern OCR has two stages. A *text detector* finds text regions: CRAFT predicts a character-region score and a link score between characters [29], EAST regresses word boxes directly [30], and DBNet learns the binarisation threshold together with the segmentation map [31]. A *recogniser* then reads each region; the CRNN combines a convolutional feature extractor with a recurrent network [32] and is trained with the connectionist temporal classification (CTC) loss [33], and TrOCR replaces both with an image and a text transformer [34]. EasyOCR combines a CRAFT detector with a CRNN recogniser and supports more than 80 languages including Bangla [35, 36]; PaddleOCR is a comparable toolkit [37]. Errors that remain after recognition can be corrected with a spelling dictionary [38], an n-gram language model [39] or a byte-level sequence model such as ByT5 [40]. Cloud OCR, for example Google Cloud Vision [41], gives higher accuracy but requires internet access and sends the user's images to a third party. Bangla OCR is a harder problem because of the large number of conjunct characters (*juktakkhor*), the connecting head line (*matra*) and vowel diacritics above and below the base letter. Deep learning classifiers such as Inception V3, VGG-16 and Vision Transformers have been applied to isolated handwritten Bangla characters [42], and datasets such as Ekush provide large, writer-labelled collections of Bangla characters [43]. Bangla also needs *text shaping* when it is drawn: a shaping engine such as HarfBuzz [44] reorders vowel signs and forms conjunct glyphs. Text rendered without shaping does not look like real print, which matters when synthetic images are used to test an OCR engine (Chapter 5).

2.3.3 Object Detection

Object detection locates and labels objects in an image. The YOLO family of single-stage detectors [45] is the most common choice in assistive devices because it runs in real time [46, 24, 25, 47, 48]. YOLOv8 from Ultralytics [49] is anchor-free, uses a CSPDarknet backbone with C2f blocks, a PAN-FPN neck and a decoupled head, and is provided in nano, small, medium and larger sizes. Pre-trained YOLOv8 models detect the 80 classes of the COCO dataset [50]. On a CPU-only board, the model is usually exported to ONNX and run with ONNX Runtime [51], and its weights are quantised to 8-bit integers to reduce size and time [52].

2.3.4 Speech Output and Feedback

Speech synthesis has progressed from concatenative synthesis to neural TTS [53]. Online engines such as gTTS give natural voices but need the internet. Offline engines include espeak-ng, a compact formant synthesiser that supports Bangla [54], and Piper, a fast local neural TTS sys-

tem that runs on a Raspberry Pi [55]. Bone-conduction headsets and vibration motors are used as additional channels so that the user can still hear the environment [21, 26].

2.4 Currency Recognition and Authentication

Currency recognition is an important task for visually impaired people because banknotes of different values often have similar size and texture. Classical systems identify the country and denomination with template matching, colour analysis in the LAB space and size measurements [56, 57]. For Bangladeshi Taka, an early neural network with axis-symmetrical masks reported 98.57% average accuracy over eight denominations [58], lightweight CNNs have been used for real-time recognition for visually impaired users [59], and a dual-stream MobileNet/EfficientNet model has been proposed for robust recognition [60]. These methods are whole-image classifiers: they assume that one note fills the image.

Detection-based approaches localise the note in a full scene. The NSTU-BDTAKA dataset provides 3,111 manually annotated images for Taka detection with YOLOv5, reaching a precision of 0.95 and a recall of 0.93 [61]. A YOLOv8-nano model with a squeeze-and-excitation head for US Dollar, Euro and Taka detection reported 0.972 mAP@0.5 and 0.804 mAP@0.5:0.95 [62]. Training on synthetic images is an established way to avoid manual labels: pasting cut-out objects onto real backgrounds works for instance detection [63] and as an augmentation for segmentation [64], and randomising the appearance of synthetic scenes transfers to real images [65, 66]. This motivates the compositing approach used in this thesis. Among assistive applications, BLIND ASSIST [67], Divya-Drishti [68] and VisioGuide [69] include a currency feature, usually for Indian Rupees and usually on a smartphone.

Counterfeit detection is much less studied in assistive systems. Bank equipment uses ultraviolet light, magnetic ink and infrared sensors, which are not available on a glass. With a visible-light camera, the security thread, hologram strip, watermark and micro-printing must be inspected from close views, and one view is often not enough. The JaalTaka dataset provides six close-up views of genuine and counterfeit 500 and 1,000 Taka notes [70]. This motivates the multi-view authenticator in this work.

2.5 Multi-View Learning with Missing Views

Multi-view recognition combines several images of one object. The multi-view CNN encodes each view with a shared network and pools the features [71]; it is usually evaluated on rendered views of ModelNet shapes [72] and, more recently, on real views of retail products in MVP-N [73]. Later models replace pooling with attention, in the style of vision transformers [74]. These models are normally trained and tested with the same number of views. In multi-modal learning the related problem of a missing input is handled by randomly dropping inputs during training, as in ModDrop [75]. For an assistive device the number of views is not under control: a blind user may capture one view or six. Chapter 6 shows that a model trained with a fixed number of views can lose more than 25 points with one view, and that training on random prefixes of the views removes the loss.

2.6 Evaluation Leakage and Shortcuts

A test score is meaningful only if the test data are independent of the training data. Leakage between the two is a common cause of results that do not reproduce [76], and deep networks readily learn a shortcut that separates the classes in one dataset without solving the task [77].

Banknotes offer such a shortcut. Counterfeits printed from one plate carry the same serial number, so a split made by physical note can still put copies of one print in both training and test data. Chapter 5 reports this for JaalTaka and defines a print-disjoint split.

2.7 Prediction with a Reject Option

A classifier that may abstain can trade coverage for accuracy. Chow gave the optimal rule when the class probabilities are known [78], and selective classification applies the idea to deep networks by answering only above a confidence threshold [79]; the confidence itself must be calibrated [80]. Conformal prediction gives a guarantee that needs no assumption about the model: if validation and test examples are exchangeable, a new score exceeds the highest of n validation scores with probability $1/(n+1)$ [81]. The verdict policy of the glass (Section 3.4.5) uses exactly this property.

2.8 Evaluating Assistive Devices with Users

Assistive devices are finally judged by their users. The System Usability Scale gives a ten-item usability score [82], and the NASA Task Load Index measures perceived workload [83]. Many of the prototypes reviewed in this chapter report neither, and give no task times or error counts with blind participants. This thesis prepares such a study but does not report one.

2.9 Real-World Applications

2.9.1 Reading Assistants

Reading assistants are the most common category. Raspberry Pi readers with Tesseract and gTTS [6, 7, 8], translators from English to regional languages [6], and deep learning readers that also describe images in printed documents [84] have been reported. Sinhala and Kannada readers [69] show that local-language support is a real need in South Asia.

2.9.2 Multi-Function Wearables

Multi-function devices combine several modules. The Third Eye clips a camera to spectacles and uses a smartphone for text, face and object recognition [85]. A Raspberry Pi 4 system with LiDAR and Camera Module 3 combines YOLOv8n detection, Tesseract OCR, face detection and distance measurement [24]. Other systems add ultrasonic obstacle sensing, GPS and GSM alerts [86, 87, 88]. EchoLight uses YOLOv5, OCR and TTS through a bone-conduction headset [46].

2.9.3 Mobile Applications

Smartphone apps such as BLIND ASSIST [67], IV Smart [48] and assistants based on NLP [47] are convenient but require the user to aim a phone and use a touch screen, which many blind users find difficult.

2.10 Summary of Reviewed Assistive Reading Systems

Before comparing technologies, it helps to see what the devices themselves look like. Table 2.10.0.1 puts five existing reading aids side by side, from the first electronic one to a recent research prototype, and ends with the device built in this thesis. Looking at them in order tells a short story. The Optacon let a blind person read ordinary print sixty years ago, but by touch, slowly, and only after long training [13]. Commercial products such as OrCam MyEye made the idea small enough to clip on a spectacle frame and replaced touch with speech [3]. Glass-type devices and research prototypes then added scene description and face recognition, usually by sending pictures to a server [21, 23]. What none of them offers is the combination a blind person in Bangladesh needs: Bangla, Taka, and no dependence on the internet.

Table 2.10.0.1: Summary of reviewed assistive reading systems for the visually impaired

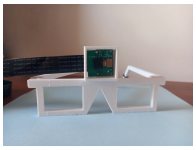
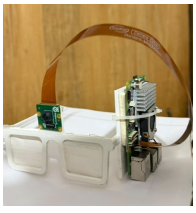
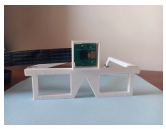
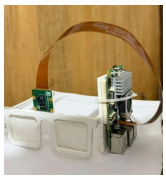
Device	System	How it works	What it gives the user	Missing features and limitations
	Optacon (1966) [13]	A small hand-held camera is moved along the line; the shape of each letter is reproduced on a pad of vibrating pins under one fingertip	Any printed text can be read without a sighted helper	Reads by touch only, no speech output; slow; needs months of training; no longer manufactured
	OrCam MyEye [3]	A camera and speaker clipped to the side of a spectacle frame; recognition runs inside the device	Reads printed text aloud; recognises faces, products and some banknotes	No Bangla language support; no Bangladeshi Taka (currency) detection; too expensive for most users in Bangladesh
	Google Glass scene analysis (2021) [21]	Camera of a Google Glass; pictures are analysed by a cloud vision service; bone-conduction audio	Spoken description of the scene in front of the user	Does not work without the internet; no Bangla language support; no Taka detection; costly hardware
	eSight eyewear [89]	Head-worn camera with two small screens in front of the eyes; magnifies and raises contrast	People with low vision can read and see faces with their remaining sight	Of no use to a totally blind user; does not read text aloud; expensive
	AIris (2024) [23]	Camera on a 3D-printed frame, a Raspberry Pi, and a remote server that runs the models	Text reading, scene description, face recognition, money counting by voice request	Needs a remote server and the internet; English only, no Bangla; no Taka detection
	**SARA (this work)**	Camera on a spectacle frame, Raspberry Pi 5, five buttons; all models except one optional mode run on the device	Bangla and English reading, objects, nine Taka notes, cautious counterfeit check, Bangla speech	Text reading is slow on the Pi (about 19 s per capture); not yet tested through the glass camera or with users

Image credits: Optacon, by DrBliss, public domain; OrCam MyEye, by Ylip, CC BY-SA 4.0; Google Glass Explorer Edition, by Ted Eytan, CC BY-SA 2.0; eSight 4, by eSight Digital, CC BY-SA 4.0 (all four from Wikimedia Commons). AIris prototype reproduced from [23]. The Google Glass photograph shows the hardware used in [21], not that study's own set-up.

2.11 Comparative Evaluation of Assistive Vision Systems

Table 2.11.0.1 compares representative systems from the literature with the system developed in this work. The comparison shows that most systems have strong individual capabilities; however, few combine offline operation, local-language support and local currency handling in one wearable design.

Table 2.11.0.1: Overview of assistive vision systems for visually impaired users, with key technologies, strengths and limitations

Picture	System	What it is built with	What it does well	What it cannot do (limitations)
–	Deep-learning smart glass [22]	A neural network that brightens dark pictures, and a transformer model that recognises objects	Recognises objects even at night; describes the scene in detail	Large models need strong hardware; English labels only; no currency detection
	Google Glass scene analysis [21]	Google Glass camera; pictures are analysed by a Microsoft Azure cloud service; bone-conduction sound	Uses ready-made commercial glasses; answers in real time	Does not work without the internet; no Bangla; the glasses are expensive
	AIris [23]	Camera on an eyeglass frame, a Raspberry Pi, and a remote server that runs the models	Reads text, recognises faces and describes the scene	The models run on a distant server, so it needs the internet; English only
–	Raspberry Pi 4 blind assistant [24]	YOLOv8n object detector, Tesseract text reader, LiDAR distance sensor, simple face detector	Tells how far away objects are; has several functions in one device	Reads English text only; no currency detection
–	Vision Voice [7]	Raspberry Pi 3, Tesseract text reader, Google online speech (gTTS)	Cheap and simple to build	Only reads text; speech needs the internet
–	BLIND ASSIST [67]	Android phone app with a small on-phone currency model and a text reader	Text reading and currency recognition in one app	The user must hold and aim a phone and use a touch screen
–	NSTU-BDTAKA [61]	YOLOv5 detector trained on 3,111 hand-labelled photos	Detects Bangladeshi Taka notes	Every photo had to be labelled by hand; only a computer experiment, not a device for blind users
	**SARA (this work)**	Camera on a spectacle frame, Raspberry Pi 5, five buttons; EasyOCR, YOLOv8, offline Bangla speech	Works offline (except one optional mode); speaks Bangla; names all 9 Taka notes; its counterfeit check never wrongly accuses	Text reading is its slowest mode on the Pi; user trials are under way

"–": no picture of the device is available. Picture credits as in Table 2.10.0.1.

Table 2.11.0.2 focuses on currency recognition. Direct numerical comparison should be read with care because each study uses a different dataset, class set and test protocol.

Table 2.11.0.2: Comparison of currency recognition methods related to Bangladeshi Taka

Work	**What it does**	**How the training images were labelled**	**mAP@ 0.5[a]**	**mAP@ 0.5:0.95[b]**	**Precision / Recall, or Accuracy[c]**
Axis-symmetrical masks [58]	Tells which Taka note is in the picture (one note per picture; does not find its position)	One label for the whole picture (no boxes)	–	–	Accuracy 98.6%
YOLOv5 Taka [61]	Finds Taka notes in the picture and names each one	3,111 images, boxes drawn by hand	–	–	0.95 / 0.93
YOLOv8n-SE [62]	Finds and names notes of several countries' currencies	Boxes drawn by hand	0.972	0.804	0.965 / 0.952
This work (YOLOv8s)	**Finds and names all 9 types of Bangladeshi currency notes (BDT: 2, 5, 10, 20, 50, 100, 200, 500 and 1,000 Taka)**	**22,333 images, labelled automatically (no hand labelling)**	**0.995***	**0.849***	**0.995 / 0.996***

[a] mAP@0.5: how often a note is found and named correctly, counting a box as correct when it overlaps the true note by at least half (1.0 = perfect). [b] mAP@0.5:0.95: the same, but also rewarding boxes that fit the note tightly; a stricter score. [c] Precision: of the notes the model reported, the share that were right. Recall: of the notes in the pictures, the share the model found. "–": not reported by that study.

*Measured on held-out synthetic pictures made the same way as the training data; see Chapter 6 for the results on independent real photographs, which are lower. Each study used its own data, so the numbers are not directly comparable.

2.12 Key Research Areas Relevant to SARA

Recent work on assistive devices relevant to this thesis falls into four areas, summarised in Table 2.12.0.1.

Text Reading Systems use OCR and TTS to read printed material. They are mature for English but weak for Bangla and depend on online speech.

Smart Glasses and Wearables integrate cameras, processors and audio on the body. They increasingly rely on cloud processing for advanced functions.

Object and Scene Recognition relies on YOLO-family detectors and captioning models, usually with English labels.

Currency Recognition and Authentication covers denomination classification and detection; authentication with a visible-light camera is still largely unexplored in assistive devices.

Table 2.12.0.1: Key research categories in assistive vision technology

Research area	**What the works in this area do**	**What is still missing for a blind user in Bangladesh**	**Studies**
Text reading systems	Read printed text aloud: finger-worn and Raspberry Pi readers using the Tesseract or EasyOCR text readers; some describe pictures in documents or translate into regional languages	Mostly English; weak or no Bangla reading; often need an online speech service	[16, 14, 6, 7, 8, 18, 84]
Smart glasses and wearables	Put the camera, computer and sound on the body: camera glasses with voice output, scene description in the cloud, light Arduino glasses that vibrate, glasses that send emergency alerts	Advanced functions increasingly depend on a cloud server and the internet	[20, 22, 21, 23, 85, 26, 87]
Object and scene recognition	Tell the user what is around them: YOLO object detectors (v4, v5, v8), distance with LiDAR or ultrasonic sensors, face detection, GPS navigation	Object names are given in English	[46, 24, 25, 48, 86, 47]
Currency recognition	Tell which note it is: template and colour matching, neural-network classifiers, YOLO detection of Taka and other currencies, currency features in phone apps	Checking whether a note is genuine with an ordinary camera is almost unexplored in assistive devices	[56, 57, 58, 61, 62, 67, 68]

2.13 Research Gap

The review leaves a clear picture. The pieces of an assistive glass exist: cameras are cheap, object detectors run in real time, OCR engines read many languages, and small computers can carry all of it. What is missing is not another piece. It is the fit between those pieces and one particular user, a blind person in Bangladesh, and an honest account of how well the result works. We group what is missing into three kinds of gap.

Gap 1: No existing device fits the user

Every system in Tables 2.10.0.1 and 2.11.0.1 fails this user on at least one point. The commercial glasses cost far more than most users here can pay and do not speak Bangla or know Taka. The low-cost Raspberry Pi readers read English only and speak through an online service, so they fall silent when the connection drops. The recent research glasses send pictures to a server. Work on Bangladeshi Taka exists, but as computer-vision experiments on a desktop, not inside something a person can wear. Nobody has put Bangla reading, Taka recognition and Bangla speech together in one device that works without the internet.

Gap 2: Three method problems are unsolved

- **A blind user cannot promise how many pictures the device gets.** Multi-view models are trained and tested with a fixed number of views. What happens to a counterfeit check when the user manages only one view instead of six has not been studied.
- **A blind user cannot check the answer.** A sighted person who is told "this note is counterfeit" can look again. A blind person cannot, and may accuse someone wrongly. How a device should word an uncertain verdict for someone who cannot verify it has not been addressed.
- **Taka detectors depend on hand-labelled data.** The published detectors were trained on a few thousand manually annotated images. Whether a detector can be trained without any manual labels and still work on other people's photographs is open.

Gap 3: The evidence is weaker than it looks

- **Counterfeit results may measure memory, not detection.** Counterfeit datasets are split by image or by note. No study checks whether the test counterfeits are copies of prints that were also in the training data, so a high score may only show that the model recognises reprints it has already seen.
- **Modules are rarely measured one by one, and almost never on the target hardware.** Many prototypes report that the system "works" without an error rate for each function, without timings on the device, and without saying what failed.
- **Users are missing from the evaluation.** Few prototypes report task times, errors or usability scores with blind participants.

Table 2.13.0.1 lists each gap, where it shows up in existing devices and research, what SARA does about it, and how far this thesis gets. The last column shows how far each gap is addressed in this thesis.

Table 2.13.0.1: Research gaps and how this thesis responds to them

Area	**Research gap (what is missing)**	**In existing devices and research**	**What SARA does**	**Status in this thesis**
Device	No affordable wearable that speaks Bangla and recognises Taka without the internet	Commercial devices are English-only and costly; research glasses send pictures to a server	A glass that speaks Bangla, on a Raspberry Pi 5; four of five modes run on the device	Solved as a prototype
Method	Counterfeit checks assume all six views of a note are captured	Multi-view models are trained and tested with a fixed number of views	Training on random prefixes of views keeps accuracy from one to six views (Section 3.5)	Solved
Safety	No safe way to give an uncertain counterfeit result to a user who cannot check it	Not addressed in the reviewed devices	Never says “counterfeit”; says “likely genuine” only above a safe threshold, otherwise “check by hand” (Section 3.4.5)	Addressed: no counterfeit passed
Data	Taka detectors need thousands of hand-labelled images	NSTU-BDTAKA: 3,111 manually labelled images	22,333 training images labelled automatically (Chapter 5)	Addressed for full-note photos
Evaluation	Counterfeit tests mix copies of the same print in training and test	Datasets are split by image or by note; prints are not checked	Serial-number audit and a test split with unseen prints (Section 5.3.1)	Addressed
Evaluation	Prototypes rarely report the accuracy of each feature or the speed on the real device	“It works” without numbers	Every feature measured on held-out data; speed measured on the Pi 5; failures reported	Done
Evaluation	Few tests with blind users	Rare in prototypes	Study protocol, logging and a Bangla voice questionnaire prepared; trials started	In progress

3 System Design and Methodology

3.1 Framework of SARA

The framework of SARA follows an input–process–output structure (Figure 2). The user gives commands only through physical buttons, the camera provides the visual input, the on-board models interpret the scene according to the active mode, and the result is returned through speech and vibration.

Input

- **Camera frames:** 640×480 colour frames from a Raspberry Pi Camera Module 2 (Sony IMX219, fixed focus) read through the Picamera2 library [90], or from a USB webcam.
- **Button events:** five push-buttons for MODE, ACTION, READ, VOLUME UP and VOLUME DOWN, read through GPIO interrupts.

Process

- **Mode manager:** keeps the active mode (text, object, currency, online description, emotion) and routes frames and button events to it.
- **Vision models:** EasyOCR for Bangla/English text, YOLOv8 for COCO objects, a YOLOv8s Taka detector, a multi-view authenticator and a watermark classifier for genuine/counterfeit notes, and an EfficientNet-V2-S expression recogniser.
- **Language processing:** Bangla script detection, mixed-script segmentation and translation of labels to Bangla.

Output

- **Speech:** Piper neural TTS or espeak-ng through the ALSA audio system, in Bangla or English.
- **Vibration:** a short pulse from a vibration motor that confirms a detected note.
- **Field log:** one line per event (button, mode, result, time taken) and periodic system readings in two CSV files.

3.2 Overview of SARA System Design

SARA is designed as a pair of spectacles with a small camera at the bridge and a processing unit worn at the waist or in a pocket. The design follows five principles derived from the problems described in Chapter 1:

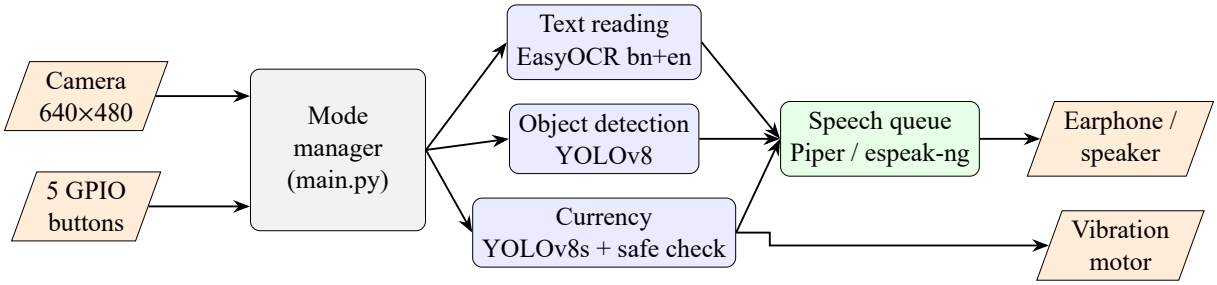


Figure 2: Input–process–output framework of SARA. The emotion mode and the optional online mode use the same path and are omitted for clarity.

1. **Offline first:** every core function runs locally. The only online feature is an optional scene-description mode that the user must choose on purpose.
2. **Bangla first:** mode names, object names and currency names are spoken in Bangla; text is read in the language in which it is written.
3. **Buttons, not screens:** the device has no display. Every function is reachable with five buttons that can be found by touch.
4. **On-demand heavy work:** slow tasks (OCR, currency) run only when the user presses ACTION; only the object mode scans continuously.
5. **Safe before decisive:** when the device is not sure, it says so. It never tells the user that a note is counterfeit.

3.2.1 General Overview of SARA

When the device boots it announces that the smart glass is starting and then speaks the name of the active mode. The user cycles through the modes with the MODE button. In *text reading mode*, ACTION captures and recognises text, speaks it once and stores it, and READ speaks the stored text again. In *object mode*, the glass scans the scene every 1.5 seconds and announces up to three objects, for example “in front: chair, person”. In *currency mode*, ACTION detects the note, speaks its value in Bangla and gives a vibration pattern; for a 500 or 1,000 Taka note it then gives the result of the safe check, and it can ask the user to hold the note up to a light for the watermark check. In *emotion mode*, ACTION announces the facial expression of the nearest face. The *online mode* describes the scene when the internet and an API key are available. VOLUME UP and VOLUME DOWN change the volume in 10% steps. The name of a mode is spoken immediately after MODE is pressed, while its model loads in the background.

3.2.2 Structural Characteristics of SARA

The design places only the camera on the spectacle frame and carries the Raspberry Pi 5, the push-button pad, the vibration motor and the battery in a small enclosure at the waist or in a pocket. Separating the processing unit from the frame keeps the glasses light and moves the heat of the processor away from the face.

The first prototype (Figure 3) is simpler than this design. It is a 3D-printed white spectacle frame with the camera module fixed above the bridge, so that the camera looks where the wearer’s face points. The Raspberry Pi 5, with its heat sink and fan, is tied to the right temple of the frame, and a flat ribbon cable loops from the camera to the board. This was the quickest way to get a working device on a person’s head, and it is enough for testing the software. It also shows why the design calls for a separate enclosure: with the board on one temple the frame is heavy on one side, the fan sits next to the ear, and the ribbon cable stands above the head.

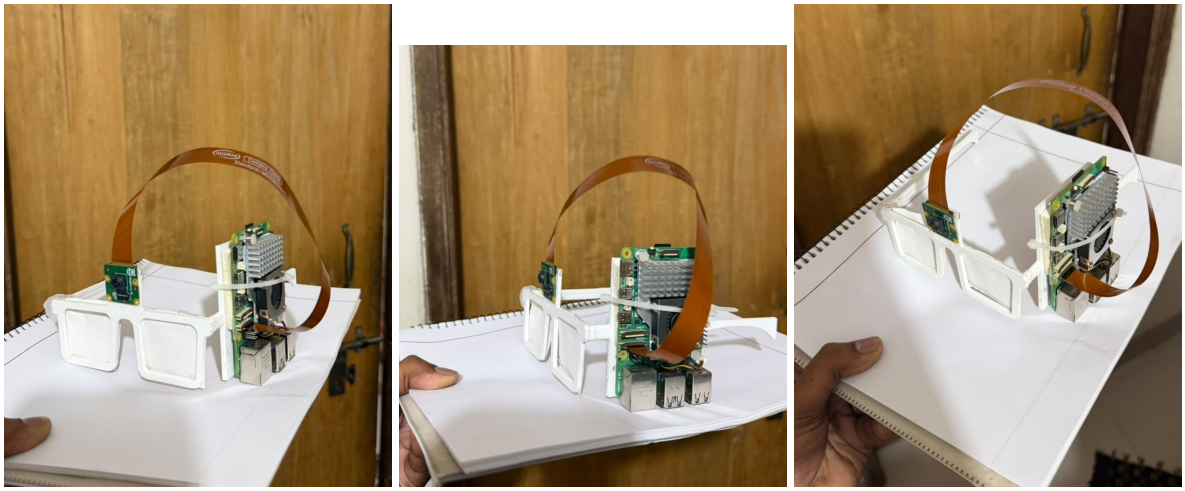


Figure 3: First prototype of SARA: the camera module above the bridge of a 3D-printed frame (left), the Raspberry Pi 5 with heat sink and fan tied to the right temple (centre), and the ribbon cable between camera and board (right)

3.2.3 Interactive Features and Interface Design

Several interaction decisions were made specifically for users without sight:

- **Capture is separated from reading.** The user holds the camera still while pressing ACTION, and can then relax and listen by pressing READ. The text can be replayed as often as needed.
- **Repetition control.** In object mode, each object class has a 4-second cooldown so the glass does not repeat "chair, chair, chair".
- **Speech interruption.** Switching mode clears the speech queue immediately so the user does not wait for old messages.
- **Mixed-script speech.** A sentence such as "Bus 96 Metrocentre" inside Bangla text is split, and each part is sent to the correct voice.
- **Vibration confirmation.** A vibration confirms that a note was detected, even in a noisy market.
- **Guided capture.** For the watermark check the glass tells the user what is wrong with the picture ("too dark", "hold still") instead of silently failing.
- **Cautious wording.** The counterfeit check has two spoken outcomes, "likely genuine" and "check by hand". The word "counterfeit" is never spoken.

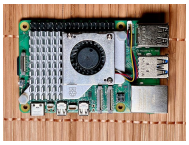
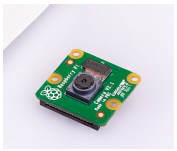
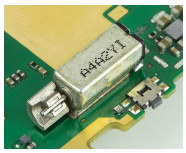
3.3 Hardware Components

Table 3.3.0.1 summarises the hardware of the prototype. Each component is described below.

Processing Unit

The Raspberry Pi 5 is the brain of the system. It runs Raspberry Pi OS (64-bit), Python 3, OpenCV, PyTorch and ONNX Runtime. All inference runs on the CPU. The board also provides the GPIO pins for the buttons and the vibration motor, and the audio output.

Table 3.3.0.1: Hardware components of SARA

Picture	Component	Details	What it does in SARA
	Raspberry Pi 5 with active cooler	Small computer with a four-core 2.4 GHz processor and 4 or 8 GB of memory; no graphics accelerator is used	The “brain”: runs all the AI models and the program
	Camera Module 2	8-megapixel Sony IMX219 sensor with a fixed-focus lens; used at 640×480 pixels	Takes the pictures of text, notes and surroundings
	Push-buttons (5)	Press-to-click buttons connected to Pi pins 17, 27, 24, 22 and 23	Let the user control the glass without a screen: MODE, ACTION, READ, VOLUME UP, VOLUME DOWN
	Vibration motor	Small motor that shakes when switched on; driven from Pi pin 13	A short buzz tells the user something was found (for example a note)
	Earphone / speaker	Wired 3.5 mm or USB earphone, or a small speaker	Speaks every result to the user
	Power bank	USB-C power bank (the official Pi 5 supply is rated 5 V, 5 A)	Lets the user carry the glass anywhere
	Spectacle frame	3D-printed frame with the camera above the nose bridge	Holds the camera where the user looks, so the glass is worn like ordinary spectacles

Picture credits (Wikimedia Commons): Raspberry Pi 5 with active cooler, by SimonWaldherr, CC BY 4.0; Camera Module v2, by the Raspberry Pi Foundation, CC BY-SA 4.0; tactile push-buttons and vibration motor, by Raimond Spekking, CC BY-SA 4.0; earphones, by Berthold Werner, CC BY-SA 3.0; USB power bank, by Pinguinguy, CC0. Spectacle frame: our first prototype. The pictures show the type of part, not necessarily the exact model used.

Camera

The Camera Module 2 (IMX219) is connected through the CSI ribbon cable and read with the Picamera2 library; the OpenCV video interface receives no frames from this camera on the Raspberry Pi 5. A USB webcam can be used instead: the camera manager tries Picamera2 first, then OpenCV, and the choice can be forced with one configuration value. The resolution of 640×480 was chosen as a balance between detail and processing load. The lens has a fixed focus, so close text and notes are not sharp at every distance. The first field test (Section 6.10) suggests that this resolution and lens may be too weak for reading; the resolution can be raised with a configuration value.

Buttons

Five push-buttons connect GPIO pins to ground with internal pull-up resistors. A 250 ms software debounce removes false double presses. The button map is given in Table 3.3.0.2.

Table 3.3.0.2: Button map of SARA

Pi pin	Button	What happens when it is pressed
17	MODE	Moves to the next mode, in the order text → object → currency → online → emotion, and says the mode's name
27	ACTION	Does the main job of the current mode: text mode reads the text in view; object mode says the objects in view now; currency mode names the note; online mode describes the scene; emotion mode names the facial expression
24	READ	Repeats the last text that was read
22	VOL_UP	Makes the speech 10% louder
23	VOL_DOWN	Makes the speech 10% quieter

Audio Output

Audio is played with the ALSA tools `aplay` and `amixer`. The default volume is 80%. An earphone keeps the information private; a small speaker can be used at home.

Vibration Motor

A small vibration motor, driven from a PWM pin (BCM 13), gives one short pulse (50 ms) when a note is detected. Separate patterns for a genuine and a counterfeit verdict exist in the code but are not used under the safe policy of Section 3.4.5. The motor driver was tested with simulated GPIO on the laptop.

Power

The system is powered by a USB-C power bank. Battery life has not been measured, because the Raspberry Pi 5 has no battery gauge and a USB-C power meter was not available; it is part of future work.

3.4 Software Design

The software is written in Python and organised as a small core with plug-in modes (Table 3.4.0.1). Every mode implements the same interface (`activate`, `deactivate`, `process_frame`, `cleanup`), which allows modes to be added without changing the core.

Table 3.4.0.1: Software modules of SARA

Program file	**What it does**
`main.py` (SmartGlass)	Starts and stops the glass, switches modes, passes each button press to the current mode, and runs the object-detection loop
`config.py`	All settings in one place: button pins, camera, model files, decision thresholds, speech and logging
`button_handler.py`	Notices button presses and ignores accidental double presses
`utils.py`	Shared helpers: logging, telling Bangla from English, the speech engine and the camera
`modes/ocr_mode.py`	Text mode: reads Bangla and English text and keeps it for the READ button
`modes/object_mode.py`	Object mode: finds everyday objects with YOLOv8 and names them in Bangla, without repeating itself too often
`ocr_text_region.py`	Picks the main block of text, puts the lines in reading order and cleans up the result
`modes/currency_mode.py`	Currency mode: finds and names Taka notes with YOLOv8s, runs the safe counterfeit check, and buzzes the motor
`modes/watermark_check.py`	Finds the watermark area on a note held against light and checks it
`modes/capture_guide.py`	Checks that a picture of a note held against light is bright and sharp enough, and tells the user how to hold it
`modes/emotion_mode.py`	Emotion mode: names the facial expression of the nearest person
`modes/claude_mode.py`	Online mode (optional): describes the scene using an online AI service; needs the internet and an API key
`field_log.py`	Records what happened and how the Pi was doing (for field tests) in a CSV file
`preview.py`, `live_diag.py`	Test window for the laptop: keyboard keys act as buttons; shows boxes and the outputs of all models
`voice_study.py`	Spoken questionnaire for the user study, answered by voice
`roboeye/` package	Counterfeit checker, emotion recogniser, vibration and speaker drivers, and heat-map pictures of what the models look at (Grad-CAM)
`scripts/deploy_pi5.sh`, `smart_glass.service`	Installs everything on the Pi and starts the glass automatically when the Pi is switched on

3.4.1 Text Reading Module

The text module uses EasyOCR [35]: a CRAFT text detector [29] and a CRNN recogniser with CTC decoding [32, 33], with the Bangla and English recognisers loaded together. The pipeline

used in the glass (called *v2* in the code) was fixed by the experiments of Section 6.7 and has five steps:

1. **Pre-processing.** The frame is converted to grey scale and its contrast is enhanced with CLAHE [91]. Binarisation is not used; it increased the error rate.
2. **Detection and recognition.** EasyOCR reads the whole frame once, with detector settings chosen on validation data (text threshold 0.8, low-text threshold 0.3, link threshold 0.4, greedy decoding).
3. **Confidence filter.** Boxes with a confidence below 0.1 are dropped. The value was 0.2, chosen on rendered text, until a test on real photographs showed that too many words were then not read at all (Section 6.7.3).
4. **Main text block.** Starting from the most confident tall box, boxes of similar height that touch it are added. This block is the sign or label the user faces; stray text in the background is dropped.
5. **Line ordering and clean-up.** The boxes are grouped into lines and read from top to bottom and left to right. The text is normalised to Unicode NFC, invisible characters and stray symbols at word edges are removed, and digits are put in the script of the words around them.

The main-text-block step reuses the boxes that EasyOCR already returns, so it costs no extra time. If EasyOCR cannot be loaded, the mode falls back to Tesseract 5 [27], and for single handwritten letters to a small CNN trained on Ekush [43]. Algorithm 3.1 gives the steps.

Whole pages are read line by line. When the main text block holds at least four lines, the text detector runs once, the lines are put in reading order, and each line is recognised and handed to the speech queue as soon as it is read. The user hears the first line while the rest of the page is still being read, instead of waiting for the whole page. Signs and labels with fewer lines take the path above.

Algorithm 3.1 Text reading with the main text block (pipeline v2)

Input: camera frame F; minimum confidence $c_{\min} = 0.1$; height ratio $r = 0.6$; gap factor $g = 1.5$
Output: text to speak

$G \leftarrow \text{CLAHE}(\textsc{Grey}(F))$ ▷ contrast boost only
$B \leftarrow \{\, b \in \textsc{EasyOCR}(G) : \text{conf}(b) \geq c_{\min} \,\}$ ▷ boxes with text and confidence
if $B = \emptyset$ **then**
    return "no text found"
end if
$s \leftarrow \arg\max_{b \in B}\ h(b) \cdot \text{conf}(b)$ ▷ tall, confident seed box
$K \leftarrow \{s\}, \quad B \leftarrow B \setminus \{s\}$
repeat ▷ grow the block
    $\bar{h} \leftarrow \text{median}_{k \in K}\, h(k)$
    for all $b \in B$ **do**
        if $r \leq h(b)/\bar{h} \leq 1/r$ **and** b lies within $g\,\bar{h}$ of some $k \in K$ **then**
            $K \leftarrow K \cup \{b\}, \quad B \leftarrow B \setminus \{b\}$
        end if
    end for
until no box was added
group K into lines by vertical centre (within half a text height); sort lines top to bottom and boxes left to right
return $\textsc{CleanUp}$(joined line text) ▷ Unicode NFC, stray symbols, digit script

3.4.2 Object Detection Module

The object module runs a COCO-pretrained YOLOv8 model (small variant, with the nano variant as fall-back; the nano variant was the one timed on the Raspberry Pi 5) on the camera frame every 1.5 seconds with a confidence threshold of 0.50. An earlier version first shrank the frame to 320×240; the detector enlarges every picture to its 640-pixel input anyway, so this saved no time and cost detail (Section 6.8). The three most confident unique classes are translated to Bangla with a lookup table of the 80 COCO classes and announced, subject to a per-class cooldown of 4 seconds.

3.4.3 Currency Detection Module

The currency module is a cascade that degrades gracefully when models are missing:

1. **Stage 0 – YOLOv8s Taka detector:** the detector trained in this work (Chapter 5) localises and classifies the nine denominations. On the Raspberry Pi 5 the INT8 ONNX export is used; on a laptop the PyTorch weights. A denomination is announced only when the detector's confidence is at least 0.60. This value was chosen on validation data so that notes of an unknown class, such as the 1 Taka note, are named less often. When two or more notes pass this confidence, the glass counts them, names them from the largest and speaks the total; a box that overlaps a surer one by more than half is taken as the same note seen twice.
2. **Stage 1 – HSV colour profile:** if no detector is available, the largest note-like contour (aspect ratio 1.5–2.8, area ≥ 8,000 pixels) is found and its colour histogram is compared with the dominant colour of each denomination.
3. **Stage 2 – MobileNetV3-Small classifier:** if trained weights exist, the note crop is classified and the result overrides Stage 1 when its softmax score is at least 0.65.

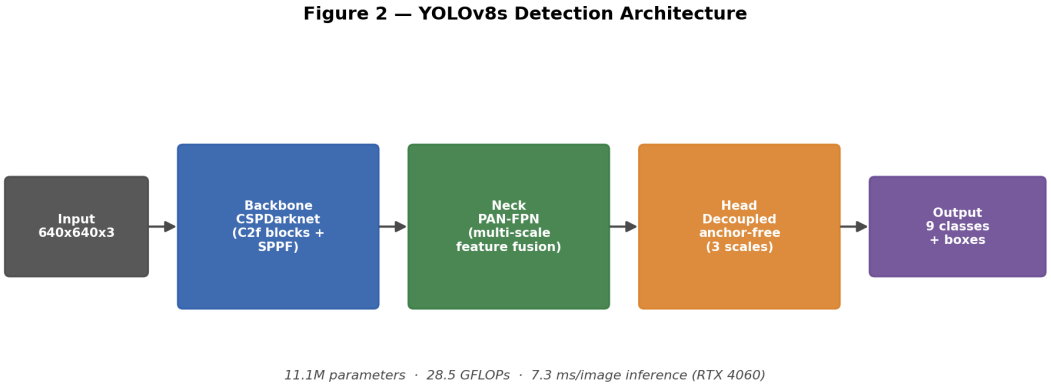


Figure 4: YOLOv8s detection architecture used for Taka detection: CSPDarknet backbone, PAN-FPN neck and decoupled anchor-free head predicting 9 classes and box coordinates

3.4.4 Currency Authentication Module

The authenticator decides whether a note is genuine or counterfeit from up to six close-up views (for example the portrait, the security thread, the hologram strip and the serial number). Each view is resized to 128×128 pixels, normalised, and encoded by a frozen MobileNetV3-Small [92] feature extractor followed by a TinyViT [93] block. The per-view features are fused by a quality-, diversity-, uncertainty- and information-gain-aware fusion head (called Q-DUIG in

this project), which weights sharp, informative and non-redundant views more strongly. The fused feature produces one authenticity logit, whose sigmoid gives the probability p that the note is genuine. In the experiments p is thresholded at 0.5 (Figure 5); the glass uses the stricter policy of Section 3.4.5.

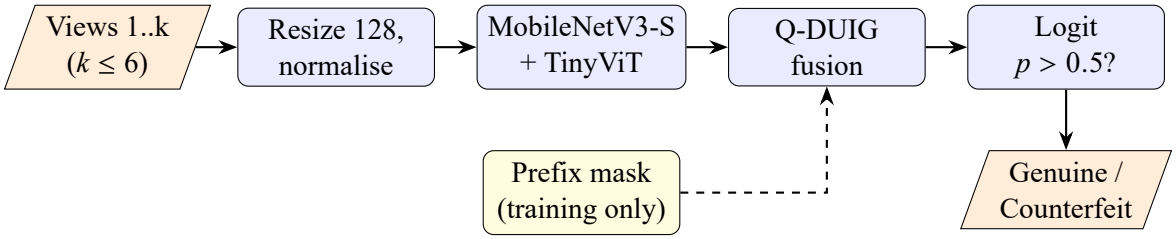


Figure 5: Multi-view currency authenticator. During prefix-robust training, a random prefix of 1 to 6 views is kept so the model learns to decide with any number of views.

3.4.5 Safe Verdict Policy on the Glass

The authenticator was trained on close-up views, but the glass sees a whole note. To give the model an input of the shape it knows, the note box found by the Taka detector is resized to 640 pixels in width and turned to landscape, and four windows are cut from it at the positions where JaalTaka views 1 to 4 lie on a note. The positions were measured on 200 training notes by registering each view to a whole-note template with SIFT features [94] and RANSAC [95]. The four windows are scored by the authenticator, which returns p.

The spoken verdict follows a rule fixed before any test score was read (called policy E in the project):

- if $p > \tau$, the glass says *shombhoboto ashol* (“likely genuine”);
- otherwise it says *ashol kina hate jachai korun* (“check by hand whether it is genuine”);
- it never says *jaal* (“counterfeit”).

The threshold $\tau = 0.99959$ is the highest p that any counterfeit note of the validation split received. This is selective prediction [78, 79] with a guarantee from conformal prediction [81]: if a new counterfeit is exchangeable with the n validation counterfeits, the probability that its score is above all of theirs, and so above τ, is

$$P(p_{\text{new}} > \tau) = \frac{1}{n+1}, \tag{3.1}$$

which is $1/89 = 1.1\%$ for $n = 88$. The guarantee holds only for notes that resemble the validation notes; a change of camera or of counterfeit print weakens it by at most the total-variation distance between the two distributions. The policy applies to 500 and 1,000 Taka notes, the only denominations in JaalTaka; for other notes the glass says that the check was not done. An image-quality gate rejects dark, blurred or covered frames before any verdict is given. Algorithm 3.2 gives the complete currency answer.

3.4.6 Watermark Check with Guided Capture

A genuine note has a watermark that becomes visible when the note is held against a light. The sixth JaalTaka view is such a back-lit photograph. The watermark check has three parts (Figure 6).

Capture guide. After the denomination is spoken, the glass asks the user to hold the note up to a light. Every frame is tested for brightness (mean grey level at least 73.9) and sharpness

Algorithm 3.2 Currency answer with the safe verdict policy

Input: frame F; announce threshold 0.60; safe threshold $\tau = 0.99959$
Output: spoken sentence

$(d, c, \text{box}) \leftarrow$ most confident note found by the YOLOv8s Taka detector in F
if no note **then**
    return "No note detected. Hold it in front of the camera."
end if
if $c < 0.60$ **then**
    return "Could not confirm the note. Hold it closer."
end if
$S \leftarrow$ name of note d in Bangla
if $d \notin \{500, 1000\}$ **then**
    return S + "the counterfeit check was not done"
end if
$N \leftarrow$ note crop resized to width 640, turned to landscape
$V_{1..4} \leftarrow$ windows of N at the positions of JaalTaka views 1–4 (measured on training notes)
$p \leftarrow \textsc{Authenticator}(V_{1..4})$ ▷ probability that the note is genuine
if $p > \tau$ **then**
    return S + "likely genuine"
else
    return S + "check by hand whether it is genuine" ▷ the word "counterfeit" is never spoken
end if

(variance of the Laplacian at least 106.7, a standard focus measure [96]). The two limits are the 2nd percentile of the validation back-lit photographs, so about 96% of acceptable photographs pass. A rejected frame produces a spoken reason, at most every 2.5 seconds, and the guide gives up after 10 seconds.

Watermark window. The first version registered the photograph to a note template with SIFT and RANSAC and cut a fixed box. The current version is a MobileNetV3-Small [92] that regresses the four corners of the window directly from a 320×320 copy of the photograph. It needs no template and works on photographs where registration fails. Its training labels are the registration boxes mapped back into each photograph, so it learned where that box is; it did not discover the window by itself. A quadrilateral that is not convex or has an implausible area is rejected and the user is asked to hold the note straight. Registration remains as a fall-back.

Watermark classifier. The window is classified as "watermark present" or not by a MobileNetV2 [97], quantised to 8-bit integers (2.6 MB). In the research experiments its score is combined with the authenticator's score by a logistic regression fitted on the validation split (the *hybrid*):

$$p_{\text{hyb}} = \sigma\big(\beta_0 + \beta_1\,\ell_{\text{net}} + \beta_2\,\ell_{\text{wm}} + \beta_3\,m\big), \qquad \ell = \log\frac{p}{1-p}, \tag{3.2}$$

where ℓ_{net} and ℓ_{wm} are the log-odds of the multi-view network and of the watermark classifier, $m = 1$ when no watermark window could be found (then $p_{\text{wm}} = 0.5$), σ is the logistic function, and the weights β are fitted on validation notes only. A note is called genuine when $p_{\text{hyb}} \geq 0.5$.

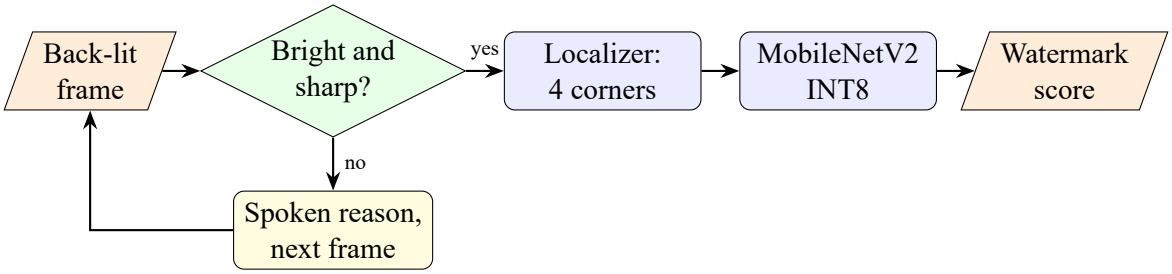


Figure 6: Watermark check with guided capture

Algorithm 3.3 gives the guided capture step by step.

Algorithm 3.3 Guided back-lit capture and watermark check

Input: note value $d \in \{500, 1000\}$; brightness limit $\mu_{\min} = 73.87$; sharpness limit $\lambda_{\min} = 106.75$; time limit $T = 10$ s; prompt gap 2.5 s
Output: spoken watermark answer

say "Hold the note up to the light"
while elapsed time $< T$ **do**
  $F \leftarrow$ latest frame; $C \leftarrow$ note crop from the Taka detector
  if mean$(C) < \mu_{\min}$ **then**
    $e \leftarrow$ "Hold the note closer to the light"
  else if Var$(\nabla^2 C) < \lambda_{\min}$ **then**
    $e \leftarrow$ "Keep the note still"
  else
    $Q \leftarrow$ Localizer(C) ▷ four corners of the watermark window
    if Q is convex and of plausible area **then**
      $w \leftarrow$ WatermarkClassifier(Warp(C, Q))
      if $w > 0.5$ **then**
        return "The watermark is clear"
      else
        return "The watermark is not clear. Check by hand whether it is genuine."
      end if
    end if
    $e \leftarrow$ "Hold the note straight in front of the light"
  end if
  say e if 2.5 s have passed since the last prompt
end while
return "The picture was not clear. Check by hand whether it is genuine."

3.4.7 Denomination-Matched Watermark Residual (DMWR)

The MobileNetV2 above learns the watermark from data, but it does not say *what* it compares. We therefore designed a second watermark classifier from the physics of a back-lit banknote. This is one of the new methods of this thesis.

Image model. When a note is held against a lamp, the camera records the light that passes through the paper. With $L(x, y)$ the brightness of the lamp behind the note and $t(x, y)$ the transmittance of the paper, the recorded grey image is approximately

$$I(x, y) \approx L(x, y)\, t(x, y). \tag{3.3}$$

The watermark is made by changing the thickness of the paper, so it lives in t; the lamp L changes slowly across the note and differs from one photograph to the next.

Step 1: remove the lamp (quotient image). The crop of the watermark window is resized to 128×128 pixels and divided by a strongly blurred copy of itself. Because L is smooth, the blur keeps L and the division cancels it:

$$\tilde{Q} = \frac{I}{G_\sigma * I} \approx \frac{t}{G_\sigma * t}, \qquad Q = \frac{\tilde{Q} - \operatorname{mean}(\tilde{Q})}{\operatorname{std}(\tilde{Q})}, \tag{3.4}$$

where $G_\sigma *$ is a Gaussian blur with $\sigma = 16$ pixels. Q is a picture of the paper thickness alone, the same under a bright or a dim lamp.

Step 2: the genuine watermark portrait of each note value. For each denomination d, the portrait T_d is the pixel-wise median of the quotient images of the *genuine training* notes of that value:

$$T_d(x, y) = \operatorname*{median}_{n \in \mathcal{G}_d} Q_n(x, y), \tag{3.5}$$

where $\mathcal{G}_d$ is the set of genuine training notes of value d (235 notes of 500 Taka and 260 of 1,000 Taka). If a value had fewer than 8 such notes, the median over all 495 genuine training notes would be used. Validation and test notes never enter a portrait, and no counterfeit note is needed to build it.

Step 3: what is left over (residual). A genuine watermark should look like the portrait. A counterfeit has to imitate a change of paper thickness by other means, so it is expected to leave a pattern that the portrait does not explain:

$$R = Q - T_d. \tag{3.6}$$

Step 4: four summary numbers. The agreement with the portrait and the size and texture of the residual are summarised by

$$\mathbf{s} = \Big[\, \rho(Q, T_d),\ \ \operatorname{std}(R),\ \ \operatorname{Var}\big(\nabla^2 R\big),\ \ \operatorname{mean}\big|Q - G_2 * Q\big| \,\Big], \tag{3.7}$$

$$\rho(A, B) = \frac{\sum (A - \bar{A})(B - \bar{B})}{\sqrt{\sum (A - \bar{A})^2 \sum (B - \bar{B})^2}}, \tag{3.8}$$

that is, the normalised correlation with the portrait, the strength of the residual, the sharpness of the residual (variance of its Laplacian), and the energy of fine detail. Each number is standardised with the mean and standard deviation of the training notes, giving $\widehat{\mathbf{s}}$.

Step 5: decision. Two small convolutional streams ϕ_P and ϕ_R (three stride-2 convolutions with 16, 32 and 64 channels each) read the quotient image and the standardised residual $\bar{R}$. Their pooled features and the four numbers are joined by one linear layer:

$$p_{\text{wm}} = \operatorname{softmax}\Big(W\big[\, \phi_P(Q)\ ;\ \phi_R(\bar{R})\ ;\ \widehat{\mathbf{s}} \,\big] + \mathbf{b} \Big)_{\text{genuine}}, \qquad \text{“watermark clear” if } p_{\text{wm}} \geq 0.5. \tag{3.9}$$

The whole classifier has 47,818 parameters (0.21 MB), against 2.23 million (9.15 MB) for the MobileNetV2. It is trained for 12 epochs with AdamW (learning rate 10^{-3}), with small rotations ($\pm 5°$) and shifts (± 3 pixels) applied before the residual is formed; the epoch is chosen by validation AUC and the threshold stays at 0.5. Algorithm 3.4 summarises the method, and Section 6.4.2 gives its results.

Algorithm 3.4 Denomination-matched watermark residual (DMWR)

Training (once)

Input: watermark crops of training notes with value d and label genuine/counterfeit

for all training crops I_n **do**
  $Q_n \leftarrow \textsc{Quotient}(I_n)$ ▷ Eq. (3.4)
end for
for all note values d **do**
  $T_d \leftarrow$ pixel-wise median of Q_n over genuine training notes of value d ▷ Eq. (3.5)
end for
fit the mean and standard deviation of $\mathbf{s}$ on the training notes
train $\phi_P, \phi_R, W, \mathbf{b}$ with cross-entropy; keep the epoch with the best validation AUC

Checking one note

Input: watermark crop I, note value d

$Q \leftarrow \textsc{Quotient}(I)$; $R \leftarrow Q - T_d$ ▷ Eq. (3.6)
$\widehat{\mathbf{s}} \leftarrow$ standardised $\left[\rho(Q, T_d), \mathrm{std}(R), \mathrm{Var}(\nabla^2 R), \mathrm{mean}\,|Q - G_2 * Q|\right]$
$p_{\mathrm{wm}} \leftarrow$ Eq. (3.9)
return "watermark clear" if $p_{\mathrm{wm}} \geq 0.5$, else "check by hand"

3.4.8 UV Security-Fibre Check

Bangladeshi 200, 500 and 1,000 Taka notes carry short security fibres that glow under ultraviolet (UV) light, on paper that stays dull, while ordinary white paper glows as a whole. Our own UV photographs of genuine notes show exactly this (Figure 7). A genuine note therefore *has* UV marks; a note without them is the suspicious one. As an additional, optional check, a UV lamp beside the camera is switched on after the note has been named, and the fibres inside the note box are counted.

Method (excitation-normalised fluorescence residual). The lamp's own light reaches the camera as blue. Anything that fluoresces gives off light of another colour. For the paper of one note, green and red are a fixed share of blue, estimated in the same picture as $a = \mathrm{median}(G/B)$ and $b = \mathrm{median}(R/B)$. What that share does not explain is fluorescence:

$$E = \max\left(G - aB,\ R - bB\right), \qquad r = \frac{\mathrm{tophat}(E)}{G_6 * B + 1}, \tag{3.10}$$

where tophat is the white top-hat with a 9-pixel disc (it keeps thin bright structures) and $G_6 * B$ is the local lamp level. Fibres are the connected parts of $r > 0.12$ that are 5 to 70 pixels long and thin, in a note crop brought to a width of 600 pixels. In addition, a fibre pixel must be clearly less blue than the paper and lie on lit paper:

$$\frac{\max(G - aB,\ R - bB)}{B} > c, \qquad B > \tfrac{1}{2}\,\mathrm{median}(B), \tag{3.11}$$

with $c = 0.22$. This colour condition was added after a simulation showed that, without it, the colour smear that JPEG compression leaves at the edges of print was counted as fibres (Section 6.4.3). The note is accepted when at least T fibres are found ($T = 1$ with the colour condition). Because a, b and the lamp level are measured in the same picture, the brightness and distance of the lamp cancel: multiplying the lamp intensity by any factor leaves a, b and (up to the constant 1) r unchanged. A change of the camera's colour balance does not cancel: the paper still gives $E = 0$, but the contrast of a fibre is multiplied by the green-to-blue gain ratio, so T and the threshold on r belong to one camera.

The check is switched off by default and its wording is cautious ("UV marks found, likely genuine" or "UV marks not found, check by hand"), because it has not been tested on coun-

terfeit notes or through the glass camera, and its fibre-free side has so far only been simulated (Section 6.4.3).

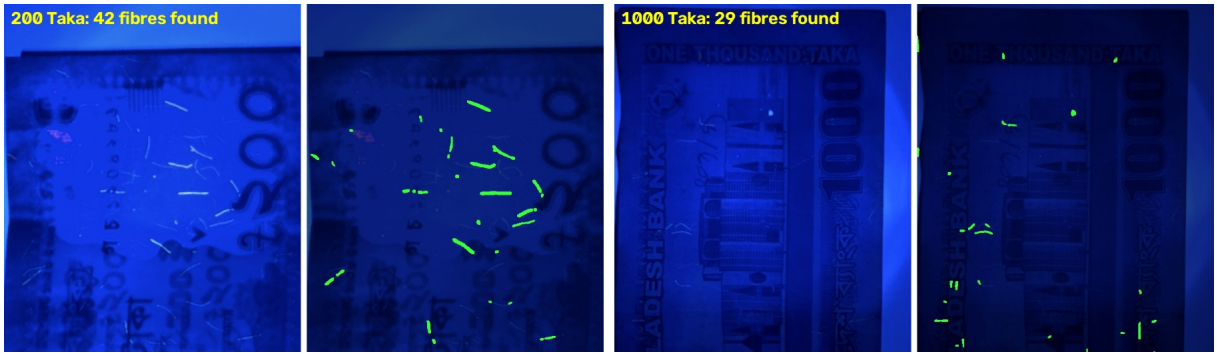


Figure 7: Genuine 200 and 1,000 Taka notes under a UV lamp (left of each pair) and the security fibres found by the check (green, right of each pair)

3.4.9 Emotion Mode

The emotion mode finds the nearest face with the YuNet detector [98] (an OpenCV Haar cascade is the fall-back) and classifies it into seven expressions (happy, sad, angry, surprise, fear, disgust, neutral) with an EfficientNet-V2-S network [99] at 224×224 pixels, averaging the prediction with that of the mirrored face. A FER+ model [100] is kept as a fall-back. When ACTION is pressed, the glass speaks the expression in Bangla. In the desktop harness the detected emotion can also change the style of the speech: slower with a short reassuring phrase for sad, fearful or angry expressions. Until 3 October 2026 the recogniser existed only in the desktop harness; it was added to the glass application as a mode on that date.

3.4.10 Field Log

To study the glass in real use without loading the Raspberry Pi, every event is put on a queue and written by a background thread to `events.csv` (time, mode, button, result fields, duration); a second file, `system.csv`, records memory, CPU temperature and throttling state every 10 seconds. Saving the captured frame is optional. A report script summarises a run.

3.4.11 Voice Interaction and Feedback

All speech passes through one queue served by a worker thread, so that modes never block each other. The `TTSEngine` decides the language of each segment: if more than 25% of the characters are in the Bangla Unicode block (U+0980–U+09FF) the segment is Bangla, otherwise English. Mixed text is split into Bangla and English runs and into sentences with a 0.15-second pause between them. Each language is spoken with a Piper voice [55] when its model is installed (a Bangla and an English voice, loaded once and kept in memory through Piper's Python package) and with espeak-ng [54] otherwise, at 135 words per minute with a lower pitch and a small gap between words. A voice that fails is remembered and not tried again in that run. Every sentence is rendered completely before it is played, and the next sentence is rendered while the current one is heard; the audio of short fixed prompts such as the mode names is kept, so they start at once. Rendering first matters on the Pi: when synthesis and playback ran together, the player ran out of samples whenever text reading held all four cores, and the voice broke up. In the desktop harness on Windows, Bangla is spoken with an online voice and English with the Windows speech engine. The queue holds at most five waiting messages; older ones

are dropped so that the user always hears current information, except the lines of a page, which are never dropped. One message is always spoken whole: an earlier version queued sentences separately, so a long text pushed its own first sentences out of the queue. Punctuation, digits and the Bangla full stop stay with the language around them; before, each of them was cut out of a Bangla sentence as a one-character English piece with its own pause, which made Bangla reading break up.

3.4.12 Overall System Flow and Logic

Figure 8 shows the control flow of the application. The program runs several threads: the main thread, a camera thread that always keeps the latest frame, the speech worker, the object-scanning loop, an inference thread for ACTION requests, and GPIO interrupt threads. An `_inferring` flag prevents two heavy OCR or currency jobs from running at the same time.

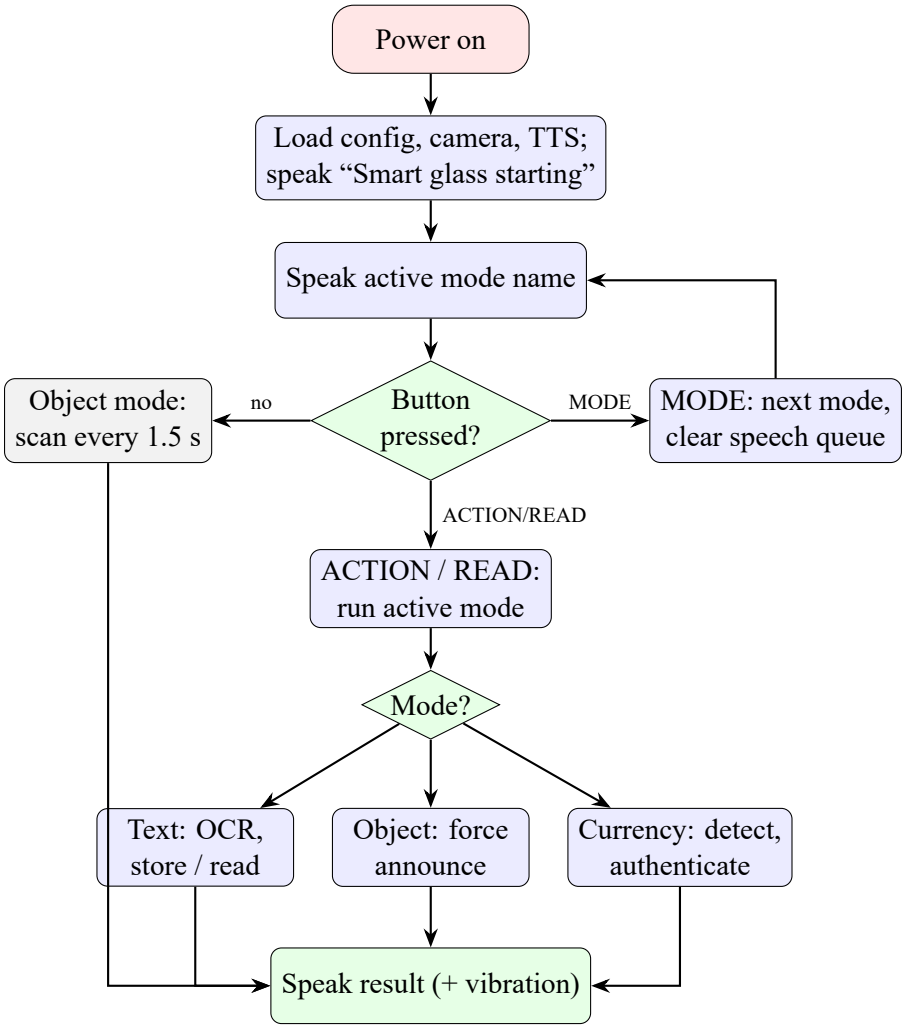


Figure 8: Overall control flow of SARA

3.5 Methodology: Building SARA

The development of SARA followed an iterative methodology in four phases: requirement analysis, data engineering and model development, system integration, and evaluation.

3.5.1 Requirement Analysis

The requirements were derived from the review of existing systems (Chapter 2) and from the situation of visually impaired people in Bangladesh: the device must work offline, speak Bangla, read Bangla and English, recognise Taka notes, help with counterfeit notes, and be usable without a screen. These requirements defined the modes, the button interface and the choice of offline models.

3.5.2 Data Engineering and Model Development

Each vision task was developed as a separate machine learning problem with its own dataset, training procedure and held-out evaluation:

- **Currency detection:** a copy-paste compositing pipeline generated a large, automatically labelled detection dataset from clean note images and real backgrounds, and a COCO-pretrained YOLOv8s was fine-tuned on it (Figure 9).
- **Currency authentication:** a CNN+ViT baseline was trained on six views; after observing its collapse with fewer views, the prefix-robust protocol was designed and compared against it on the same note-disjoint split. After the serial-number audit, the main methods were retrained and compared on a print-disjoint split, together with the watermark check and ResNet-50 baselines [101].
- **Emotion recognition:** several backbones were fine-tuned on the official RAF-DB training set and compared on the official test set.
- **Text reading:** EasyOCR was kept as the engine, and the pipeline around it was built in seven steps on a new benchmark (Section 5.5.1).
- **Objects:** a COCO-pretrained YOLOv8 was used without retraining and evaluated on a COCO subset.

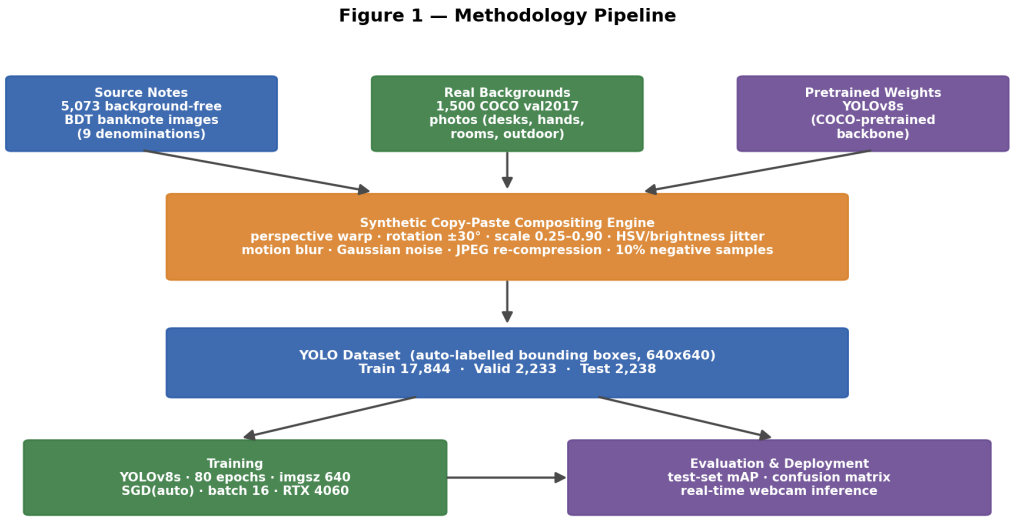


Figure 9: Currency detection methodology: source notes, COCO backgrounds and COCO-pretrained YOLOv8s weights feed the compositing engine, which produces the labelled dataset used for training, evaluation and deployment

Prefix-Robust Multi-View Training

A model trained with all six views on every sample sees only one "number of views" during training. At test time a blind user may capture only one or two views. This is a distribution shift in the number of views. The prefix-robust multi-view training (PRMVT) protocol removes it by sampling, for every training batch, a random prefix of the ordered views, so that every view count from 1 to 6 has positive probability during training. The protocol has two stages:

1. Six epochs of mixed view dropout with a per-view gate, batch normalisation per view count, an auxiliary single-view loss and a light contrastive loss.
2. Three epochs of fine-tuning at learning rate 3×10^{-4}, where half of the batches use all six views so that six-view accuracy is not lost.

The checkpoint is selected by the mean validation accuracy over one to six views; the test split is never used for selection. Algorithm 3.5 shows one training step.

Algorithm 3.5 Prefix-robust multi-view training (PRMVT)

Input: training notes, each with six ordered views $(v_1, \ldots, v_6)$ and a label y; stage-1 epochs 6, stage-2 epochs 3
Output: authenticator that works with any number of views

for each epoch and each batch $\mathcal{B}$ **do**
  if stage 2 **and** the batch is one of the half kept at six views **then**
    $k \leftarrow 6$
  else
    $k \sim \text{Uniform}\{1, \ldots, 6\}$ ▷ random prefix length
  end if
  keep views $v_1, \ldots, v_k$ of every note in $\mathcal{B}$ and mask the rest
  $h_i \leftarrow \textsc{Encoder}(v_i)$ for $i \le k$ ▷ MobileNetV3-Small + TinyViT
  $p \leftarrow \sigma\big(\textsc{Fusion}(h_1, \ldots, h_k)\big)$ ▷ batch normalisation for view count k
  $\mathcal{L} \leftarrow \text{BCE}(p, y)$ + auxiliary single-view loss + light contrastive loss (stage 1)
  update the weights with $\nabla \mathcal{L}$
end for
keep the checkpoint with the best mean validation accuracy over $k = 1, \ldots, 6$

A simpler variant, the same network with one shared batch normalisation and prefix sampling only, was added later as a controlled comparison and turned out to be the most accurate (Section 6.3).

3.5.3 System Integration

The trained models were integrated as modes of the glass application. Integration was first done on a Windows laptop with a webcam through a preview window, where the keyboard keys M, A, R, +, − and Q replace the buttons, and then deployed on the Raspberry Pi 5 with a deployment script and a systemd service.

3.5.4 Evaluation Protocol

The same rules were applied to every experiment:

- **Three splits.** Models are trained on the training split. Every choice (checkpoint, threshold, parameter, which method to adopt) is made on the validation split. The test split is read once for a finished method and is never used for a choice.

- **Split by physical unit.** Notes, writers and, for OCR, phrases and fonts are kept in one split only.
- **Three seeds.** Trained models are repeated with seeds 42, 43 and 44, and the mean and standard deviation are reported.
- **Paired tests.** Two methods on the same test items are compared with the exact McNemar test [102] or, for error rates per image, the Wilcoxon signed-rank test [103]; families of tests are corrected with Holm's method [104]. Proportions are given with Wilson 95% intervals [105].
- **Negative results.** A method that does not help is reported and not adopted, even if it looks better on the test split.

The metrics per module are:

- **Currency detection:** precision, recall, mAP@0.5 and mAP@0.5:0.95 on held-out composites; top-1 denomination accuracy on independent photographs.
- **Authentication:** accuracy for 1 to 6 views on the note-disjoint and the print-disjoint split; expected calibration error (ECE) [80]; robustness under image corruptions; for the deployed policy, the number of counterfeits passed as genuine and of genuine notes confirmed.
- **Emotion:** accuracy, macro precision, recall and F1 on the official RAF-DB test set.
- **OCR:** character error rate (CER) and word error rate (WER), the edit distance [106] between the recognised text and the ground truth divided by the length of the ground truth, after Unicode NFC normalisation.
- **Objects:** mAP@0.5 on the 5,000 photographs of COCO val2017.
- **Speed:** median and 95th-percentile time per call over 100 calls on the Raspberry Pi 5, and memory, temperature and throttling state during a 30-minute mixed run.

An append-only database (`research_results/experiments.json`, 204 experiments in 11 comparison groups) records each experiment with its result file, and a claim registry links the 298 numerical claims of the project to their files; a script checks that each claimed value equals the value in the file. All numbers in Chapter 6 are copied from these files. Measurements that do not yet exist are marked as not measured.

4 Implementation Process

4.1 Design Overview

The implementation of SARA was guided by usability for people without sight, by the limited computing power of a Raspberry Pi, and by the need to work without the internet. The system was not designed as a single large model; instead, each function is a small, replaceable mode that uses the best available offline model and falls back to a simpler method if a model is missing. The work progressed in four stages: a user-centred interaction design, a desktop prototype with a webcam, deployment and measurement on the Raspberry Pi 5, and a first hands-on test of the glass.

4.1.1 User-Centred Design

A person who cannot see the device must be able to find every control by touch and must never need a screen. For this reason the glass uses five physical buttons with distinct positions, confirms every action with speech, and speaks the name of the mode after every mode change. Heavy tasks run only on request, so that the user decides when the device speaks. Messages are short, in Bangla, and can be repeated with the READ button in text mode. When the device cannot do what was asked, it says why: “no note found, hold it in front of the camera”, “hold the note closer to the light”, “no face found”.

4.1.2 Prototype Development Environment

Most development and all model training were done on Windows 11 laptops with NVIDIA GPUs. The authentication, watermark, emotion and OCR experiments, and all evaluations reported in this thesis, were run on an RTX 3050 Laptop GPU with CUDA 12.4 and PyTorch 2.6 [107]. The same Python code runs on the Raspberry Pi 5; only the camera, button and audio back-ends differ.

4.2 Hardware Implementation

4.2.1 Component Assembly

The camera module is mounted above the bridge of the spectacle frame so that it points where the user’s face points. In the first prototype the Raspberry Pi 5 is tied to the right temple of the frame and the ribbon cable loops over the head from the camera to the board (Figure 3); Figure 10 shows it being worn. In the intended design the cable runs along the temple to a separate enclosure, which carries the board and the button pad, the earphone connects to the audio output, and the vibration motor sits in the enclosure so that the user can feel it in the hand or pocket.



Figure 10: The first prototype worn during a trial session. A 200 Taka and a 100 Taka note are held in front of the camera for the currency mode. The photographs are used with the wearer's consent.

4.2.2 Connectivity and Wiring

1. Raspberry Pi 5 Power

The Raspberry Pi 5 is powered through its USB-C port from a power bank. The system starts automatically when power is connected.

2. Camera Module 2

The camera is connected to the CSI connector and read with Picamera2. On the Raspberry Pi 5 the OpenCV capture interface opened the camera device but received no frames, so the camera manager now tries Picamera2 first, then OpenCV, and finally scans the video devices while skipping the Pi's internal image-processing nodes. If the camera is already in use by another program, the manager stops with a message that says so. A USB webcam is selected with `CAMERA_BACKEND` and `CAMERA_INDEX`; the frame size is set with `CAMERA_WIDTH` and `CAMERA_HEIGHT`.

3. Push-Buttons

Each button connects one GPIO pin (BCM 17, 27, 24, 22, 23) to ground. The pins are configured as inputs with internal pull-up resistors, and a falling edge generates an interrupt that is debounced in software for 250 ms using the `rpi-lgpio` library. The library creates a notification pipe in its working directory; because the project was kept on a FAT-formatted USB drive, which cannot hold pipes, the working directory of the library is set to the system's temporary folder. A pin that is already claimed by another process produces an error message with instructions instead of a stack trace.

4. Vibration Motor

The vibration motor is driven with PWM from BCM pin 13. Patterns are defined as lists of (on, off) durations in milliseconds; a detected note gives one pulse of 50 ms.

5. Audio

Speech is sent to the default ALSA device with `aplay`; the volume is changed with `amixer`. Each sentence is first rendered completely to 16-bit mono PCM in memory and only then handed to `aplay`, at the sample rate of the voice that produced it and with a half-second device buffer. A USB or headset microphone, when one is plugged in, is used for voice commands.

4.3 Software Deployment and Functional Integration

4.3.1 Environment Setup and Libraries

The software stack is summarised in Table 4.3.1.1. On the Raspberry Pi, the deployment script `scripts/deploy_pi5.sh` installs the system packages (libcamera, Picamera2, espeak-ng, ALSA utilities, OpenBLAS), creates a Python virtual environment, installs the Python requirements, the Piper ARM64 program and an English voice, and copies the model files. The virtual environment is created in the user's home folder with access to the system packages: Picamera2 is a system package, and the FAT-formatted drive that holds the project cannot store the symbolic links a virtual environment needs. A pre-flight script (`scripts/pi5_preflight.py`) then checks that every model and library is present before the application is started.

Integration Stages

Integration was carried out step by step:

1. Camera and speech alone: live frames and spoken test sentences.
2. Button handling: mode cycling with spoken mode names.
3. Each mode added and tested separately in the Windows harness.
4. Concurrency: object scanning in parallel with on-demand OCR and currency, protected by an inference flag.
5. Packaging: deployment script, systemd service and logging with rotation (5 MB, 3 backups).
6. Deployment on the Raspberry Pi 5 and benchmark of every mode.
7. Field log, live algorithm panel and study tools.

Core Implementation Modules

Camera Manager

The `CameraManager` runs a background thread that reads frames continuously and keeps only the most recent one. Modes always receive a fresh frame, and a slow OCR call never causes a backlog of old frames.

Mode Switching

Loading a model takes one to two seconds, which is too long to wait in silence. When MODE is pressed, the name of the mode is spoken at once and the model is loaded and warmed up in a background thread; a lock prevents two models from loading at the same time, and the currency mode is preloaded when the application starts. On the laptop CPU this reduced the first currency check after a mode change from 661 ms to 43 ms.

Table 4.3.1.1: Main libraries and tools used in SARA

Library / tool	**What it is**	**What we use it for in SARA**
Python 3	Programming language	The whole program is written in it
OpenCV [108]	Image-processing library	Cleaning up pictures before the models see them, finding faces (YuNet or Haar), lining up two pictures of a note (SIFT), reading a USB camera
Picamera2	Raspberry Pi camera library	Taking pictures with the Pi camera on the glass
PyTorch, torchvision	Deep-learning framework	Training and running the counterfeit-check and emotion models (MobileNetV3, EfficientNet-V2-S, TinyViT)
Ultralytics YOLOv8	Object-detection library	Training and running the object and Taka detectors, and saving them in faster formats (ONNX, TorchScript)
ONNX Runtime	Fast model runner for the CPU	Running the compressed (INT8) Taka detector, the watermark finder and checker, and a back-up emotion model (FER+)
EasyOCR	Text-reading (OCR) library	Reading Bangla and English text
HarfBuzz, FreeType	Text-drawing libraries	Drawing Bangla letters correctly in the test images used to measure text reading
Tesseract, PaddleOCR, KenLM, SymSpell	Other text readers and spelling correctors	Compared with EasyOCR in the text-reading tests; not used on the glass
sounddevice, SpeechRecognition	Microphone and speech-recognition libraries	Recording and understanding spoken answers in the voice questionnaire
Piper, espeak-ng	Offline text-to-speech programs	Speaking all results without the internet (Piper: natural voice; espeak-ng: simpler, robotic voice)
rpi-lgpio	Pin-control library for the Pi 5	Reading the buttons and driving the vibration motor
pytorch-grad-cam	Model-explanation library	Heat maps that show which part of the picture the Taka detector looked at (Grad-CAM)
systemd	Linux service manager	Starting the glass automatically when the Pi is switched on, and restarting it if it crashes

Language Detection and Speech Segmentation

The speech engine decides the language of each piece of text by counting Bangla characters, and splits mixed-script text into runs that are spoken with the correct voice:

```
def detect_language(text: str) -> str:
    """Return 'bn' if >25% of non-space characters are Bangla, else 'en'."""
    stripped = text.replace(" ", "")
    if not stripped:
        return "en"
    bangla_count = len(_BANGLA_RE.findall(stripped))
    return "bn" if (bangla_count / len(stripped)) > 0.25 else "en"
```

Speaking a mixed Bangla–English sentence as one block forces the wrong voice onto half of it, which made early OCR output sound garbled. Splitting on script boundaries fixed this while keeping the original reading order.

Text Reading with EasyOCR

The OCR mode loads the EasyOCR reader for Bangla and English lazily on first use, because it needs about 600 MB of memory; on a machine with a GPU it uses the GPU, on the Pi the CPU. The frame is contrast-enhanced with CLAHE and read once with the tuned settings. The step that matters most is the selection of the main text block, shown below as it is in the code. It starts from the box with the largest product of height and confidence and repeatedly adds boxes that have a similar height and lie within one and a half text heights of the block:

```
def dominant_block(det, min_conf=0.2, h_ratio=0.6, gap=1.5):
    boxes = [b for b in _boxes(det) if b["c"] >= min_conf]
    if not boxes:
        return []
    seed = max(boxes, key=lambda b: b["h"] * b["c"])
    block, rest = [seed], [b for b in boxes if b is not seed]
    grown = True
    while grown:
        grown = False
        for b in list(rest):
            hs = float(np.median([x["h"] for x in block]))
            if not (h_ratio <= b["h"] / max(hs, 1.0) <= 1 / h_ratio):
                continue # different text size
            if any(b["x0"] - x["x1"] < gap * hs and
                   x["x0"] - b["x1"] < gap * hs and
                   b["y0"] - x["y1"] < gap * hs and
                   x["y0"] - b["y1"] < gap * hs for x in block):
                block.append(b) # touches the block
                rest.remove(b)
                grown = True
    return block
```

The boxes of the block are then grouped into lines by their vertical centre, each line is sorted from left to right, and the clean-up rules are applied. The text is spoken once directly after capture and stored, so that READ can repeat it.

EasyOCR enlarges the image towards a canvas size before detection. A canvas of 640 pixels makes OCR about three times faster on the Pi than the default of 2,560, but on a selection set of phrases that the OCR pipeline had never seen it missed the pre-set margin of 0.5 points (13.84%

against 13.29% CER, Wilcoxon $p = 0.48$), so the default stays at 2,560 and the faster setting is available through `OCR_CANVAS_SIZE`.

Object Detection with YOLOv8

The detection loop runs only in object mode. Every 1.5 seconds it takes the latest frame, resizes it to 320×240, runs YOLOv8 with confidence 0.50, keeps the three most confident unique classes whose cooldown has expired, translates them to Bangla with `labels_bn.json`, and queues a sentence of the form "in front: *object 1*, *object 2*". Pressing ACTION forces the next scan to ignore the cooldown.

Currency Detection and Authentication

The currency mode first runs the YOLOv8s Taka detector. The logic that follows was shaped by three failures. An earlier version fell back to a colour guess when the detector found nothing, and announced wrong denominations; the colour guess is now used only when no detector is installed. An earlier version announced every detection, and named 1 Taka notes, which the detector does not know, as other notes; a detection is now announced only at a confidence of at least 0.60. And an earlier version spoke "genuine" or "counterfeit" from one crop of the note, which was wrong on 20–59% of genuine whole-note photographs (Section 6.5.1); it was first switched off and then replaced by the safe policy. The simplified logic is:

```
def process_frame(self, frame): # simplified
   hits = self.detect_live(frame) # YOLOv8s Taka + safe check
   if hits and hits[0]["conf"] < config.CURRENCY_ANNOUNCE_CONF:
      return NOT_SURE_HOLD_CLOSER # possibly an unknown note
   if hits:
      self._buzz("detect")
      if hits[0]["auth"] in ("likely_genuine", "check_by_hand"):
         return hits[0]["text"] # value + safe verdict
      return hits[0]["text"] + CHECK_NOT_DONE # other denominations
   if self._yolo is not None:
      return NOTE_NOT_FOUND_MESSAGE
   ... # no detector: colour fall-back
```

For a 500 or 1,000 Taka note the four view windows are cut from the detected box and scored, and the text becomes the denomination followed by "likely genuine" or "check by hand". When the watermark check is switched on (the default on the Pi), the capture guide then asks the user to hold the note up to a light and runs the watermark path of Section 3.4.6. Every watermark check is written to a study log with the participant, the condition (guided or not), the time, the rejected frames and the outcome.

The detector is also used in a live desktop application that draws boxes on the webcam image and speaks the detected note when SPACE is pressed (Figure 11). Frames darker than a mean intensity of 80 are brightened with a gain of 3.0 and a gamma of 0.5 before detection, which fixed a failure observed in dim rooms.

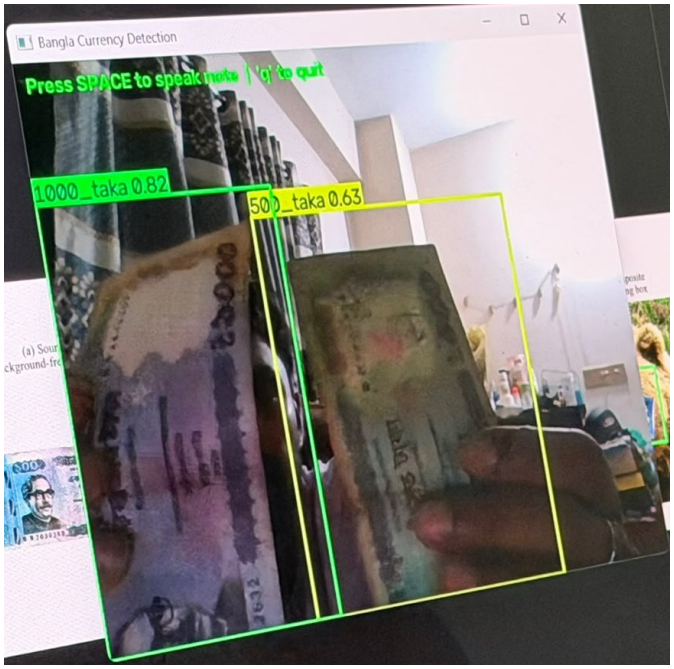


Figure 11: Live detection application on a laptop webcam: a 1,000 Taka and a 500 Taka note held by hand are detected with confidence 0.82 and 0.63

To check that the detector looks at meaningful parts of the note, Grad-CAM [109] heat maps were produced. Figure 12 shows that the activation concentrates on the portrait and the denomination numerals rather than on the background.

Figure 8 — Grad-CAM Heatmap: Model Focus on Note's Portrait Area

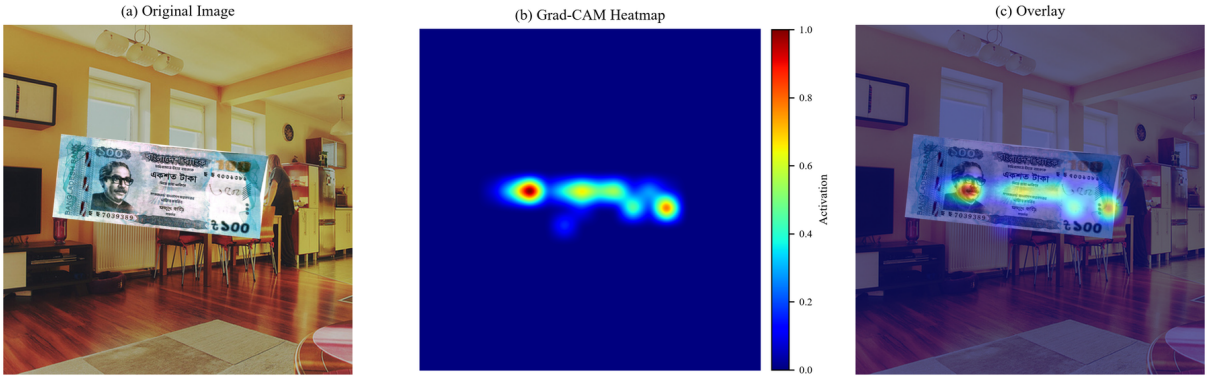


Figure 12: Grad-CAM explanation for a 100 Taka note: (a) input, (b) heat map, (c) overlay. The model focuses on the portrait and numeral regions.

Model Export for the Edge

The Taka detector was exported to TorchScript, ONNX and INT8-quantised ONNX. The first INT8 export quantised the whole network, including the detection head; its class scores collapsed (highest score 0.007) and it detected no note on the test images, so it was discarded. The INT8 model now used is statically quantised with ONNX Runtime, calibrated on 200 validation images, with the detection head and the first convolution kept in 32-bit floating point. On the full synthetic test split it matches the FP32 ONNX model (mAP@0.5 0.995 for both; mAP@0.5:0.95 0.823 against 0.821), and the file is 18.0 MB compared with 22.5 MB for the PyTorch weights.

Emotion Mode

The emotion mode detects the nearest face with YuNet, crops and resizes it to 224×224, and classifies it with EfficientNet-V2-S, averaging the prediction with that of the mirrored face. It speaks one sentence, "the person in front looks happy" (in Bangla), or says that no face was found and that the camera should be pointed at a face. The model file is 82 MB. Until 3 October 2026 emotion recognition ran only in the desktop harness, where it adapted the speaking style; an earlier project report described it as working on the glass, which was wrong, and the mode was added after the first field test showed that it was missing. On the glass the camera faces outward, so the mode describes the person in front of the user, not the user.

Optional Online Scene Description

The online mode sends the current frame to a cloud vision–language model and speaks a short Bangla or English description. It supports the Anthropic and the OpenAI API, requires an internet connection and an API key, and is never used by the other modes. Its accuracy and latency have not been measured: the account supplied for the test had no credit, and the evaluation script returned an error for every request.

Field Log and Live Algorithm Panel

The field logger writes two CSV files per run. `events.csv` has one row per event with the columns time, event, mode, latency, result, denomination, confidence, verdict, score, language and detail; `system.csv` has the CPU temperature, throttling state, load, free memory and the memory of the application every 10 seconds. Rows are put on a queue and written by a separate thread, so logging does not slow the modes; if the queue is full, the event is counted as dropped. For demonstrations and debugging, the preview window has a panel (key L) that runs every model on the current picture and shows its output and time.

Voice Study Tool

For the user study, a session runner asks the participant's name by voice, lets the participant use the glass, and, when the experimenter presses D, asks six yes/no questions in Bangla (ease of use, clarity of speech, help with currency, help with reading, trust, willingness to use daily). Answers are spoken. The microphone is read at its native sample rate, recording starts after 0.2 seconds of sound and stops after 1 second of silence, so that a click is not taken for an answer; the recording is sent to an online speech recogniser in Bangla and, if no yes or no is found, in English, and every alternative transcription is searched for a yes or no word. The experimenter can also answer with the Y and N keys. Each session receives an identifier (P001, P002, …) and is saved automatically with the recordings, the recognised text and whether each answer came by voice or key. This tool needs the internet and runs on the laptop, not on the Pi.

Voice Commands

The five buttons can also be operated by speaking. A small offline recogniser (Vosk, 68 MB English model) listens on the microphone and is given a closed list of 39 command phrases as its grammar, so it can only answer with one of them or with "unknown"; it is not open dictation. Table 4.3.1.2 lists them. A mode word both selects the mode and triggers one capture, so "check money" replaces pressing MODE until the currency mode is reached and then ACTION. Three rules keep the glass from acting by mistake: the whole utterance must be one command phrase (a command word inside a longer sentence is ignored), every word must be recognised with a

confidence of at least 0.7, and the microphone is ignored while the glass itself is speaking and for 0.4 s afterwards. An optional wake word (“glass”) can be required in noisy places. The commands call the same handlers as the buttons, so both work side by side. The command words are English because no offline Bangla recogniser of this size was available; results are in Section 6.9.3.

Table 4.3.1.2: Voice commands of SARA

Spoken command	**What the glass does**
“read text”, “read”, “text”	Text mode; one capture is read aloud
“money”, “check money”, “taka”	Currency mode; one note is checked
“objects”, “find objects”, “around me”	Object mode (it announces by itself)
“describe”, “describe scene”	Online description of one picture
“emotion”, “face”	Emotion mode; one face is read
“next mode”	Same as the MODE button
“capture”, “scan”, “check”	Same as the ACTION button
“repeat”, “say again”	Same as the READ button
“louder”, “quieter”, “volume up”, “volume down”	Volume
“stop”, “quiet”	Stops the voice at once

Start at Boot

The glass runs as a systemd service of the user who owns it, so that it starts without a keyboard or screen, restarts automatically after a crash, and has the sound system of that user’s session. The script `scripts/install_service.sh` fills in the project folder and the Python environment and installs the unit:

```
[Service]
Type=simple
WorkingDirectory=<project>/savior_glass
ExecStart="<environment>/bin/python" "<project>/savior_glass/main.py"
Restart=on-failure
RestartSec=5
Environment=SHOW_PREVIEW=0
```

4.3.2 Windows Development Harness

Because hardware testing is slow, the same application runs on a Windows laptop with a webcam and a preview window. The keys M, A, R, +, − and Q replace the five buttons and the power switch, L opens the algorithm panel, and in a study session D ends the use phase and N starts the next participant. The window shows the camera image, the detections and a status panel. On Windows, English is spoken with the system speech engine and Bangla with an online voice, because the system engine cannot speak Bangla; offline Bangla speech is tested on the Pi with espeak-ng.

4.3.3 Error Handling and Functional Validation

Every model is loaded inside a `try` block. If a model file is missing, the mode logs a warning and falls back to the next stage instead of crashing: the watermark window falls back from the learned localizer to SIFT registration, OCR from EasyOCR to Tesseract, object detection

from YOLOv8s to YOLOv8n, speech from Piper to espeak-ng, and emotion recognition from EfficientNet-V2-S to FER+. One such fall-back had hidden a fault: when Piper failed on the Pi it produced no sound at all instead of falling back, which was found and fixed during the Pi benchmark. Camera failures produce a spoken message ("camera not ready"). The speech queue drops old messages when full, so that a burst of detections never delays the user by more than a few seconds.

4.4 Testing and Calibration

Testing proceeded from single functions to the complete system.

Camera and OCR Testing

The OCR pipeline was first tuned by comparing recognition confidence on rendered words. This was replaced by the benchmark of Section 5.5.1, in which every choice is made on validation phrases and fonts and scored by the error rate, not by the engine's own confidence.

Currency Detector Testing

The detector was evaluated on held-out composites, run live twice for about five minutes each without errors, checked with diagnostic webcam captures in dim light (mean brightness 40 raised to 174 by the brightening step), and evaluated on independent photograph collections.

Authenticator Testing

All authentication models were evaluated with 1, 2, 3, 4, 5 and 6 views on the same note-disjoint test split, with 13 image corruptions and with calibration metrics, and the main models again on the print-disjoint split. The path used by the application (localizer and INT8 classifier) was checked against the research path on the 222 test photographs and made the same decision on 99.1% of them.

Voice and Audio Calibration

The espeak-ng voice was slowed from the default 150 to 135 words per minute, the pitch was lowered from 50 to 45, and a word gap was added, which made Bangla speech easier to follow. A 0.15-second pause between segments separates sentences.

End-to-End Interaction Testing

Complete sequences (boot, mode change, capture, read, currency detection, volume change) were run in the Windows harness and then on the Raspberry Pi 5, where every mode was timed (Section 6.9.2). A first hands-on test of the glass followed (Section 6.10). A structured test with visually impaired users has not yet been carried out.

4.5 Deployment Problems on the Raspberry Pi 5

Bringing the application from the laptop to the Pi exposed problems that no laptop test could show. They are listed in Table 4.5.0.1 because each of them stopped the glass from starting.

Table 4.5.0.1: Problems met when deploying on the Raspberry Pi 5, and their fixes

What went wrong	Why it happened	How we fixed it
The Python environment could not be created	The project was on a USB drive formatted as FAT, which cannot store the shortcut files (symbolic links) the environment needs	Created the environment in the home folder of the Pi instead, still able to use the Pi's own camera library
The button library stopped with "file not found"	It needs to make a special file, which the same USB drive cannot hold	Told the library to make that file in the Pi's temporary folder
"Camera: no frames received"	The OpenCV camera reader gets no pictures from the Pi camera on the Pi 5	Use the Pi's own camera library (Picamera2) first
"Camera in use" or "GPIO busy"	A second copy of the program was already running and holding the camera and buttons	A clear error message that tells the user to stop the other copy
A library (ONNX Runtime) was missing	It was not in the list of libraries to install on the Pi	Added it, plus a check before start-up that every library and model file is present
No sound at all when the Piper voice failed	The program did not switch to another voice	Switch automatically to the espeak-ng voice
The Bangla Piper voice could not be used	The ready-made Pi version of Piper cannot turn Bangla letters into sounds for that voice	Piper is now called through its Python package, which speaks the Bangla voice; espeak-ng remains the fall-back
The voice broke up on the Pi	Speech was synthesised and played at the same time, so the player ran out of samples while text reading held all four cores; the 16 kHz English voice was also played at 22.05 kHz	Each sentence is rendered completely, then played at its own sample rate with a short silence before it (not yet re-tested on the Pi)

5 Dataset and Data Engineering

5.1 Dataset

SARA combines several vision tasks, and each task needs its own data. Table 5.1.0.1 lists every dataset used in this project, what it was used for, and how it was split. All training, validation and test splits are disjoint; where the same physical note or person could appear in more than one image, the split was made by the physical unit (note or writer), not by image. For counterfeit notes even that is not enough, as Section 5.3.1 shows.

Table 5.1.0.1: Datasets used in this project

Dataset	**What it contains**	**What we used it for**	**How it was divided (train / validation / test)**
BanglaTaka [110]	5,073 photos of Taka notes with the background cut away; all 9 note types	The notes that were pasted onto backgrounds to make training pictures	80% / 10% / 10%, by physical note
COCO [50]	Everyday photographs (rooms, streets, desks, hands)	Backgrounds for the Taka pictures; scenes for the text-reading test	Random parts of photos used as backgrounds
Synthetic Taka detection set (made in this work)	22,333 pictures of notes pasted onto real backgrounds, labelled automatically	Training and testing the Taka detector	17,839 / 2,212 / 2,282 pictures
Fresh synthetic test	566 pictures of test notes on backgrounds never used in training	A second test of the Taka detector	Test only
Bangla Money (Kaggle)	1,536 photos of 8 note types; 101 photos of the 1 Taka note	Testing the Taka detector on other people's photos	Test only (never used for training)
NSTU-BDTAKA [61]	186 photos with boxes around notes; 1,144 close-ups of notes held in a hand	Testing the Taka detector on other people's photos	Test only (never used for training)
JaalTaka [70]	1,390 genuine and counterfeit 500 and 1,000 Taka notes, each photographed from 6 views (8,340 photos)	Training and testing the counterfeit check	By note: 974 / 208 / 208 notes; by print (no counterfeit print in two parts): 946 / 222 / 222
Bangladeshi Counterfeit Currency Image Dataset	Photos of whole notes: 1,199 genuine, 87 counterfeit (about 4 physical counterfeit notes)	Checking that the glass's counterfeit answer is safe	Test only
MVP-N [73], ModelNet40 [72]	Objects photographed or drawn from many views	Checking whether our finding about the number of views also holds outside currency	The datasets' own divisions
RAF-DB [111]	15,339 face photos, each labelled with one of 7 expressions	Training and testing emotion recognition	Official division: 12,271 train / 3,068 test
OCR benchmark (made in this work)	Bangla and English phrases drawn with correct Bangla letter shapes; 432 pictures per phrase set	Choosing and testing the text-reading settings	Different phrases and fonts for validation and for test
Ekush [43]	367,018 handwritten Bangla letters and signs, 122 kinds	A back-up reader for single handwritten letters	By writer: 2,429 / 303 / 305 writers
COCO val2017	5,000 photos with 80 kinds of objects	Testing the object mode	Test only
UV note photographs (made in this work)	118 phone photos of genuine 200, 500 and 1,000 Taka notes under a UV lamp	Testing the UV fibre check	By capture order: blocked 5-fold

5.1.1 Implications of These Datasets on Our Research

The datasets define what each result means. The Taka detector is trained only on synthetic composites, so its test score on composites measures performance on data that is statistically similar to training; the raw photograph set is closer to real use. JaalTaka is split by physical note, so the authenticator is never tested on a note it has seen in training; but many of its counterfeit notes are copies of the same print, so the note-disjoint split measures the recognition of known prints, and a second, print-disjoint split is needed. All its notes come from one capture collection, so camera and session changes are not tested. RAF-DB is a standard benchmark with an official split, which allows comparison with published work. The OCR benchmark consists of rendered text, some of it placed in real photographs; it separates the effect of each pipeline step cleanly, but real signs, glare and curved packets will be harder. None of the datasets was photographed through the glass camera.

5.2 Currency Detection Data Engineering

5.2.1 Source Currency Dataset (Foreground)

The banknote images come from the BanglaTaka dataset, "A Diverse Image Dataset for Bangladeshi Currency Recognition", on Mendeley Data [110]. It contains 5,073 background-free images of Taka notes of all nine circulating denominations, captured with different mobile phones and cropped to the note (Table 5.2.1.1). Because each image is a tight crop of one note, it is ideal foreground material for compositing.

Table 5.2.1.1: Per-denomination image counts in the BanglaTaka source dataset

Note (Taka)	2	5	10	20	50	100	200	500	1000	**Total**
Images	445	660	419	480	416	565	423	1,243	422	**5,073**

5.2.2 Background Dataset

Real backgrounds were taken from the COCO val2017 split [50], which contains natural photographs of rooms, desks, streets, hands and outdoor scenes. We downloaded 1,500 images programmatically. Using real photographs instead of plain colours or noise, as in an earlier internal version of the pipeline, was the single most important change for generalising to real camera frames, because it forces the detector to separate the note from realistic clutter, texture and lighting.

5.2.3 The Domain-Gap Problem

A model trained only on background-free notes learns that the note fills the whole frame on a plain background. An early version of this project labelled every training image with a box covering the entire frame. It scored well on its own synthetic test set but failed to detect notes on a live webcam, where the note is small, rotated and surrounded by objects. This failure motivated the compositing redesign.

5.2.4 Copy-Paste Compositing Process

Each source note is pasted onto a randomly chosen and randomly cropped COCO background after a chain of random transformations (Figure 13). Because the position and size of each

pasted note are known exactly, the YOLO label is computed directly from the paste geometry. For a note of pixel size (w, h) pasted with its top-left corner at (x, y) on a square canvas of size $S = 640$:

$$x_c = \frac{x + w/2}{S}, \qquad y_c = \frac{y + h/2}{S}, \tag{5.1}$$

$$b_w = \frac{w}{S}, \qquad b_h = \frac{h}{S}, \tag{5.2}$$

where (x_c, y_c) is the normalised box centre and (b_w, b_h) the normalised box size. This closed-form labelling produced a fully labelled dataset with zero manual annotation. Algorithm 5.1 gives the whole procedure.

Algorithm 5.1 Copy-paste compositing with automatic labels

Input: background-free note images with their value; COCO background photographs; canvas size $S = 640$

Output: training pictures with YOLO labels

for all source notes N with value d **do**
  for $j = 1$ **to** 4 **do** ▷ four composites per note
    $B \leftarrow$ random crop of a random COCO photograph, resized to $S \times S$
    $N' \leftarrow N$ tilted (perspective, 70% of the time), rotated ($\pm 30°$), scaled to fill 25–90% of B
    $N' \leftarrow$ colour, brightness and contrast changed (Table 5.2.5.1)
    choose a random position (x, y) so that N' (size $w \times h$) lies inside B; paste N' onto B
    apply motion blur (20%), noise (50%) and JPEG compression to the picture
    write the label $\left(d,\ \frac{x+w/2}{S},\ \frac{y+h/2}{S},\ \frac{w}{S},\ \frac{h}{S}\right)$ ▷ exact box, no hand labelling
  end for
end for
add background-only pictures (10% of all) with empty labels
split by source note: 80% training, 10% validation, 10% test



Figure 13: Copy-paste compositing applied to a 100 Taka note: (a) background-free source note, (b) real COCO background, (c) note after perspective warp, rotation, scaling and colour jitter, (d) final composite with the automatically generated bounding box and label

5.2.5 Augmentation Operations

Table 5.2.5.1 lists the domain-randomisation operations. The ranges were chosen to cover, and in most cases slightly exceed, the variation expected from a camera held by a person.

Table 5.2.5.1: Domain-randomisation operations used in compositing

What is imitated	Change made to the note	Setting	Real situation it imitates
How the note is held	Tilted (perspective warp)	70% of pictures; corners moved by up to 8%	Note not held flat to the camera
	Rotated	Up to 30° either way	Note held at an angle
	Resized	Note fills 25–90% of the picture	Note held near or far
	Moved	Random position in the picture	Note not in the centre
Light and colour	Colour shifted (HSV jitter)	Hue ±18°; saturation ×0.5–1.5; brightness ×0.4–1.6	Different lamps, sunlight, old faded notes
	Brightness and contrast	Brightness ×0.4–1.6; contrast ×0.7–1.3	Dark rooms and bright light
Camera faults	Motion blur	20% of pictures; small, medium or strong, horizontal or vertical	Head or hand moving
	Grain (Gaussian noise)	50% of pictures; strength about 12	Cheap camera in low light
	JPEG compression	Quality 55–92	Compressed camera pictures
No note	Background-only pictures	10% of all pictures, with no label	Teaches the detector not to see notes where there are none

5.2.6 Final Detection Dataset

Four composites were generated per source note, plus background-only negatives. The data was split 80/10/10 *by source note*, so no physical note image appears in more than one split (Table 5.2.6.1). The counts are those of the dataset as it is stored on disk; an earlier version of this thesis gave 22,315 images, a figure taken from a generation log that no longer matches the files.

Table 5.2.6.1: Composition of the synthetic Taka detection dataset

Part	Pictures with a note	Background only (no note)	All pictures	Notes in the pictures
Train	16,224	1,615	17,839	16,224
Validation	2,016	196	2,212	2,016
Test	2,052	230	2,282	2,052
Total	**20,292**	**2,041**	**22,333**	**20,292**

5.2.7 Raw Photograph Test Set

To test the detector on images that are not composites, 450 raw phone photographs of notes, 50 for each of the nine denominations, were taken from the raw BanglaTaka collection and labelled with their denomination. These images show real paper, real lighting and real camera noise, but they are not independent of the training data in the strict sense: the raw collection is the same source from which the training foregrounds were cropped, so some physical notes may overlap. This set is therefore a source-domain check, not a test.

5.2.8 Independent Photograph Collections

The real test of the detector uses photographs taken by other people and never used for training or selection. *Bangla Money*, a public Kaggle collection, has 1,536 photographs of eight of the nine denominations (no 200 Taka) and 101 photographs of the 1 Taka note, a class the detector does not know. *NSTU-BDTAKA* [61] provides 186 annotated detection images and 1,144 close-ups in which a folded note is held in a hand. A third public collection was excluded because its files are byte-identical to the training source. A further set of 566 composites was built from the test-split notes on backgrounds from a different COCO subset, as a second synthetic test.

5.3 Currency Authentication Data

The authenticator is trained on the JaalTaka collection of genuine and counterfeit Bangladeshi notes [70]. Every physical note is photographed in six ordered close-up views that show different regions and security features (Figure 14). Table 5.3.0.1 gives the statistics of the split used in all experiments.



Figure 14: The six ordered views of one genuine note in the JaalTaka collection. Each view shows a different region, such as the portrait with the hologram strip and the serial number.

An earlier split of the same data at the image level produced an accuracy of 1.00, because views of the same note appeared in both training and test data. That number was a data-leakage artefact and is not reported as a result; all authentication results in this thesis use the note-disjoint split above or the print-disjoint split below.

Table 5.3.0.1: JaalTaka split used for authentication (split seed 42)

What is counted	Number
Physical notes	1,390
Genuine / counterfeit notes	802 / 588
Photos (views) of each note	6
Photos in total	8,340
Notes for training / validation / test	974 / 208 / 208
Genuine / counterfeit notes in the training part	562 / 412
Notes that appear in two parts (train and validation, train and test, validation and test)	0 / 0 / 0

5.3.1 Serial-Number Audit

A genuine banknote has a unique serial number. A counterfeit is printed from a scanned or engraved original, so every copy of one print carries the same serial. To see how many distinct prints JaalTaka contains, the serial number at the lower left of each note was read with OCR, Bangla digits were mapped to 0–9, and the first six digits were compared (one digit can be misread). Table 5.3.1.1 gives the result.

Table 5.3.1.1: Serial numbers read from JaalTaka notes

Notes	Notes in the dataset	Serial number could be read	Different serial numbers	How often the most common serial number appears
500 Taka counterfeit	324	322	27	On 279 notes (86.6% of those read): most counterfeits are copies of one print
1,000 Taka counterfeit	261	201	66	On 43 notes, and two almost identical readings on 29 more
500 Taka genuine	397	331	233	On 1.2% of those read (almost every genuine note has its own number)
1,000 Taka genuine	401	270	227	On 1.5% of those read

Genuine notes have almost one serial per note. Counterfeit notes are dominated by a few serials: 279 of the 322 readable 500 Taka counterfeits carry the same one. The note-disjoint split is therefore not print-disjoint. On its test split, 68 of 87 counterfeits have a serial that also occurs on a training counterfeit, and a rule that needs no knowledge of counterfeiting at all, "counterfeit if the serial occurs among the training counterfeits", scores 90.2% on the 205 test notes that could be reconstructed as whole notes. This is a shortcut in the sense of [77], and it affects every result published on this split, not only ours. The data article does not mention it.

5.3.2 Print-Disjoint Split

A second split was built in which no test counterfeit shares its serial with a training or validation counterfeit: 946 training, 222 validation and 222 test notes. The test split has 121 genuine and 101 counterfeit notes, and its counterfeits come from about 24 distinct prints. This small number is the real size of the test: two detectors can only be compared on about 24 independent counterfeit prints, however many notes there are. All results labelled "unseen prints" in Chapter 6 use this split.

5.3.3 Back-Lit Views and Watermark Labels

The sixth view of each note is a back-lit photograph in which the watermark is visible on a genuine note. For the watermark classifier, the window was cut from each back-lit view after registration to a note template; registration succeeded on 867 of 946 training, 197 of 222 validation and 197 of 222 test photographs. The corner positions of these windows, mapped back into each photograph, are the training labels of the localizer.

5.3.4 Whole-Note Counterfeit Photographs

The glass sees whole notes, not close-up views. To check the deployed verdict, the Bangladeshi Counterfeit Currency Image Dataset was used, which has whole-note photographs of 500 and 1,000 Taka notes: 1,199 genuine and 87 counterfeit, of which 60 are augmented copies. The counterfeit photographs show about four physical notes, so this set can support a safety check but cannot prove it. The genuine 500 and 1,000 Taka photographs of Bangla Money and BanglaTaka were used as further genuine notes.

5.3.5 A Tool for a Larger Print-Disjoint Dataset

A dataset with about 100 unseen counterfeit prints would be needed to compare detectors with confidence. Such notes can only be obtained with a bank or the police, which was outside the reach of this project. A collection tool was written for that future work: it registers each photograph with its note and its print, refuses a print that would appear in two splits, a note that changes print, contradicting labels and duplicates, and exports the split. A second tool records the watermark window by four mouse clicks, and the localizer training script accepts these labels.

5.4 Emotion Recognition Data

Emotion recognition uses the Real-world Affective Faces Database (RAF-DB) [111] with its official training (12,271 images) and test (3,068 images) split. The class distribution of the test set is imbalanced (Table 5.4.0.1): happy faces are sixteen times as frequent as fearful ones, which later explains the weaker recall for fear and disgust.

Table 5.4.0.1: Class distribution of the RAF-DB official test set

Expression	Happy	Sad	Angry	Surprise	Fear	Disgust	Neutral	**Total**
Test photos	1,185	478	162	329	74	160	680	**3,068**

5.5 Text Recognition Data

5.5.1 OCR Benchmark

The first OCR test set of this project had 80 phrases rendered with one font. Two faults made its numbers unusable. First, the images were drawn with a graphics library that, on the development machine, did not shape Bangla text: vowel signs that belong before the consonant were drawn after it, and conjuncts were broken apart. Real print never looks like that. Second, the correction step of the OCR mode used a word list that contained the test phrases. A new benchmark was therefore built:

- **Shaping.** Text is shaped with HarfBuzz [44] and drawn with FreeType, the way a printer or a browser does it.
- **Phrases.** Short phrases of the kind a visually impaired person needs to read (signs, counters, labels, directions). The validation set has 24 Bangla and 24 English phrases; the test set has 48 other phrases. A further 24 phrases were used once, for the canvas-size decision.
- **Fonts.** Validation images use Nirmala UI and Noto Serif Bengali for Bangla and Georgia and Verdana for English; test images use Nirmala UI, Noto Sans Bengali and Tiro Bangla, and Arial, Times and Calibri. In all, 10 Bangla and 9 Latin fonts are used, and the fonts used to train a recogniser are never scored.
- **Conditions.** Every phrase appears three ways: on a clean page, on a distorted page (rotation, blur, low resolution), and as a sign placed into a real COCO photograph at 640×480 pixels, the frame size of the glass.
- **Seeds.** Three image seeds, giving 432 images per phrase set.

Every choice was made on the validation set; the test set was read once per finished method. For the fine-tuning experiment, training text came from general frequency lists of 50,000 Bangla and 50,000 English words, with every word of every benchmark phrase removed.

5.5.2 Ekush Handwritten Characters

As an additional study of Bangla recognition, a CNN classifier for 122 classes of isolated handwritten Bangla characters (vowels, consonants, conjuncts, modifiers and digits) was trained on the Ekush dataset [43]. The data was split by writer (2,429 / 303 / 305 writers; 292,974 / 35,436 / 38,608 images) so that no writer's handwriting appears in both training and test data.

5.6 Object Detection Data

The object mode uses a YOLOv8 model pre-trained on COCO without retraining. To check its accuracy in our pipeline, it was evaluated on the 5,000 photographs of COCO val2017 (an earlier evaluation used a random subset of 250 of them).

5.7 Note on User Survey

This work does not yet include results from a study with visually impaired participants. Trial sessions and a survey have started (Section 6.10), and their data will be added when they are complete. A protocol has been prepared and its hypotheses and analysis were written down in advance (8–12 participants; tasks for denomination identification, the watermark check with and without spoken guidance, and reading; measures including task time, accuracy, NASA-TLX [83] and the System Usability Scale [82]). The software for it exists: the study log of the watermark check, an analysis script, and the voice-driven questionnaire. The questionnaire tool was tried in three pilot sessions run by the project team, each about three and a half minutes long, to test that names and spoken answers are recorded; in the last two sessions five and six of the six answers were recognised from speech. These sessions test the tool. They are not study data, and no user-study result is reported in this thesis. Conducting the study, with ethical approval, is the most important next step (Chapter 7).

6 Result and Discussion

6.1 Performance Evaluation Overview

The evaluation of SARA covers Taka detection, counterfeit authentication on known and on unseen prints, the watermark check, the safety of the spoken verdict, emotion recognition, text recognition, object detection, speed on the Raspberry Pi 5, and a first field test. Each module was evaluated on held-out data that was not used for training or model selection, using the protocol of Section 3.5.4. Evaluations were run on an NVIDIA RTX 3050 Laptop GPU; device figures were measured on a Raspberry Pi 5 with 8 GB of memory. Every number in this chapter was copied from a saved result file of the project. When a measurement does not exist, it is written as *not measured* rather than estimated. Several figures differ from an earlier draft of this thesis; Section 6.11 lists the corrections.

6.2 Taka Detection Performance

6.2.1 Training Behaviour

The YOLOv8s detector was fine-tuned from COCO weights for 80 epochs at 640×640 pixels with batch size 16, SGD (initial learning rate 0.01, momentum 0.937, weight decay 0.0005, three warm-up epochs), mosaic 1.0 and mixup 0.15. Horizontal and vertical flips were disabled because a mirrored note is not a real note. Training converged quickly: mAP@0.5 was above 0.97 after 8 epochs and above 0.99 after 11 epochs, while mAP@0.5:0.95 kept improving slowly as the boxes became tighter (Figure 15).

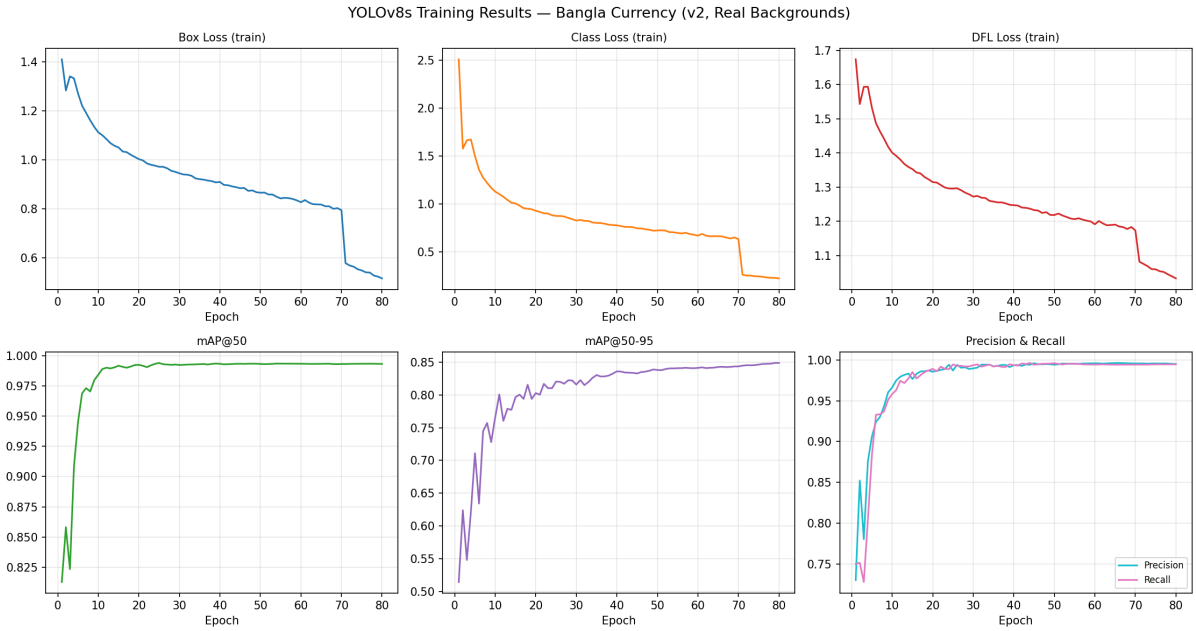


Figure 15: Training curves of the YOLOv8s Taka detector: box, class and DFL losses (top) and mAP@0.5, mAP@0.5:0.95, precision and recall (bottom) against epoch

6.2.2 Results on Held-Out Composites

Table 6.2.2.1 reports the performance on the 2,282 held-out test composites.

Table 6.2.2.1: Taka detection performance on the synthetic test set (2,282 images)

Measure	**Value**	**What it means**
Precision	0.995	Of the notes the detector reported, 99.5% were really there and correctly named
Recall	0.996	Of the notes in the pictures, the detector found 99.6%
mAP@0.5	0.995	Overall finding-and-naming score, with a box counted as correct if it overlaps the note by at least half (1.0 = perfect)
mAP@0.5:0.95	0.849	The same score with stricter box overlaps; lower, because the boxes are not always tight around the note
Time per picture (RTX 3050 laptop graphics card)	13.9 ms	Time for the detector itself
Time to prepare the picture / to sort the boxes	3.2 ms / 2.2 ms	Extra time before and after the detector
mAP@0.5 on 566 fresh synthetic pictures	0.992	Almost the same on notes pasted onto backgrounds never seen in training

The gap between mAP@0.5 (0.995) and mAP@0.5:0.95 (0.849) shows that the detector almost always finds and correctly names the note, but that the box edges are less precise at strict IoU thresholds. One plausible reason is the slight anti-aliasing at the pasted, warped note borders, which does not correspond to a sharp physical edge.

Results are uniform across the nine classes (Table 6.2.2.2, 0.836–0.861), including the smallest source class (10 Taka, 419 source images), so the model does not simply favour the most frequent class (500 Taka, 1,243 source images). The normalised confusion matrix (Figure 16)

Table 6.2.2.2: Per-denomination mAP@0.5:0.95 on the synthetic test set

Taka	2	5	10	20	50	100	200	500	1000
mAP@ 0.5:0.95 (1 = perfect)	0.856	0.843	0.847	0.861	0.848	0.836	0.854	0.853	0.842

is almost perfectly diagonal; the remaining errors are missed or spurious detections against the background, not confusions between two denominations. For a currency reader this is the safer failure mode: the glass may ask the user to hold the note again, but it rarely names the wrong value.

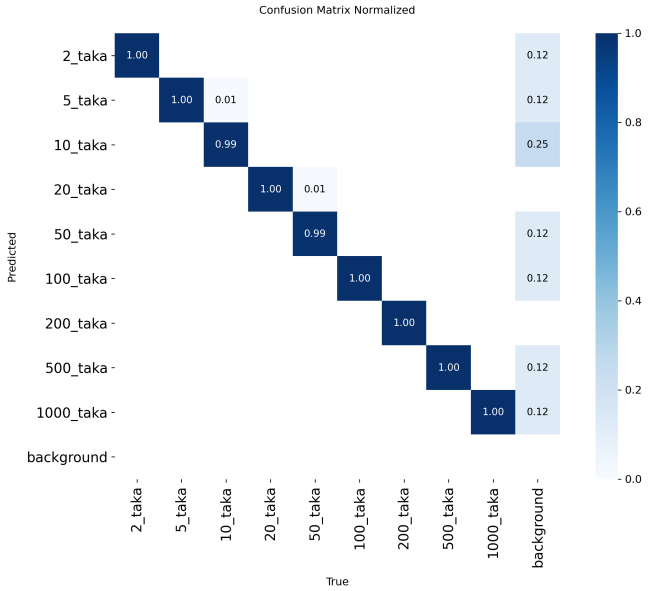


Figure 16: Normalised confusion matrix of the Taka detector on the synthetic test set

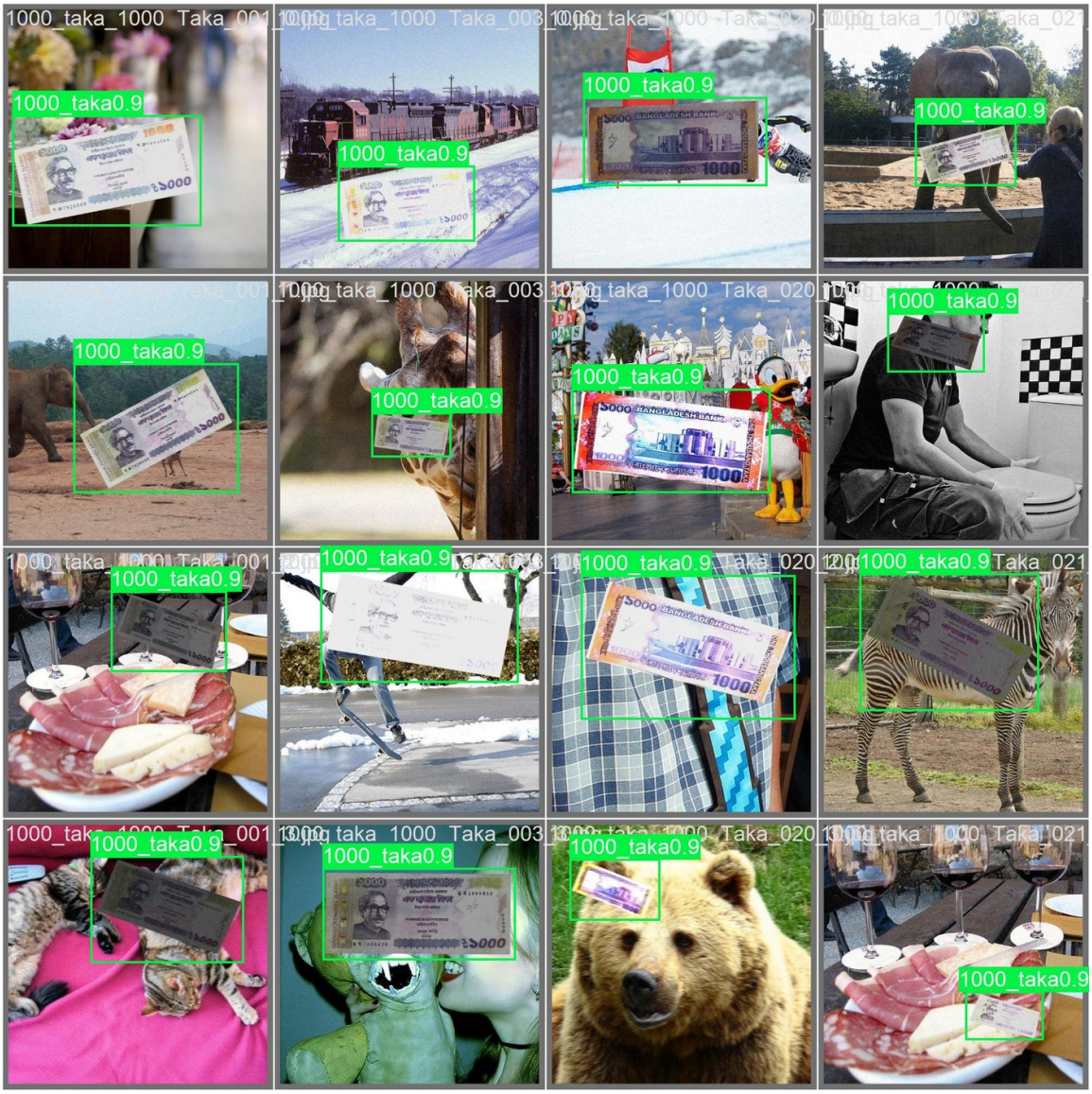


Figure 17: Example predictions on held-out composites with predicted class and confidence. The note is localised across different scales, rotations and backgrounds.

6.2.3 Results on Raw Phone Photographs (Source Domain, Not Held Out)

Table 6.2.3.1 shows the result of the same detector on the 450 raw phone photographs (50 per denomination). A note counts as *detected* if any box is predicted, and as *correct* if the top-scoring box has the true denomination.

On raw photographs the detector finds 98.4% of the notes and names 96.9% correctly. Because these photographs come from the same source as the training foregrounds, this number does not measure transfer to new notes; the independent collections in Table 6.2.4.1 do (91.5% on Bangla Money). The errors are informative: 2 and 5 Taka are both small, similar-coloured notes, and the 1000 and 500 Taka notes share a similar layout and portrait. As explained in Chapter 5, some physical notes in this set may also occur in the training foregrounds, so this is an optimistic estimate. The 450 images were drawn from the raw folder without excluding the photographs used as training foregrounds, so this table is a source-domain check, not a test; the independent results below replace it as the real-photograph result.

Table 6.2.3.1: Taka detection on 450 raw phone photographs (50 per denomination). Source-domain check: these photographs come from the detector's training source and are not held out. Independent results: Table 6.2.4.1.

Real note	Note found (of 50)	Named correctly (of 50)	Named correctly (%)	Mistakes
2 Taka	49	47	94.0	2 called 5 Taka; 1 not found
5 Taka	50	50	100.0	–
10 Taka	48	48	96.0	2 not found
20 Taka	50	50	100.0	–
50 Taka	48	46	92.0	1 called 5 Taka; 1 called 500 Taka; 2 not found
100 Taka	48	48	96.0	2 not found
200 Taka	50	50	100.0	–
500 Taka	50	50	100.0	–
1000 Taka	50	47	94.0	3 called 500 Taka
Overall	**443 (98.4%)**	**436**	**96.9**	**7 not found; 7 called the wrong value**

6.2.4 Results on Independent Photographs

Two public collections that were never used for training or selection were scored with the same detector at confidence 0.25 (Table 6.2.4.1). A third, "A Diverse Image Dataset for Bangladeshi Currency Recognition", was excluded because its files are byte-identical to the training source.

Table 6.2.4.1: Taka detection on independent photograph collections

Photo collection (taken by other people)	Photos	Result
Bangla Money (Kaggle): whole notes of 8 types	1,536	Named correctly in 91.5% of photos; no note found in 3.9%; when a note was found, it was named correctly 95.2% of the time
NSTU-BDTAKA: photos with boxes drawn around the notes	186	Finds notes with a box score (AP@0.5, ignoring the value) of 0.796; 83.2% of reported notes were real (precision); 69.4% of real notes were found (recall)
NSTU-BDTAKA: close-ups of folded notes held in a hand	1,144	Named correctly in only 18.5%; no note found in 66.9%
Bangla Money: 1 Taka notes, which the detector was never taught	101	57 not found (the right outcome); 44 wrongly named as another note (37 of them as 5 Taka)

The detector transfers to full-note photographs taken by other people, with 1000 Taka the weakest class (78.4%, mostly read as 500 Taka). It does not transfer to close-ups in which fingers cover part of a folded note.

6.2.5 Unknown Notes

The detector has no “unknown note” output, and it named 44 of the 101 one-Taka photographs as a known denomination (43.6%). A simple rule reduces this: a denomination is announced only when the confidence is at least 0.60, a value chosen on validation data. On the Bangla Money test photographs the share of 1 Taka notes named as another value falls from 43.6% to 22.8%. The rule is used in the glass; about one unknown note in five is still named wrongly.

6.2.6 Live Check with a Laptop Webcam

The detector was also run live on a laptop webcam with real notes held in the hand (Figure 18). The application draws a box with the denomination and the confidence on the camera image and speaks the note when the space bar is pressed. In these sessions 10, 50, 100, 200, 500 and 1,000 Taka notes were named correctly at confidences between 0.63 and 0.89, including worn and folded notes, notes held at an angle, and two or three notes in one picture. The pictures also show the detector’s weak points. The boxes are loose: they include the hand and part of the background. When notes are close together the boxes overlap, and in the last picture the 500 Taka box also covers the 200 Taka note. These pictures are illustrations from informal sessions. No error rate was counted, so they add no number to the results above.

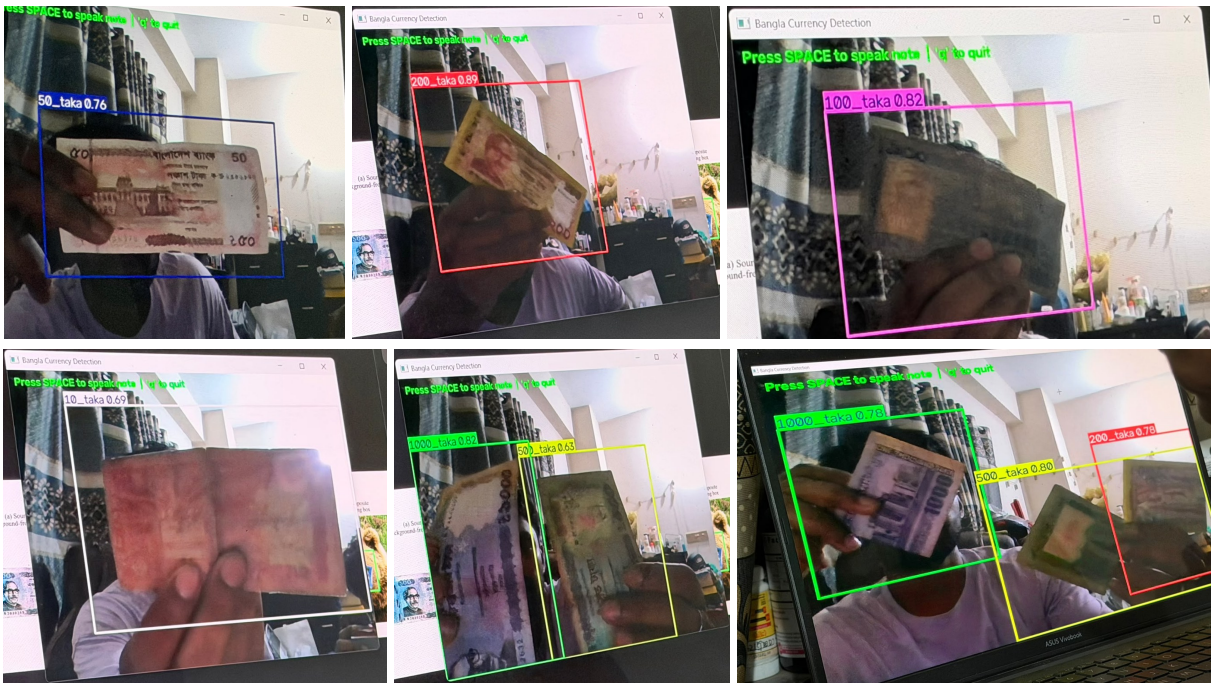


Figure 18: Live detection of real notes with a laptop webcam (photographs of the screen). Top: 50, 200 and 100 Taka. Bottom: a worn 10 Taka note, 1,000 and 500 Taka together, and three notes in one frame.

6.2.7 Several Notes in One Picture

The count and the spoken total were measured on 900 pictures built for this test: one, two or three notes of the detector’s test split on backgrounds that were not used in training, with the training generator’s tilt, rotation, colour, blur and compression, and notes that overlap by at most 5%. The counting rule has no tuned number. The count was right in 99.7% of the pictures and every note, and so the total, in 99.2% (one note: 299 of 300; two notes: 298 of 300; three notes:

296 of 300). No note was counted twice. These are synthetic pictures on the laptop; a hand holding a fan of overlapping notes will be harder.

6.2.8 Fall-Back Classifier

The MobileNetV3-Small classifier used as a fall-back reached a best validation accuracy of 99.2% on the nine-class BanglaTaka crops. This is a whole-note classification score on clean crops, not a detection score in a scene.

6.2.9 Comparison with Related Work

Compared with published detectors (Table 2.11.0.2), our mAP@0.5:0.95 of 0.849 on composites is higher than the 0.804 reported for a custom-head YOLOv8n multi-currency model [62], and our precision and recall (0.995/0.996) are higher than the 0.95/0.93 of the YOLOv5 baseline trained on 3,111 manually annotated images [61]. Our labelled dataset is about seven times larger and required no manual annotation. However, the test sets are different and our headline scores are on synthetic images, so the comparison indicates competitiveness, not superiority on a common benchmark.

6.3 Taka Authentication Performance

6.3.1 Accuracy for One to Six Views

All authentication models were evaluated on the same 208 test notes with the first k views, for $k = 1, \ldots, 6$. Table 6.3.1.1 compares the CNN+ViT baseline, which was trained with all six views, with the prefix-robust model (PRMVT).

Table 6.3.1.1: Authentication accuracy on 208 test notes with the first k views (seed 42)

Photos (views) of the note given	**Earlier model (CNN+ViT), trained with all 6 views**	**Our model (PRMVT), trained with 1 to 6 views**	**Improvement (percentage points)**
1	73.56%	**97.12%**	+23.6
2	87.02%	**98.08%**	+11.1
3	91.35%	**98.08%**	+6.7
4	91.83%	**98.08%**	+6.3
5	89.90%	**98.56%**	+8.7
6	91.83%	**98.08%**	+6.3

The baseline is much weaker with one view (73.6%) than with six (91.8%). An earlier version of our fusion model trained only on six views showed the same problem even more strongly: 96.6% with six views but only 68.8% with one view. This is the view-count distribution shift described in Section 3.5. The prefix-robust protocol removes it: PRMVT stays between 97.1% and 98.6% for every number of views. The difference to the baseline is statistically significant under McNemar's test at one view ($p = 4.3 \times 10^{-11}$) and at six views ($p = 0.0020$). After a Bonferroni correction over 36 algorithm-by-view comparisons, the one-, two-, three- and five-view gains remain significant, while the four- and six-view gains do not; this correction is reported so that the six-view improvement is not overstated. The PRMVT numbers in this chapter were re-evaluated after a numerical fault was fixed: the entropy of a fully confident prediction was

computed as $0 \cdot \log 0$, which is undefined, and the model then reported a probability of 0.5 for that note. The fault affected 23 of the 208 test notes at six views and made the earlier figures slightly lower (six-view accuracy 97.6% instead of 98.1%).

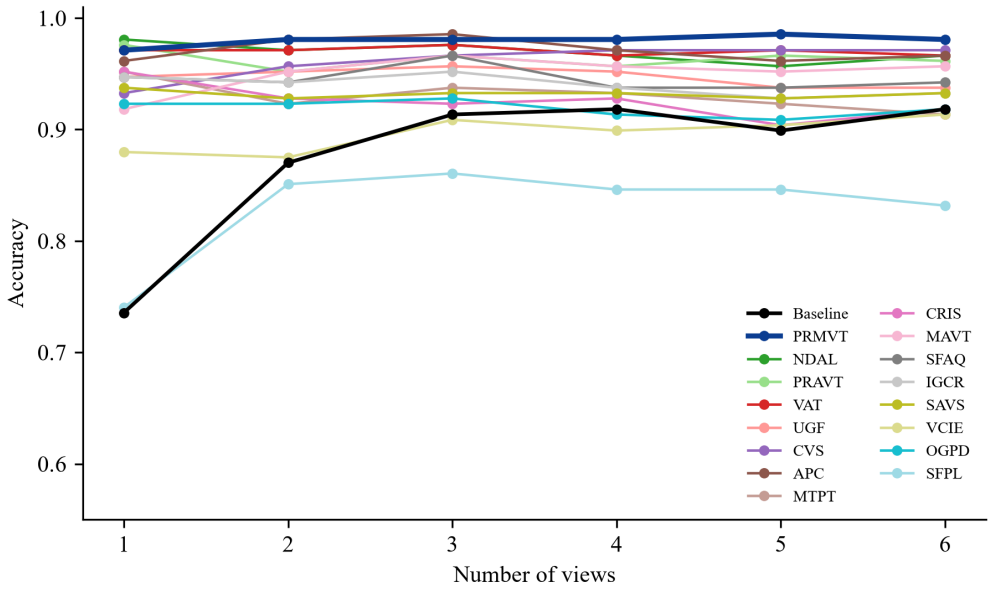


Figure 19: Authentication accuracy against the number of views for the CNN+ViT baseline, PRMVT and the other training variants tried in this project

6.3.2 Controlled Comparison: What Causes the Gain

PRMVT differs from the baseline in several ways at once (fusion head, batch normalisation per view count, auxiliary losses, prefix sampling). To isolate the cause, the same network was trained four ways, with fixed six-view or prefix sampling and with shared or per-view-count batch normalisation, on three seeds each (Table 6.3.2.1).

Table 6.3.2.1: Same network, four ways of training it: test accuracy (%) on 208 notes, average ± standard deviation over three training runs (seeds 42–44). "Normalisation" is batch normalisation (BN).

How the network was trained	1 view	2 views	4 views	6 views
Always 6 views; one normalisation for all view counts	71.2 ± 11.4	73.1 ± 17.1	90.5 ± 2.2	98.1 ± 0.5
1 to 6 views; one normalisation for all view counts	**98.2 ± 0.7**	**98.4 ± 0.3**	**98.6 ± 0.5**	**98.7 ± 0.3**
Always 6 views; separate normalisation for each view count	87.2 ± 9.7	92.8 ± 6.3	95.7 ± 4.6	99.0 ± 0.0
1 to 6 views; separate normalisation for each view count (PRMVT)	96.5 ± 1.5	98.4 ± 0.6	97.6 ± 0.8	98.1 ± 1.0

Prefix sampling alone is the cause. With shared batch normalisation it beats fixed-view training at one view on every seed (exact McNemar $p = 5.4 \times 10^{-20}$, 2.1×10^{-22} and 9.3×10^{-10}), and there is no difference at six views ($p \geq 0.22$). Fixed-view training is also unstable: its number of correct notes at one view ranges from 131 to 194 across seeds, while prefix training stays between 197 and 206. The reason is that six-view training never constrains what the network does with one view, so the result depends on the random initialisation. The simplest

variant, prefix sampling with shared batch normalisation, is the most accurate and was chosen on the validation split as the recommended model.

The effect was also tested outside currency. On MVP-N [73] with an attention-fusion head, prefix training raises one-view accuracy from 40.7% to 48.0% and six-view accuracy from 75.8% to 82.2%; with a concatenation head it does not help at one view (48.7% against 49.7%) and costs 8.6 points at six views. On ten rendered ModelNet classes [72] both trainings give 64% at one view and 77% at six. The benefit therefore depends on the fusion head and is not universal.

6.3.3 Comparison with Standard Backbones

Eighteen training variants were run with three seeds each; none beats the shared-BN prefix network. More importantly, frozen ImageNet backbones with a linear classifier per view reach the same level on this split: ResNet-50 [101] 98.1% at one view and 99.0% at six, DenseNet-121 97.6% and 99.0%, MobileNetV2 96.6% and 98.6%. In 60 paired McNemar tests between these probes and our networks, no difference was significant. **No state-of-the-art claim is made on JaalTaka.** The contribution on this split is the explanation and removal of the view-count collapse of jointly trained fusion networks, not a higher number. That a frozen general-purpose backbone reaches 98–99% is itself a sign that the split is easy, which the next section explains.

6.3.4 Repeatability over Seeds

PRMVT was trained with three random seeds (42, 43, 44). The mean accuracy was 96.5% (standard deviation 1.6 points) with one view and 98.1% (standard deviation 1.0 point) with six views; the lowest mean over the six view counts was 96.5% (Table 6.3.4.1). The result is therefore not a lucky seed, although three seeds is a small sample.

Table 6.3.4.1: PRMVT accuracy over three seeds (mean ± standard deviation)

Photos (views) given	1	2	3	4	5	6
Average accuracy (%)	96.47	98.40	98.08	97.60	98.56	98.08
Spread between runs (standard deviation, points)	1.55	0.56	0.48	0.83	0.48	0.96

6.3.5 Other Metrics and Calibration

For the six-view operating point of the fixed-count fusion model, the improvement over the baseline is consistent across metrics: accuracy 0.966 vs. 0.918, macro-F1 0.966 vs. 0.914, balanced accuracy 0.965 vs. 0.906, ROC-AUC 0.989 vs. 0.971, and expected calibration error (ECE) 0.030 vs. 0.079. A lower ECE means that the confidence reported by the model matches its real accuracy better, which is important if the glass tells the user “probably counterfeit”. For PRMVT, the six-view ECE without any calibration is 0.015, so its confidence is already well matched to its accuracy; calibration maps fitted on the validation set (temperature scaling and a quality-aware map) do not improve on it (0.017–0.018). An earlier uncalibrated value of 0.061 was caused by the numerical fault described above, which reported a probability of 0.5 for notes the model was certain about. The confusion matrices (Figure 20) show that PRMVT mainly removes the baseline’s errors on counterfeit notes that were accepted as genuine (a fraction of 0.07 of all test notes for the baseline, 0.01 for PRMVT).

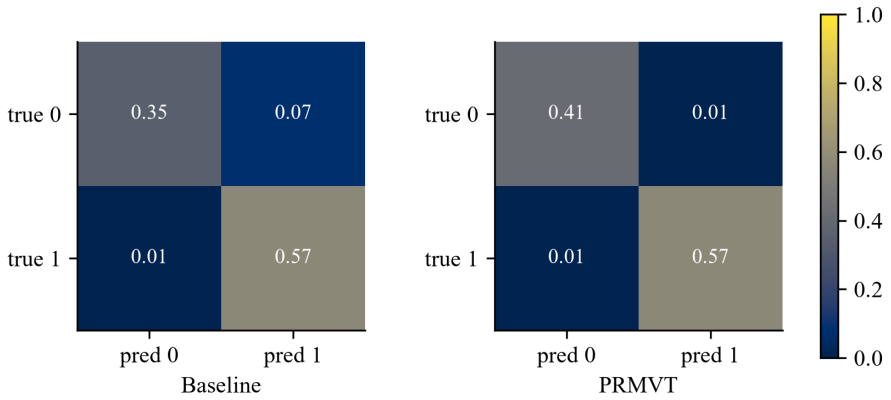


Figure 20: Normalised confusion matrices of the baseline and PRMVT at six views (0 = counterfeit, 1 = genuine)

6.3.6 Robustness to Image Corruptions

The models were tested under 13 image corruptions at several strengths (Figure 21). PRMVT is almost unaffected by Gaussian and motion blur, JPEG compression, scale change and sensor noise. It is clearly sensitive to low light, strong brightness, rotation, perspective and especially occlusion: at an occlusion of 55% of the image, six-view accuracy fell to 51.9%, which is worse than the baseline (81.3%). Fine-tuning with low-light augmentation raised accuracy at the darkest setting (brightness factor 0.2) to 93.75% while keeping 98.6% on clean images, but a first, short occlusion fine-tune did not fix occlusion (68.3%, with clean accuracy falling to 90.9%). An analysis on the validation set showed why: under occlusion PRMVT called most genuine notes counterfeit, because a covered region looks like a missing security feature, and the earlier fine-tunes had kept the CNN frozen and trained only briefly. A longer fine-tune in which every view is randomly covered and the last CNN blocks can adapt, with the epoch chosen on the validation set, raised accuracy at 55% occlusion to 88.5% (87.0% ± 2.2 over three seeds, 88.9% for their average) while clean accuracy stayed at 98–99%. This is still below 90%. If the model is allowed to answer only when it is at least 99% confident and otherwise asks the user to reposition the note, it answers about half of the occluded notes with 99.0% accuracy, and wrong verdicts fall from 11.1% to 0.5% of occluded notes. These tests use a black rectangle; a real finger is neither black nor rectangular, so occlusion by fingers remains the main open failure mode.

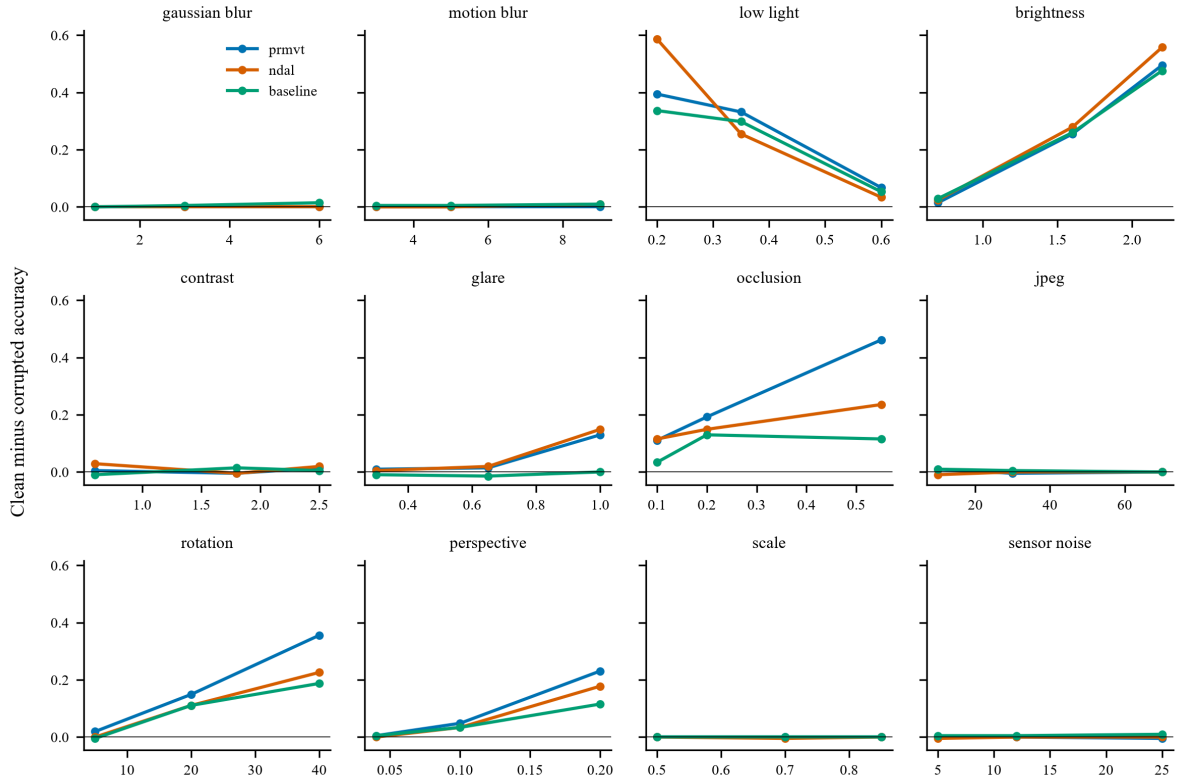


Figure 21: Accuracy drop (clean minus corrupted accuracy) at six views under 12 corruptions for PRMVT, the NDAL variant and the baseline

6.4 Counterfeit Detection on Unseen Prints

The serial-number audit (Section 5.3.1) showed that most test counterfeits of the note-disjoint split are copies of prints that are also in the training data. Three checks measure how much this matters.

Seen and unseen serials. On the note-disjoint test split, PRMVT with one view catches 67 of the 68 counterfeits whose serial occurs in training (98.5%) and 17 of the 19 whose serial does not (89.5%). With 19 notes this is suggestive, not proof.

Masking the serial. When the two serial boxes are covered at test time, accuracy falls by 3 to 5 points (one view: 97.1% to 92.3%), and genuine notes lose about as much as counterfeits. Counterfeits with a seen serial are still caught (65 to 68 of 69). The model is therefore not simply reading a memorised serial; it has learned the look of the known prints.

Print-disjoint split. The decisive test trains and tests on the split of Section 5.3.2, in which every test counterfeit comes from a print never seen in training (Table 6.4.0.1).

Four findings follow.

1. **Accuracy falls by about 8 points.** The prefix network scores 98.2–98.7% on the note-disjoint split and 89.3–89.9% on unseen prints. The frozen ResNet-50 falls from 98–99% to 84–85%.
2. **The watermark recovers about 5 points.** Combining the network score with the watermark score, with a combiner fitted on validation data, gives 94.4% at one view and 95.0% at six, with almost no variation across seeds. The watermark is a physical property of a genuine note and does not depend on which print a counterfeit comes from.
3. **A properly fine-tuned ResNet-50 is as good.** A first fine-tuned ResNet-50 baseline scored 89.9% at six views, and the hybrid beat it. That baseline had been trained on views

Table 6.4.0.1: Accuracy on unseen counterfeit prints: 222 test notes, 101 counterfeit, mean ± standard deviation over three seeds

Counterfeit checker	**Accuracy with 1 view**	**Accuracy with 6 views**
ResNet-50, ready-made and not retrained, with a simple classifier on top (linear probe)	85.1%	83.8%
Our network trained with 1 to 6 views (prefix training, this work)	89.9 ± 0.7%	89.3 ± 1.3%
ResNet-50 retrained on our data, from photos at a quarter of their resolution (a fault)	92.0 ± 2.3%	89.9 ± 4.4%
ResNet-50 retrained on our data, from full-resolution photos	94.9 ± 1.8%	94.4 ± 2.6%
Watermark check alone (MobileNetV2; the 197 notes where the watermark was found by template matching)	92.9% (AUC 0.976)	
Watermark check, with a learned model that finds the watermark (all 222 notes)	92.6 ± 0.5%	
Combined: our network + watermark check	**94.4 ± 0.5%**	**95.0 ± 0.0%**

decoded at a quarter of their resolution because of a fault in an image cache. Trained on full-resolution views it reaches 94.4–94.9%, level with the hybrid. Per seed, the exact McNemar test between the hybrid and the quarter-resolution baseline at six views gives $p = 2.7 \times 10^{-5}$, 0.035 and 0.51; against the full-resolution baseline the hybrid is not better, and adding that baseline to the hybrid does not help (94.6%). We therefore claim a tie, not a win. The two differ in their errors: at six views the hybrid misses 7 of 101 counterfeits on every seed and calls 4 of 121 genuine notes counterfeit, while the fine-tuned ResNet-50 misses 7, 9 and 18 and calls 1 genuine note counterfeit. The hybrid is the more stable of the two.

4. **A serial blacklist is useless against new prints.** It catches none of the unseen-serial counterfeits, by construction.

The test split contains about 24 distinct counterfeit prints. At the level of prints, the uncertainty of every figure in Table 6.4.0.1 is roughly ±12 points, so differences of a few points between methods cannot be established with this dataset.

6.4.1 Learned Watermark Localizer

The watermark window was first found by registering the photograph to a template. Registration fails on 25 of the 222 test photographs (11%). The learned localizer replaces it (Table 6.4.1.1).

On the notes both methods can read, the localizer and registration do not differ (McNemar $p = 1.0$ on every seed). The localizer adds the 25 notes on which registration fails, so coverage rises from 197 to 222 of 222 at the same accuracy. It is also faster: 15.9 ms against 97.9 ms on the laptop CPU, and 50 ms against 372 ms on the Raspberry Pi 5. The application path (localizer in ONNX and the INT8 classifier) found a window on all 222 test photographs, scored 93.2%, and made the same decision as the research path on 99.1% of them. The localizer was trained on one currency, with labels derived from registration; that it transfers to another currency has not been shown.

Table 6.4.1.1: Learned watermark localizer on the print-disjoint test split, three seeds

Measure	Average of 3 runs
How far the found watermark corners are from the true corners (as a share of the photo size)	0.025
Overlap with the window found by template matching (median IoU; 1 = identical)	0.927
Accuracy with the learned window, on the 197 notes template matching could handle	92.6%
Accuracy with the template-matching window, same 197 notes	92.9%
Accuracy on the 25 notes template matching could not handle	93.3%
Accuracy on all 222 test notes	**92.6 ± 0.5%**
Genuine notes called counterfeit (of 121) / counterfeit notes passed as genuine (of 101)	5.0 / 11.3

6.4.2 Watermark Classifiers Compared, Including DMWR

Several watermark classifiers were trained on the same back-lit windows and scored on the same 197 print-disjoint test notes on which the window was registered (Table 6.4.2.1). Every model was chosen on validation data and the test was read once; trained models were repeated with seeds 42, 43 and 44.

Table 6.4.2.1: Watermark classifiers on the 197 registered test notes with unseen counterfeit prints

Watermark classifier	Parameters	File size	Accuracy	AUC
MobileNetV2, fine-tuned (used in the glass, 2.6 MB as INT8)	2.23 M	9.15 MB	92.9%	0.976
Four MobileNetV2 models averaged, threshold 0.924 chosen on validation	4 × 2.23 M	–	93.4%	0.978
EfficientNet-B0 (best of ten architectures by validation AUC)	4.01 M	–	91.2 ± 1.6%	0.964
DMWR (this work, Section 3.4.7)	**47,818**	**0.21 MB**	**91.5 ± 0.3%**	**0.950**
DMWR, quotient-image stream + four numbers only	–	–	88.0 ± 3.3%	0.938
DMWR, residual stream + four numbers only	–	–	85.8 ± 5.3%	0.935
DMWR's four numbers + four simple image features, logistic regression	9	–	87.3%	0.906
Small CNN + transformer, trained from scratch	0.11 M	–	86.3 ± 0.9%	0.931

DMWR matches a network 46 times its size. DMWR reaches 91.5% (AUC 0.950) with 47,818 parameters, 1.4 points from the fine-tuned MobileNetV2 with 2.23 million. On the same notes the difference is not statistically significant (exact McNemar $p = 0.375$, 0.688 and 0.375 for the three seeds), while the model file is about 44 times smaller (0.21 MB against 9.15 MB). It is also very stable: the three seeds give 91.4%, 91.9% and 91.4%, and it calls only 1, 5 and 2 of the 98 genuine notes "not clear".

Both the portrait and the residual are needed. Each stream on its own reaches 85.8–88.0% and varies by 3–5 points between seeds; together they reach 91.5% with a spread of 0.3

points. The genuine portrait and what is left over after subtracting it therefore carry complementary information, which supports the physical idea behind Equations (3.4)–(3.6). Even the four summary numbers alone, in a logistic regression with nine weights, reach 87.3%. Figure 22 shows the steps on two test notes.

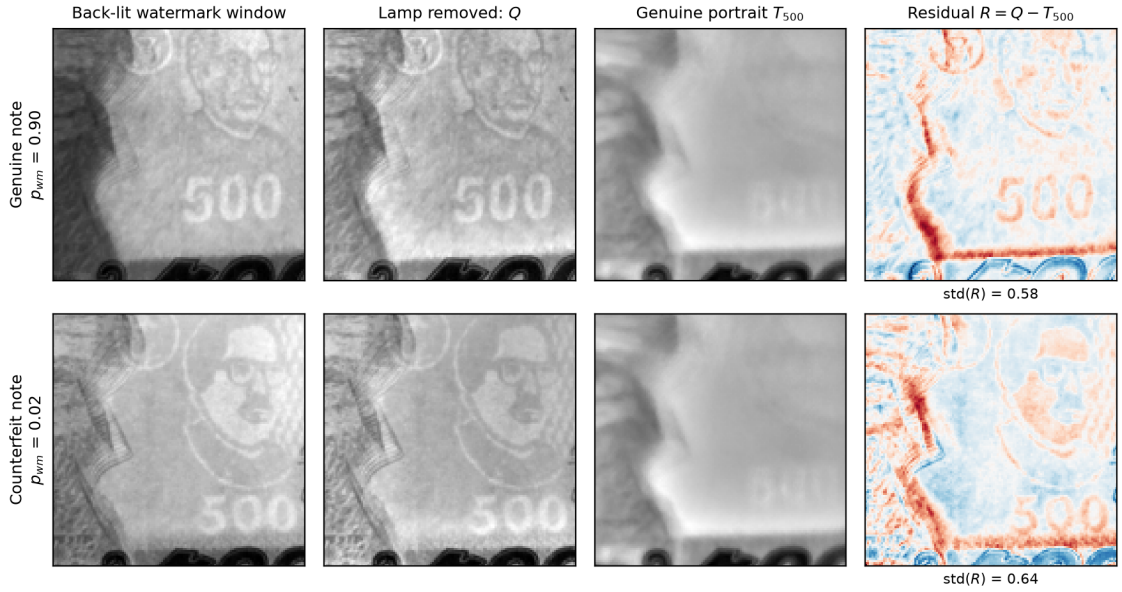


Figure 22: DMWR on two 500 Taka test notes with unseen prints (seed 42). Left to right: the back-lit watermark window; the quotient image Q with the lamp removed; the genuine portrait T_{500} built from training notes only; the residual R (red above, blue below the portrait). On the counterfeit note the printed portrait stands out in the residual.

Larger models and ensembles. A search over ten architectures, ranked by validation AUC, chose EfficientNet-B0 (91.2 ± 1.6%), and an average of four MobileNetV2 models with a validation-chosen threshold reached 93.4%, the highest watermark result in the table; its difference to one MobileNetV2 is one note. Used in the hybrid of Equation (3.2) instead of MobileNetV2, the EfficientNet-B0 watermark gives 94.6%, 95.5% and 94.6% at six views. The MobileNetV2 stays in the glass because it is fast on the Pi and is already exported to INT8 with the same decisions as the full-precision model; DMWR is the light, explainable alternative for even smaller hardware.

6.4.3 UV Security-Fibre Check

The check of Section 3.4.8 was measured on 118 phone photographs of genuine 200, 500 and 1,000 Taka notes under a UV lamp, taken for this project. The Taka detector found the note box on 111 of them. No counterfeit note was available, so the fibre-free side is a patch of the plain surface beside the note in the same photograph.

A first test set the threshold on the first 60% of each value's photographs and tested on the last 40%: 29 of 43 genuine notes were accepted. The last photographs of the 500 and 1,000 Taka notes show the back of the note from further away, which the first part did not contain, so the threshold had been set for a different kind of photograph. The check was therefore evaluated again by blocked five-fold cross-validation over capture order: each block of consecutive photographs is judged with a threshold set on the other four blocks, by the same rule (the largest threshold that accepts at least 95% of the training notes). The fibre finder itself was not changed. Table 6.4.3.1 gives both.

Table 6.4.3.1: UV fibre check on photographs of genuine notes (no counterfeit note was tested)

Test	Genuine notes accepted	200 Taka	500 Taka	1,000 Taka	Plain-surface patches rejected
First test: one split by capture order	29 of 43 (67%)	19 of 19	7 of 16	3 of 8	21 of 21
Blocked 5-fold, threshold from the other blocks	**103 of 111 (92.8%)**	45 of 45	36 of 40	22 of 26	71 of 71

With the blocked protocol the check recognises 92.8% of the photographs of genuine notes (95% Wilson interval 86.4–96.3%). In the first test, halving the brightness of the photographs left the result unchanged (29 of 43), while a slight defocus lowered it to 13 of 43: the fibres are thin, and the UV picture has to be close and sharp. The thresholds are low (4 to 10 fibres), and the only fibre-free examples so far are smooth surfaces.

A simulated printed copy, and a repair. To see what the check does with print, fibre-free printed sheets were simulated: normal-light photographs of genuine notes (security fibres are invisible in normal light), given the lamp's measured colour and brightness, the camera noise measured on the UV photographs, and JPEG compression. Noise alone made the check accept none of 120 sheets. JPEG compression at quality 90 made it accept 112 of them (93%): compression stores colour more coarsely than brightness, and the colour smear at the edges of print was counted as fibres. The 92.8% above therefore measured that thin coloured structures are found on a note, which a printed copy has as well. A brightness condition did not help (balanced accuracy 0.50). The colour condition of Equation (3.11) does. With a colour threshold of 0.2, a genuine note shows a median of 5 strongly coloured fibres instead of 41 counted structures. Table 6.4.3.2 gives the nested measurement: for each fold, the colour threshold and the fibre count were chosen on four blocks of genuine photographs and on development sheets, and applied to the fifth block and to held-out sheets.

Table 6.4.3.2: UV fibre check before and after the colour condition. Genuine side: real photographs. Fibre-free side: simulated printed sheets; no counterfeit note was photographed.

Measure	First check	With the colour condition
Genuine photographs accepted (blocked 5-fold)	103 of 111 (92.8%)	95 of 111 (85.6%; 77.9–90.9)
200 / 500 / 1,000 Taka	45 of 45, 36 of 40, 22 of 26	44 of 45, 31 of 40, 20 of 26
Held-out simulated fibre-free sheets rejected	4 of 240 (1.7%)	88.6% (mean of five folds)
With the setting now in the glass: held-out sheets rejected	–	223 of 240; with JPEG quality 50: 48 of 60

The repaired check gives up seven points of genuine acceptance and, in the simulation, rejects about nine in ten fibre-free sheets. A single strongly coloured fibre decides, which is fragile; it is what these hand-held, already compressed phone photographs support. This is still not evidence about counterfeit notes: a real counterfeit's ink and paper may behave differently under the lamp, and none has been photographed.

6.4.4 Capture Guide

The brightness and sharpness limits of the capture guide were set on validation photographs and checked on the 222 back-lit test photographs (Table 6.4.4.1). Dark and defocused frames are rejected completely, and 96% of the clean photographs pass. The limits come from JaalTaka photographs, not from the glass camera, and have to be set again for it.

Table 6.4.4.1: Capture guide on 222 back-lit test photographs

Photo condition	Accepted	Rejected as too dark	Rejected as blurry
Good photo, unchanged	213 (95.9%)	4	5
Made darker (25% of the brightness)	0	222	0
Out of focus (blur $\sigma = 4$)	0	4	218
Shaken (motion blur over 21 pixels)	11 (5.0%)	4	207

6.5 The Verdict on the Glass

6.5.1 Authentication on Whole-Note Photographs

The glass applies the authenticator to one crop of the note found by the detector, not to the close-up views it was trained on. This deployed path was scored on photographs that were not used for training or selection (Table 6.5.1.1).

Table 6.5.1.1: Deployed genuine/counterfeit verdict on whole-note photographs (one crop, threshold 0.5)

Photos	Notes checked	Wrong answer (with 95% range of uncertainty)
Bangla Money, genuine notes	1,464	47.7% (45.2–50.3)
BanglaTaka raw photos, genuine notes	441	59.2% (54.5–63.7)
Counterfeit-currency set, genuine notes	1,187	20.4% (18.2–22.8)
Counterfeit-currency set, counterfeit notes	75	40.0% (29.7–51.3)

A genuine note called counterfeit, or a counterfeit called genuine, is the most harmful error for a blind user. The high close-up accuracy in Table 6.3.1.1 does not carry over to whole-note crops, so this verdict was switched off.

Five ways to repair it were tried (Table 6.5.1.2). Cutting the whole note into windows shaped like the training views improves the ranking of genuine and counterfeit notes (AUC 0.64 to 0.83), and a frozen ResNet-50 on such windows ranks best (0.966), but at a threshold of 0.5 every one of them still calls too many genuine notes counterfeit or passes too many counterfeits. Fine-tuning the authenticator on synthetic whole notes looked better on synthetic test data (accuracy 87.3% against 79.0%) but worse on real photographs (AUC 0.689), and at its own validation threshold it passed 14 of 20 real counterfeit photographs as genuine. Under the rule fixed beforehand it was not deployed.

Table 6.5.1.2: Genuine whole-note photographs called counterfeit at $p = 0.5$, and ranking quality (AUC) on the counterfeit set

Counterfeit checker	Genuine notes wrongly called counterfeit			How well it separates genuine from counterfeit (AUC, 1 = perfect)
	Counterfeit-currency set	Bangla Money	BanglaTaka	
PRMVT, the whole note given as one view (first version)	27.1%	45.1%	61.5%	0.640
PRMVT, the note cut into four views like the training photos	5.6%	43.4%	60.2%	0.829
Ready-made ResNet-50 with a simple classifier, on the cut views	1.6%	12.5%	16.5%	0.966
A classifier trained on the counterfeit-currency set itself	0.1%	26.4%	86.7%	0.886
PRMVT retrained on computer-made whole notes	3.5%	34.2%	45.0%	0.689
Safe policy used in the glass (never says "counterfeit")	**0%**	**0%**	**0%**	–

Table 6.5.2.1: Deployed verdict policy: "likely genuine" above τ, otherwise "check by hand"

What was checked	Result	95% range of uncertainty
Genuine notes wrongly called "counterfeit"	0 (the glass never says it)	–
Counterfeit notes wrongly called "likely genuine": JaalTaka test, 1 to 4 views	0 of 88	0–4.2%
Counterfeit notes wrongly called "likely genuine": photos of whole notes, through the glass program	0 of 20	0–16.1%
Genuine notes confirmed as "likely genuine": JaalTaka test	69 of 120 (57.5%)	
Genuine notes confirmed: whole notes, counterfeit-currency set	192 of 1,138 (16.9%)	
Genuine notes confirmed: whole notes, Bangla Money	1 of 277 (0.4%)	
Genuine notes confirmed: whole notes, BanglaTaka	6 of 296 (2.0%)	
Wrong answers in bad light, when too-dark or blurry photos are rejected first	0 of 208 in each condition	0–1.8%

6.5.2 Safety of the Deployed Policy

The policy of Section 3.4.5 was fixed on validation data (0 of 88 validation counterfeits passed, 66 of 120 validation genuine notes confirmed) and then scored once (Table 6.5.2.1).

The policy is safe on everything that was measured: no counterfeit was passed, and in 1,889 runs of the application the word “counterfeit” was never spoken. The price is usefulness. On close-up views it confirms 57.5% of genuine notes, but on whole-note photographs taken by other people it confirms almost none (0.4–2.0%) and sends nearly every note to a check by hand. The safety evidence on whole notes is also narrow: it rests on about four physical counterfeit notes, and the highest counterfeit score (0.99567) lies close to the threshold (0.99959). The glass is therefore safe but, for whole notes, seldom decisive. Without the quality gate, confidence alone is not enough under bad light: at a brightness factor of 0.35, 59 of 208 one-view verdicts were wrong; with the gate, none.

6.5.3 Other Training Variants and Negative Results

Many other training variants were tried on the same split (Figure 19); eighteen methods have all three seeds. Several of them reach 95–98% at one view, for example a focal-style loss variant (NDAL) [112] with 98.1% at one view, so the accuracy alone does not single out one algorithm; the common factor is that all of them see varying numbers of views during training. Some variants failed: a simulated federated version stayed at 74.0% with one view and below the baseline from two views on, and one retrained variant became worse than its first version. We also tried learned policies that choose which view to capture next and when to stop (rules based on image quality, confidence, diversity and uncertainty, and a cost-aware rule). On PRMVT, averaged over three seeds, none of them beat simply taking the views in their fixed order (96.5% with one view and 97.6–98.6% with two to six): the learned rules picked a weaker first view and reached 89.3–90.2% with one view, and the cost-aware rule with early stopping stopped after one view at 89.3%. A prefix stopping rule on the fixed order reached 97.1% with an average of 1.01 views, which is simply because PRMVT is already accurate with one view. Earlier, much lower figures for these policies (down to 42%) were caused by evaluation faults that have since been fixed, not by the policies themselves. Values from a single seed also shrank when more seeds were added (one variant fell from 98.1% to 96.8% at one view), and about 110 training runs have read this test split, which is a further reason not to rank methods by small differences on it. These negative results are reported so that the method is not presented as better than it is.

6.5.4 Speed

On the RTX 3050 laptop GPU, one six-view authentication takes a median of 42.6 ms (95th percentile 69.5 ms). On the Raspberry Pi 5 it takes 37 ms with one view and 90 ms with six (Section 6.9.2).

6.6 Emotion Recognition Performance

Several models were trained and compared on the official RAF-DB test set (Table 6.6.0.1). The EfficientNet-V2-S model with flip test-time augmentation gave the best accuracy, 86.5%, and is used in the system.

The accuracy is 86.5% and the macro-F1 is 0.795 (Table 6.6.0.2). Happy, surprise, neutral and sad are recognised well; fear and disgust are the weakest classes because they have the fewest training images and are often confused with sad and neutral (Table 6.6.0.3). The published state of the art on this split is about 92% (POSTER V2 [113]), so our model is a reasonable but not state-of-the-art recogniser. For adaptive speech, the most important decision is whether a person looks stressed (sad, fear, anger, disgust) or not, which is less demanding than the full seven-class task.

Table 6.6.0.1: Emotion recognition models compared during development

Model	Tested on	Accuracy	Used in the glass?
FER+ (ready-made ONNX model)	FER-2013 held-out photos (5,741)	57.9%	Only as a back-up
MobileNetV3-Large	RAF-DB official test (3,068)	81.0%	No (replaced)
ViT face-expression model, retrained	RAF-DB official test	84.3% (best point)	No
EfficientNet-B2 (pre-trained on AffectNet), retrained	RAF-DB official test	85.2% (then got worse: over-fitting)	No
EfficientNet-V2-S, also checking the mirror image of each face (flip test-time augmentation)	**RAF-DB official test**	**86.5%**	**Yes**

Table 6.6.0.2: Results for each expression of the EfficientNet-V2-S emotion model on RAF-DB (3,068 test faces). Precision: of the faces given this label, the share that were right. Recall: of the faces with this expression, the share recognised. F1: a balance of the two.

Expression	Test faces	Precision	Recall	F1
Happy	1,185	0.946	0.925	0.936
Sad	478	0.839	0.839	0.839
Angry	162	0.789	0.809	0.799
Surprise	329	0.867	0.872	0.870
Fear	74	0.774	0.554	0.646
Disgust	160	0.671	0.588	0.627
Neutral	680	0.815	0.890	0.851
Average over the 7 expressions	**3,068**	**0.815**	**0.782**	**0.795**

Table 6.6.0.3: Confusion matrix of the EfficientNet-V2-S emotion model on the RAF-DB official test set. Each row is the real expression and each column the model's answer; the bold diagonal counts the correct answers.

Real ↓ / Predicted →	Happy	Sad	Angry	Surprise	Fear	Disgust	Neutral
Happy	**1096**	11	8	12	1	11	46
Sad	14	**401**	5	4	1	14	39
Angry	7	2	**131**	4	4	4	10
Surprise	6	8	3	**287**	5	5	15
Fear	3	8	5	11	**41**	2	4
Disgust	11	15	13	4	0	**94**	23
Neutral	21	33	1	9	1	10	**605**

6.7 Text Recognition Performance

All results in this section are character error rates (CER) on the benchmark of Section 5.5.1. Every choice was made on the validation set; test figures were read once. Earlier OCR figures of this project (3.2% for English and 27.8% for Bangla) were measured on images in which Bangla was drawn without text shaping and are withdrawn.

6.7.1 The Seven Steps

The pipeline was built in seven steps, each tested on validation data (Table 6.7.1.1).

Table 6.7.1.1: How the text-reading pipeline was built, step by step, on the validation set. Error is the character error rate (CER): the share of letters read wrongly, lower is better. “Kept” marks what went into the glass.

Step	What was tried	Error (CER)	Kept
1. Which text to read	EasyOCR alone, words in the order it returns them	17.8%	
	+ words put into lines, top to bottom	15.1%	
	+ words the reader is unsure about removed	5.2–6.3%	
	+ only the main block of text kept (background text ignored)	3.4%	yes
2. Cleaning the picture first	Contrast boost only (CLAHE)	3.4%	yes
	Turning the picture black-and-white (Sauvola / Niblack)	5.2% / 12.5%	
	Straightening a tilted page	8.3%	
3. Other text readers	Tesseract 5 on the main text block	23.4%	
	TrOCR on the main text block	78.0%	
	PaddleOCR (English only; it has no Bangla model)	43.4%	
4. Reader settings	Best of 31 EasyOCR settings	2.8%	yes
5. Correcting the text afterwards	Simple clean-up rules (stray symbols, spacing)	2.3%	yes
	Spelling dictionary	3.3%	
	Letter-sequence language model (KenLM)	2.7%	
	ByT5 AI correction model (it rewrote and added text)	105.4%	
6. Several picture sizes	Read at three sizes and vote	2.9%	
7. Retraining the reader	Reader retrained on computer-made words	2.5–2.7%	

Where the errors were. On photographs of signs the first pipeline had a CER of 45.0%. Inspection showed that the sign itself was usually read correctly; the errors were text read from the background and words in the wrong order. Selecting the main text block removes both, and photo-scene CER falls to 6.4%. This single step gives most of the total gain.

What did not help. Binarisation and perspective correction hurt. Tesseract [27] was the best other engine but had about seven times the error; the available TrOCR models [34] returned unrelated words for Bangla; PaddleOCR has no Bangla model. A spelling dictionary [38] and a character language model [39] both replace correct but uncommon Bangla words by common ones. The ByT5 corrector [40] rewrote and extended the text and took several seconds per line. Reading at three scales costs three OCR passes and is not better than one. Replacing the CRAFT detector by DBNet [31] (3.4%), EAST [30] (18.3%) or a public YOLO text model

(50.6%) inside the same pipeline was also worse than CRAFT (2.3%); the YOLO model was trained on handwriting in another script, so its result says nothing about a detector trained for this task.

6.7.2 Results on the Test Set

Table 6.7.2.1 gives the test results.

Table 6.7.2.1: Text reading on the held-out test set (432 images, three seeds), measured once per method. All numbers are error rates: lower is better.

Text reader	Letter error (CER), all	Letter error, Bangla	Letter error, English	Letter error, signs in photos	Word error (WER)
Glass program before this work	9.3%	11.9%	6.6%	10.7%	15.8%
New pipeline v2 (now in the glass)	**2.4%**	**3.2%**	**1.6%**	**2.2%**	**7.9%**
Tesseract 5 on the text block	16.6%	18.5%	14.7%	32.5%	34.4%
PaddleOCR (English items only)	11.0%	–	11.0%	33.1%	9.5%
Pipeline v2 with the DBNet text finder instead of CRAFT	2.7%	3.5%	1.8%	3.4%	–
Retrained reader (3 training runs; not adopted)	1.8–2.1%	2.5–2.9%	1.0–1.2%	1.4–2.8%	3.9–5.3%

The pipeline reduces the test CER from 9.3% to 2.4%, and for Bangla from 11.9% to 3.2%, at no extra cost in time (median 1,851 ms against 1,923 ms per image on the laptop CPU).

Two results show why choices must not be made on the test set. The CLAHE-only pre-processing was the best on validation by 0.08 points, which is within seed noise, and on test it was worse than the earlier pre-processing (3.3% against 2.4%) until the tuned parameters were added. And the fine-tuned recogniser was worse than the original on validation (2.5–2.7% against 2.3%, because English got worse) but better on test in every seed (1.8–2.1% against 2.4%). It was not adopted: switching to it because of its test result would be choosing on the test set. Fine-tuning remains the most promising next step, with English kept in the training text and with a fresh test set.

These figures describe rendered text. They do not describe the glass camera (Section 6.10).

6.7.3 Text Reading on Real Photographs

To see how far the benchmark figures carry, the pipeline was run once on real photographs: word images from the ICDAR 2019 MLT scene-text collection [114] (street signs, shop fronts and notices, each with its text typed out). 500 Bangla and 500 Latin-script words were drawn at random; words that the collection's own annotators marked as unreadable were left out. Each word was scaled to 400 pixels on its longer side and placed in a 640×480 frame, as the glass would see a sign. Nothing was tuned on these images (Table 6.7.3.1).

The benchmark figures do not carry over to real street photographs: the error is more than ten times higher. Much of it is silence. On a quarter of the Bangla words and on half of the Latin-script words nothing was read at all, which points to the text-finding thresholds and the confidence filter, both chosen on rendered text. Words rotated by 90° are not read (84% and

Table 6.7.3.1: Text reading on real photographs: word images of the ICDAR 2019 MLT collection, read once with the pipeline of the glass

Words	Number	Letter error (CER), 95% range	Word exactly right	Nothing read
Bangla, all	500	41.7% (38.0–45.5)	36.8%	24.6%
Bangla, upright	458	37.8% (34.0–41.9)	39.7%	21.6%
Bangla, all, confidence filter 0.1 (now)	500	**39.2%**	37.4%	17.0%
Latin script, all	500	64.9% (60.9–68.8)	20.4%	48.4%
Latin script, upright and at least 32 pixels high	282	46.8% (41.4–52.4)	32.6%	27.7%
Latin script, all, confidence filter 0.1	500	**62.5%**	20.4%	38.0%
Bangla, all, recogniser trained on real words (now)	500	**28.7%**	47.8%	12.4%
Latin script, all, recogniser trained on real words (now)	500	**50.6%**	30.6%	32.6%
Rendered benchmark, test set (Table 6.7.2.1); now	432	2.4%; 2.3%	–	–

99% error), and many Latin-script words are small or are French, German or Italian, for which the recogniser has no model. For orientation, the BanglaWild benchmark reports 48.4% CER for plain EasyOCR on real Bangla scene photographs [115]; the data differ, so only the order of magnitude can be compared. Reading real signs is therefore the main open problem of the text mode, next to the glass camera.

Settings chosen on real words. Because so many words were left unread, four settings of the text finder and of the confidence filter were compared on a separate real validation sample (150 Bangla and 150 Latin-script words that are not among the 1,000 test words). The rule was fixed beforehand: the lowest error on the real validation words wins, among settings whose error on the rendered validation set stays within 0.5 points. Lowering the confidence filter from 0.2 to 0.1 won (real validation 47.8% against 51.1%; rendered validation 2.58% against 2.14%). EasyOCR's own detector defaults were worse on both kinds of data. Read once on the 1,000 test words, the new setting lowers the error from 41.7% to 39.2% for Bangla and from 64.9% to 62.5% for Latin script, and the share of unread words from 24.6% to 17.0% and from 48.4% to 38.0% (bold rows of Table 6.7.3.1). The gain is real but small: the words that are now read are often read wrongly. On the rendered test set the pipeline as it now is in the glass (greedy decoder, confidence filter 0.1) scores 2.5% CER (Bangla 3.3%, English 1.7%), practically unchanged. Thresholds are now exhausted; the remaining error lies in the recogniser.

A recogniser trained on real words. EasyOCR's Bangla and English recogniser was therefore fine-tuned on real training words from the same collection (none of them among the validation or test words), mixed with rendered words, three times with different seeds. Each model was read through the glass's own pipeline. The fully fine-tuned models lowered the real-validation error from 47.8% to 34–35%, but raised the rendered-validation error from 2.58% to 3.3–3.8% (printed English to 4.9–5.8%): they learned the street words and forgot part of the clean printed text, and so failed the 0.5-point rule. Blending the weights half and half with the released ones, $\theta = \frac{1}{2}\theta_{\text{released}} + \frac{1}{2}\theta_{\text{fine-tuned}}$ (a rule added after the first model failed and before any other was

scored), kept most of the gain without the loss: real validation 36.7–37.2%, rendered validation 2.33–2.63%, all three within the rule. The one with the lowest real-validation error (seed 44) was chosen. A retry with the picture turned by 90° when nothing is read left fewer Latin words unread but did not lower the error, and was not adopted. Read once on the 1,000 test words, the chosen recogniser lowers the error from 39.2% to **28.7%** for Bangla and from 62.5% to **50.6%** for Latin script, and on the rendered test set from 2.46% to 2.27% (bold rows of Table 6.7.3.1). It has the same network as the released recogniser, so it costs no time, and it is now the glass's default.

How much the picture is enlarged. The glass enlarges a 640-pixel camera picture to 1,200 pixels before the text detector, whose time grows with the number of pixels. Widths of 1,040, 960 and 880 pixels were scored with the full-precision and the 8-bit detector. The validation rule chose 880 for both, but the single test read did not confirm it (rendered test error 3.15% and 3.12% against 2.46% at 1,200, printed Bangla 4.9%), and the validation error went up and down with the width in no order. The width therefore stays at 1,200; 880 remains a faster option whose speed on the Pi is still to be measured. This is the one place where a rule fixed beforehand was not followed, and the departure is towards the more accurate setting.

6.7.4 Handwritten Bangla Character Recognition

The CNN trained on Ekush reached 90.0% test accuracy on 38,608 characters from 305 unseen writers (best validation accuracy 91.1%). Accuracy was 94.3% for digits, 94.2% for modifiers, 93.1% for vowels, 89.9% for consonants and 87.7% for conjuncts. The lower accuracy for conjuncts confirms that compound characters are the hardest part of Bangla recognition, which is also the main source of the OCR errors above.

6.8 Object Detection Performance

An earlier evaluation on a random subset of 250 COCO val2017 photographs gave mAP@0.5 = 0.604 for the COCO-pretrained YOLOv8s. On all 5,000 photographs the same model and code give 0.551, so the subset figure was optimistic (Table 6.8.0.1). The 30 object types that matter most to a blind user (people, vehicles, furniture, bottles, cups, phones and so on, fixed before the run) score the same as all 80.

Table 6.8.0.1: Object detection on all 5,000 COCO val2017 photographs (mAP@0.5, predictions with confidence of at least 0.15)

Model	Picture size 640	Picture size 320	30 blind-user object types, size 640
YOLOv8n (timed on the Pi 5)	0.453	0.342	0.463
YOLOv8s (first choice in the glass)	0.551	0.441	0.553
YOLOv8m	0.607	0.513	0.604

The measurement also exposed a mistake in the object mode. It shrank every camera frame to 320×240 before detection, although the detector enlarges every picture to its 640-pixel input, so the shrink cost detail and saved no time. Through the glass's own path, on the same 5,000 photographs brought to the camera's size, mAP@0.5 was 0.493 with the shrink and 0.521 without it, with 10% more detections above the announce confidence. The frame is now passed on unshrunk. Buses, dogs, cats and people are detected best (AP@0.5 of 0.73 to 0.86 for YOLOv8s),

books, backpacks and handbags worst (0.17 to 0.23). The object mode reduces the effect of false alarms by using a confidence threshold of 0.50 and by announcing at most three objects per scan. Precision of the spoken announcements in real indoor use has not yet been measured.

6.9 Processing Speed

6.9.1 Laptop

Table 6.9.1.1 summarises the speed on the development laptop. On the laptop CPU (AMD Ryzen 7 5800H, all three formats measured in the same run), the Taka detector takes a median of 109 ms per 640×640 image with PyTorch weights, 139 ms with FP32 ONNX Runtime and 101 ms with the INT8 ONNX model, which is also the smallest file.

Table 6.9.1.1: Measured processing speed on the development laptop

Model	Runs on	Time per picture	File size
Taka detector YOLOv8s (PyTorch)	RTX 3050 laptop graphics card	13.9 ms	22.5 MB
Taka detector YOLOv8s (PyTorch)	Laptop processor (Ryzen 7 5800H)	109 ms	22.5 MB
Taka detector, ONNX format (full precision)	Laptop processor	139 ms	44.8 MB
Taka detector, ONNX format, compressed (INT8)	Laptop processor	101 ms	18.0 MB
Counterfeit checker PRMVT, 6 views	RTX 3050 laptop graphics card	42.6 ms	16.6 MB
Watermark check, with the learned watermark finder	Laptop processor	15.9 ms	4.3 + 2.6 MB
Watermark check, with template matching (SIFT)	Laptop processor	97.9 ms	2.6 MB
Text reading, pipeline v2	Laptop processor	1,851 ms	–

6.9.2 Raspberry Pi 5

Every mode was timed on a Raspberry Pi 5 Model B (8 GB, Debian 13) on 1 October 2026, with 100 calls per module after warm-up, followed by a 30-minute run in which all modules were called in turn (Table 6.9.2.1). The benchmark script marks its output with the device model, so a laptop run cannot be mistaken for a Pi result.

System behaviour. Memory use was 1.19 GB after the first models loaded and peaked at 1.48 GB with everything loaded, with no growth over the 30 minutes (1,229 calls). The CPU temperature was 57.9 °C idle and peaked at 77.1 °C; the throttling flag stayed at zero throughout. Under the mixed load, the worst single call took up to about four times the isolated median (currency mode: 1,206 ms against 327 ms).

Currency. INT8 quantisation halves the detector time on the Pi (254 ms against 507 ms). From button press to the end of the spoken sentence the currency mode takes about 2.3 to 2.9 seconds, of which 2.0 to 2.6 seconds is the speech itself. An earlier write-up gave 6.5 seconds; it had used the duration of a longer sentence from the text mode and was corrected.

Table 6.9.2.1: Latency on the Raspberry Pi 5 (100 calls per module)

Part of the glass	**Typical time (median)**	**Slow case (95 of 100 calls are faster)**
Currency mode, whole job (find the note + safe counterfeit check)	327 ms	560 ms
Taka detector, compressed INT8 ONNX (used on the Pi)	254 ms	339 ms
Taka detector, PyTorch / full-precision ONNX (for comparison)	507 / 485 ms	603 / 575 ms
Watermark check, learned watermark finder (used on the Pi)	50 ms	62 ms
Watermark check, template matching with SIFT (for comparison)	372 ms	854 ms
Counterfeit checker, 1 view / 6 views	37 / 90 ms	57 / 106 ms
Object mode (YOLOv8n)	207 ms	217 ms
Emotion recognition	48 ms	52 ms
Text reading of one picture (EasyOCR, canvas 2,560)	18.7 s	20.4 s

Text reading is slow. One capture takes about 19 seconds. The time is spent in the text detector, not in decoding: a greedy decoder was not faster (20.6 s). A smaller detector canvas of 640 pixels was about three times faster (20.2 s to 6.9 s on one frame), at about one point more Bangla error (25.6% against 24.5% on the selection set, raw CER before the pipeline). The faster setting did not meet the margin fixed beforehand, so it is offered as an option and is not the default. The Pi figure was measured before pipeline v2 was added; v2 costs no extra time on the laptop but has not been timed again on the Pi.

Faster reading of whole pages. For pages, the recogniser's share of the time grows with the number of lines. On 12-line pages photographed at 640×480 and 1,280×720, read on the laptop CPU limited to four threads like the Pi 5, a greedy decoder reads a page in 6.5 s against 8.9 s with beam search (27% faster), with the same validation error (2.14% against 2.18%), and was adopted. Reading at the camera's own size was faster still (5.3 s) but added 0.74 points of validation error and was rejected. In addition, pages are now spoken line by line (Section 3.4.1): the first line is ready after about 5.5 s, while speaking the page takes much longer than reading it, and the validation error stays at 2.14%. These page timings are from the laptop and still have to be repeated on the Pi.

A faster text detector. Because the time of a capture is in the text detector, the detector (CRAFT) was exported to ONNX and run with ONNX Runtime, as was done for the Taka detector. On the laptop CPU limited to four threads, the detector then takes 2.6 s instead of 5.6 s on a 1,200×900 picture, a 12-line page 5.6 s instead of 10.0 s, and one benchmark picture 1.2 s instead of 2.3 s, with the same validation error (2.14%); its outputs differ from PyTorch's by about 10^{-7}. It is now used whenever its file is present. A version quantised to 8-bit integers is faster again (detector 1.3 s, page 4.3 s) but raises the validation error to 2.79%, 0.65 points more, which is over the 0.5-point rule; a larger calibration set (128 images) brought this to 0.57 points, still over the rule. It is offered as an option and is not the default. The laptop was slower in this run than in the one above, so these figures compare with each other only. None of them is a Pi measurement.

Speech. In the first Pi run, English with Piper took 3.3 seconds for the test sentence against 4.9 seconds with espeak-ng, and Bangla could not use Piper. After the speech engine was rebuilt

on 14 October (Section 6.9.3), Bangla is spoken with a Piper voice as well; that engine has been timed on the laptop only.

6.9.3 Voice Output and Voice Commands (14 October 2026)

Voice output. After the second Pi run the voice was reported as broken up. Two causes were found in the code. First, speech was synthesised and played at the same time, so the player ran out of samples whenever text reading held all four cores. Second, the English voice produces 16 kHz audio and was played at 22.05 kHz, that is 38% too fast and too high. Each sentence is now rendered completely and then played at the sample rate of its voice, with 150 ms of silence before it so that the sound device does not swallow the first syllable. Bangla is spoken with a Piper voice through Piper's Python package, which does not have the fault of the stand-alone program. On the laptop, a Bangla sentence of 1.5 to 4.3 s is rendered in 0.2 to 0.5 s once the voice is loaded (loading takes about 3 s and is done in the background at start-up). As a rough check of intelligibility, six Bangla sentences spoken by this voice were given to an online Bangla speech recogniser: two came back exactly, two with small differences (a numeral, a spelling), and two with wrong words. The new engine has not been heard or timed on the Pi yet; if rendering proves too slow there, one setting returns the glass to espeak-ng.

Voice commands. Every one of the 39 command phrases was spoken by three synthetic voices (one Piper voice and two Windows voices), clean and with white noise added, and given to the recogniser (Table 6.9.3.1). No command was ever mistaken for another one. Thirty ordinary English sentences per voice, several of them containing command words, and ten Bangla sentences were then played as speech that is not a command. This is a check with synthetic speech on a laptop: people speaking into the glass's own microphone in a room have not been tested.

Table 6.9.3.1: Voice commands with synthetic speech (117 spoken commands, 90 English and 10 Bangla non-command sentences per noise level)

Noise added	Command carried out	Wrong command	Not heard	False commands (English / Bangla)
None	116 (99.1%)	0	1	0 / 0
White noise, 20 dB SNR	116 (99.1%)	0	1	1 / 0
White noise, 10 dB SNR	113 (96.6%)	0	4	1 / 2

Not measured on the Pi. The emotion entry was timed before the emotion mode was added to the application, and the mode has not been timed again. The mode-switch improvement was measured on the laptop only. Battery life was not measured.

6.9.4 Raspberry Pi 5, Second Run (13 October 2026)

The glass was benchmarked again on its own board after the changes of 11 to 13 October, with 30 calls per row after 3 warm-up calls, and the text reader was timed through the call the application makes when ACTION is pressed (Table 6.9.4.1). The full report, its raw files and its scripts are in `benchmark_report/`.

The board was at its temperature limit. The Pi was at 73 °C before the first timed call, after two sessions with the application. It reached 88.7 °C, and the firmware reported throttling or clock capping in 76% of the samples during the text-reading test, with the CPU clock down to 1.5 GHz instead of 2.4 GHz. The power supply was not the cause (no under-voltage) and the

fan was running. Every figure in the table is therefore the speed of a hot, throttled board; the first run ended at 77.1 °C without throttling. How much of the slowdown is heat cannot be said from this run.

Table 6.9.4.1: Second run on the Raspberry Pi 5 (13 October 2026), on a hot and throttled board

Part of the glass	**Typical time (median)**	**Slow case (95th percentile)**	**First run, 1 October**
Currency mode, whole job	399 ms	524 ms	327 ms
Taka detector, compressed INT8 ONNX (used)	306 ms	446 ms	254 ms
Watermark check, learned watermark finder (used)	64 ms	109 ms	50 ms
Counterfeit checker, 1 view / 6 views	43 / 98 ms	146 / 121 ms	37 / 90 ms
Object scan as the application runs it (YOLOv8s)	562 ms	610 ms	–
Object detector YOLOv8n alone	227 ms	342 ms	207 ms
Emotion, with a face in view	680 ms	926 ms	–
Emotion, no face in view (face search only)	48 ms	50 ms	48 ms
Text reading, 3-line English sign, as the application ran it	34.0 s	34.7 s (slowest of 3)	–
The same sign with the text found once	18.3 s	18.5 s (slowest of 4)	18.7 s
Text reading, 12-line page: first line spoken / whole page	16.4 s / 36.5 s	– / 39.9 s	–
Text reading, no text in view	27.5 s	28.6 s (slowest of 3)	–

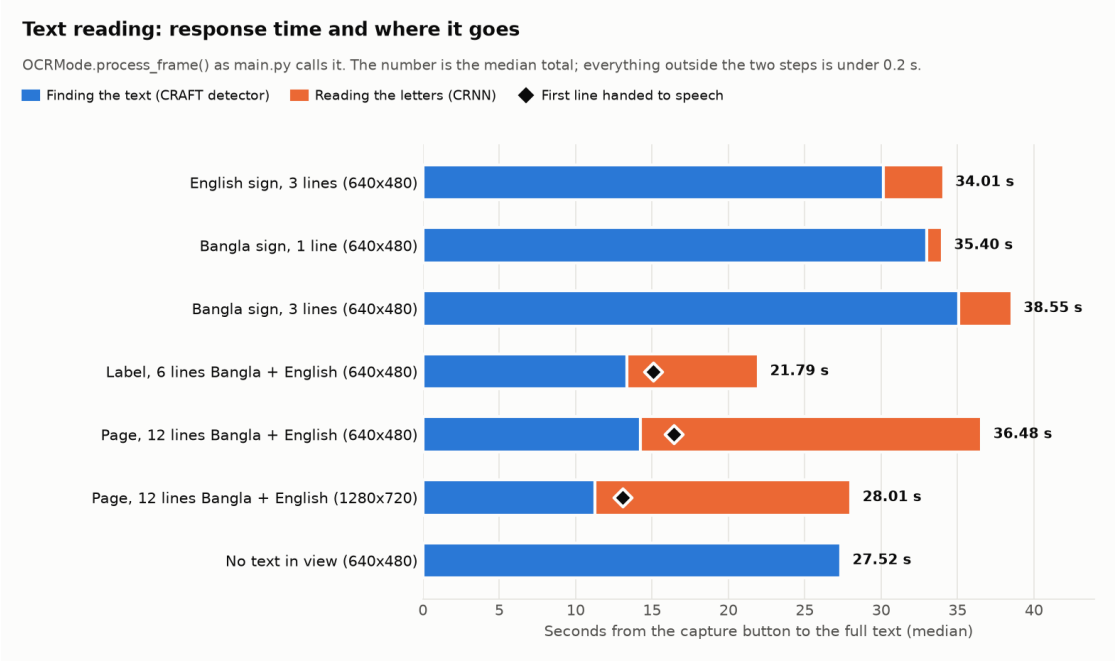


Figure 23: Text reading on the Raspberry Pi 5 on 13 October 2026: response time of each capture and where it goes (from the benchmark report)

Four things in this run are findings about the software, not about the heat.

1. **A short sign paid for the text detector twice.** The line-by-line path ran the detector to count the lines, and for fewer than four lines the normal path ran it again: 34.0 s instead

of 18.3 s. The detector now runs once per capture and both paths use its boxes; the text read is identical to before on the benchmark images.

2. **A picture with no text was not answered with "no text found".** After 27.5 s the fall-back reader for handwritten letters returned 48 letters that were not there. A picture in which the detector finds nothing is now answered at once, and the handwritten-letter reader is not tried on it.
3. **The emotion figure of the first run was a frame without a face.** With a face to classify, one answer takes 680 ms, not 48 ms.
4. **One pass of the text detector takes 11 to 18 s on this board**, with the full-precision ONNX detector and whatever is in the picture. The 8-bit detector, which costs 0.57 points of error on the rendered validation set and so does not meet the 0.5-point rule, was made the default on the Pi as a stated exception, for speed. Its speed on the Pi has not been measured yet. With the recogniser trained on real words (Table 6.7.3.1) the Pi's configuration scores 2.86% on the rendered validation set, 0.28 points above the reference, so it is now inside the rule.

The application's own log of the two sessions before the benchmark agrees with the table: currency check 406 ms (8 checks), object scan and announcement 777 ms (10), one text capture 39.9 s, three UV fibre checks at 0.8 to 1.4 s. Speech adds almost nothing to the response time (under 30 ms to synthesise a sentence). Not repeated in this run: the 30-minute sustained test, and a run on a cool board.

6.10 First Field Test with the Glass

On 3 October 2026 the glass was tried by hand for the first time with its own camera, after the deployment problems of Section 4.5 had been solved and the application started correctly. This was an informal test by the developers, not a measurement, and no error rates were recorded. Three problems were observed:

1. **Emotion detection did not work.** It was not in the glass application at all; it had existed only in the desktop harness. The mode was added the same day.
2. **Currency recognition was unreliable.**
3. **Bangla text was not read.**

The cause of the second and third problem has not been established. The suspected causes are the 640×480 frame and the fixed-focus lens of the camera, with which a note or a printed line held at reading distance may be neither large nor sharp enough; the training photographs and the benchmark images are sharp. A diagnosis script that compares the live 640×480 frame with a high-resolution capture of the same scene is ready for the device; tuning the camera settings is the next step of the project.

On the laptop with a webcam at 1,280×720, the same application was then run through all modes, with the live algorithm panel showing every model's output. Sound was at first missing on Windows and was fixed by a separate speech back-end. With the laptop webcam, notes held in the hand were recognised (Figure 18), which supports the suspicion that the problem on the glass lies in the picture from its camera and not in the detector.

Trial sessions with a visually impaired participant

After the first field test, trial sessions with the worn prototype were started with a visually impaired participant (Figure 10), and a survey is being collected. The data from these sessions were not available when this chapter was written. No result from them is reported here, and none of the conclusions of this thesis rests on them; they will be added once the survey is complete.

6.11 Corrections to Earlier Figures

Checking every claim against its result file changed several figures that appeared in an earlier draft of this thesis:

- **Detector test set.** "2,238 images, precision 0.998, recall 0.997, 7.3 ms on an RTX 4060" had no result file. The re-run on the 2,282 test images on disk gives the figures of Table 6.2.2.1.
- **Raw photographs.** The 96.9% on 450 raw photographs is a source-domain check, not a test; the independent result is 91.5%.
- **Authenticator.** A numerical fault ($0 \cdot \log 0$) changed 23 of 208 six-view predictions; all figures are after the fix.
- **Spoken verdict.** The glass first spoke "genuine" or "counterfeit", then nothing, and now follows the safe policy.
- **Bangla OCR.** The 27.8% Bangla CER came from wrongly drawn test images and a lexicon that contained the test phrases; it is replaced by the benchmark results.
- **ResNet-50 baseline.** The first fine-tuned baseline was trained on quarter-resolution views; the fair baseline removes the hybrid's apparent advantage.
- **Emotion on the glass.** A project report called the emotion feature working on the glass before it had been added to the glass application.

Calculation Methods and Full Working

This section presents the formulas used in this chapter and a worked example for each main table.

1) Detection Precision, Recall and mAP (Table 6.2.2.1)

With TP, FP and FN the numbers of true positives, false positives and false negatives at a given IoU threshold,

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad \text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

The average precision (AP) of one class is the area under its precision–recall curve; mAP@0.5 is the mean AP over the 9 classes at IoU ≥ 0.5, and mAP@0.5:0.95 is the mean over the ten IoU thresholds $0.50, 0.55, \ldots, 0.95$.

2) Raw-Photograph Detection Rate and Top-1 Accuracy (Table 6.2.3.1)

$$\text{Detection rate} = \frac{\text{notes with at least one box}}{\text{all notes}} = \frac{450 - 7}{450} = \frac{443}{450} = 98.4\%,$$

$$\text{Top-1 accuracy} = \frac{\text{notes with correct top box}}{\text{all notes}} = \frac{436}{450} = 96.9\%,$$

where $436 = 48 + 48 + 47 + 47 + 50 + 50 + 50 + 46 + 50$ is the sum of the correct notes over the nine denominations.

3) Authentication Accuracy and McNemar's Test (Table 6.3.1.1)

With 208 test notes, one-view PRMVT accuracy = 202/208 = 97.12% and baseline accuracy = 153/208 = 73.56%. McNemar's test uses only the discordant notes: n_{01} (baseline correct, PRMVT wrong) and n_{10} (baseline wrong, PRMVT correct):

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}},$$

with one degree of freedom. For example, for the fixed-count fusion model at six views $n_{01} = 3$ and $n_{10} = 13$, giving $\chi^2 = 81/16 = 5.06$ and $p = 0.024$.

4) Expected Calibration Error

Predictions are grouped into $M = 10$ confidence bins B_m:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|.$$

5) Emotion Macro-F1 (Table 6.6.0.2)

For each class c, $F1_c = 2P_cR_c/(P_c+R_c)$. For fear, $F1 = 2\times0.774\times0.554/(0.774+0.554) = 0.646$. Macro-F1 is the unweighted mean of the seven class F1 scores: $(0.936+0.839+0.799+0.870+0.646+0.627+0.851)/7 = 0.795$.

6) OCR Error Rates (Table 6.7.2.1)

$$\mathrm{CER} = \frac{S + D + I}{N},$$

where S, D and I are the character substitutions, deletions and insertions needed to turn the recognised text into the ground truth, and N is the number of ground-truth characters. WER is computed in the same way on words. Because insertions are counted, the rate can exceed 100% when an engine returns much more text than the image contains, as for the ByT5 corrector in Table 6.7.1.1.

7) Wilson Interval and the Safe Threshold (Table 6.5.2.1)

For x events in n trials with $\widehat{p} = x/n$ and $z = 1.96$, the Wilson interval is

$$\frac{\widehat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\widehat{p}(1-\widehat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}.$$

For 0 counterfeits passed out of 88, the upper limit is $\frac{z^2/n}{1+z^2/n} = \frac{3.84/88}{1+3.84/88} = 4.2\%$. The threshold τ is the highest of the $n = 88$ validation counterfeit scores, so a new exchangeable counterfeit exceeds it with probability $1/(n+1) = 1/89 = 1.1\%$.

6.12 Discussion

The results answer the research questions of Section 1.5.

System (RQ1). A five-mode assistive glass runs on a Raspberry Pi 5 within 1.5 GB of memory; in a second run after two sessions of use the board reached its temperature limit and was throttled (Section 6.9.4). Currency, objects, the watermark check and emotion respond in 50 to 330 ms; with speech, a currency answer takes under three seconds. Text reading is the slowest mode (about 19 seconds per capture, mostly in the text detector); for pages, a faster decoder and line-by-line reading are now in place.

Synthetic data for currency (RQ2). Training only on automatically labelled composites gave near-perfect scores on held-out composites and 91.5% correct denominations on 1,536 independently collected photographs, though only 18.5% on hand-held close-ups. Realistic backgrounds and physically motivated augmentation mattered more than the network architecture.

Number of views (RQ3). A model trained with a fixed number of views fails when fewer are available, and unpredictably so across seeds; training on random view prefixes fixes this and keeps accuracy at about 98% from one to six views. The controlled comparison shows that prefix sampling alone is responsible. The benefit depends on the fusion head and did not appear on every other dataset.

Unseen prints (RQ4). The usual JaalTaka split is a test of known reprints: a serial lookup scores 90.2% on it, and frozen general-purpose backbones reach 98–99%. On unseen prints accuracy falls by about 8 points. The watermark recovers about 5 of them and makes the result stable across seeds, but a properly fine-tuned ResNet-50 does as well. With about 24 test prints, the dataset cannot separate the two.

Safety (RQ5). A policy that never says "counterfeit" and confirms a note only above the highest validation counterfeit score passed none of 88 test counterfeits. It confirms 57.5% of genuine notes from close-up views; on whole notes it mostly asks for a check by hand, so raising that confirmation rate is the next goal.

Bangla OCR (RQ6). Region selection matters most: choosing the main text block removed three quarters of the error. Parameter tuning and clean-up rules added small gains; preprocessing beyond contrast enhancement, other engines, other detectors, dictionaries, language models and multi-scale reading did not help. Fine-tuning the recogniser is a promising next step.

Interaction (RQ7). Separating capture from reading, cooldowns, queue clearing on mode change, mixed-script speech, immediate mode names and spoken capture guidance were designed for users without sight. Trial sessions with a visually impaired participant have started to measure how much they help.

A general lesson. The largest corrections in this project came from looking at the data and the test protocol and not from new models: the serial numbers on the counterfeit notes, the shape of the rendered Bangla text, the resolution of a cached image, the source of a "wild" test set. Each of these had produced a good-looking number that was wrong or meant something else.

6.13 Scope of the Results

The results above should be read with the following scope in mind; each point is also a direction for the next version (Chapter 7):

1. **Glass camera.** The accuracy figures come from datasets and a laptop webcam; the camera settings of the worn glass are being tuned, and a test set taken through it is planned.
2. **User study.** Trial sessions with a visually impaired participant have started; their results will be added when the survey is complete.

3. **Counterfeit data.** The unseen-print results rest on about 24 distinct counterfeit prints of one currency; a larger print-disjoint collection would sharpen the comparison between methods.
4. **Whole-note verdict.** On whole notes the safe policy mostly asks for a check by hand; the guided watermark check and a larger set of whole-note photographs are the way to confirm more genuine notes.
5. **Text reading.** On real street photographs the letter error is about 39% for Bangla and 62% for Latin script, against 2.5% on the rendered benchmark (Section 6.7.3), and reading is the slowest mode on the Pi.
6. **UV check.** It accepts 85.6% of the photographs of genuine notes and rejects about nine in ten simulated fibre-free sheets, but has not been tested on a real counterfeit note, on real printed paper under the lamp, or through the glass camera.
7. **Objects.** With the model timed on the Pi (YOLOv8n) and small pictures, mAP@0.5 is 0.34 to 0.45; the classes are the 80 of COCO.
8. **Other items.** Close-ups covered by fingers, objects outside the 80 COCO classes, a neural Bangla voice on the Pi, and battery life are left for future versions.

7 Conclusion and Future Work

7.1 Conclusion

This thesis presented SARA, an assistive smart glass for visually impaired people in Bangladesh. The glass reads Bangla and English text, announces everyday objects in Bangla, names all nine denominations of Bangladeshi Taka, gives a cautious check for counterfeit 500 and 1,000 Taka notes, and names the facial expression of the person in front of the user. It is controlled with five physical buttons, speaks through Piper or espeak-ng voices, starts automatically on a Raspberry Pi 5, and needs the internet only for an optional scene-description mode.

Six results stand out.

Synthetic data is enough to recognise notes in ordinary photographs. A copy-paste compositing pipeline produced 22,333 labelled images without manual annotation. The YOLOv8s detector trained on them reached 0.995 mAP@0.5 and 0.849 mAP@0.5:0.95 on held-out composites and named the correct denomination for 91.5% of 1,536 independently collected photographs.

Training on random prefixes of views removes the view-count collapse. A counterfeit authenticator trained with all six views fails, and unpredictably, when a user captures fewer. With prefix training, one-view accuracy is 98.2% against 71.2% over three seeds, with no loss at six views.

The usual counterfeit benchmark measures known reprints. Most JaalTaka counterfeits carry one of a few serial numbers, and the same prints appear in training and test. On a split with unseen prints, accuracy falls from about 98% to about 90%. A check of the watermark, with a learned localizer that works on every test photograph, brings it to 95.0% with no variation across seeds, level with a fully fine-tuned ResNet-50.

A watermark can be checked with a tiny, explainable model. The denomination-matched watermark residual removes the lamp from a back-lit photograph, compares the watermark with a genuine portrait of the note value, and classifies what is left over. With 47,818 parameters it reaches 91.5% on unseen prints, statistically level with a MobileNetV2 that is 46 times larger, and it is among the most stable watermark models across seeds.

A safe spoken verdict. The glass never says “counterfeit” and says “likely genuine” only above a threshold fixed on validation data. No test counterfeit was passed (0 of 88), and in 1,889 runs the word “counterfeit” was never spoken.

For Bangla reading, what is read matters more than how. On a benchmark with correctly shaped Bangla and separate validation and test phrases and fonts, selecting the main text block, tuned settings and clean-up rules lowered the character error rate from 9.3% to 2.4%, and for Bangla from 11.9% to 3.2%. Other engines, other detectors, dictionaries, language models and multi-scale reading did not help.

On the Raspberry Pi 5 the currency check takes a median of 327 ms, and the system stays within 1.5 GB of memory and ran for 30 minutes without throttling. The next steps are tuning the glass camera, faster text reading and completing the trials with visually impaired users.

Overall, SARA shows that the parts of a useful, private and affordable assistive device for

Bangla-speaking users can be built from open models and low-cost hardware. The progress in this project came less from new network architectures than from careful data engineering and from checking what each test really measures. SARA is ready to be refined together with its users.

7.2 Future Work

7.2.1 Tuning the Glass Camera

The first task is to tune currency recognition and Bangla reading for the glass camera. The diagnosis script compares the live 640×480 frame with a high-resolution capture; depending on its result, the fix is a higher capture resolution, a camera with autofocus, or training data photographed through the glass. A test set of notes and signs photographed through the glass camera is needed in any case, and the capture-guide limits must be set again on it.

7.2.2 User Study with Visually Impaired Participants

Trial sessions have started; the next step is the full user study with 8–12 visually impaired participants, with ethical approval. Participants will identify denominations, use the watermark check with and without spoken guidance, and read signs. Task time, accuracy, requests for help, NASA-TLX workload and System Usability Scale scores will be recorded, with the participant as the unit of analysis. The study log, the analysis script and the voice questionnaire are ready.

7.2.3 A Print-Disjoint Counterfeit Dataset

A dataset with about 100 distinct counterfeit prints, collected with a bank or the police and split by print, would allow detectors to be compared with confidence, and whole-note photographs of counterfeit notes would allow the whole-note verdict to be trusted or rejected. The collection tool for it exists. A second currency would show whether the watermark approach transfers.

7.2.4 Faster and Better Text Reading

Text reading on the Pi can be made several times faster, with a lighter text detector, a smaller detector canvas or a quantised model. The recogniser fine-tuned on real words and blended with the released weights is a first step; photographs of signs in Bangladesh, with their text typed out, are the next training and test data.

7.2.5 Power, Speech and Remaining Measurements

Battery life must be measured with a power meter. A neural Bangla voice that runs on the Pi would make the speech more natural. The online mode, the emotion mode and the new OCR pipeline have to be timed on the Pi.

7.2.6 Navigation and Voice Control

Future versions can add obstacle distance (ultrasonic or depth sensors), navigation-relevant object classes such as stairs, rickshaws and open drains, and Bangla command words, once an offline Bangla speech recogniser is available, next to the English voice commands the glass already takes.

7.2.7 Mechanical Design

A lighter, integrated spectacle frame with the camera, an earpiece or bone-conduction speaker and a compact battery would make the glass more comfortable and less noticeable.

7.3 Impact and Sustainability

SARA is built from widely available hardware and open-source software, which keeps the cost low and allows local repair and extension. Because all core functions run on the device, it does not require a data plan and continues to work in rural areas with poor connectivity. The modular mode design makes it possible to add new languages, currencies or functions without redesigning the system, and the desktop harness lets students and developers improve the software without the hardware. The experiment database and the claim registry let a later team check every number of this thesis against its result file.

7.4 Social and Ethical Considerations

An assistive device that uses a camera in public raises privacy concerns. SARA processes images on the device and does not store or transmit them in the offline modes; the only online mode is optional and must be chosen by the user. The currency check can make mistakes. A wrong "genuine" could cause financial loss, and a wrong "counterfeit" could lead a blind person to accuse someone in public without being able to verify the claim. The device therefore never says "counterfeit", presents "likely genuine" as advice, and otherwise asks for a check by hand. The voice questionnaire records participants' voices and sends them to an online speech recogniser; in a real study this needs informed consent, and the recordings must be stored under a participant number, not a name. Emotion recognition of other people must be used with care and only to adapt the device's own speech, never to label or record people.

7.5 Contributions and Final Thoughts

This thesis contributes a complete assistive glass that speaks Bangla, measured on a Raspberry Pi 5, a zero-annotation data pipeline and detector for Bangladeshi Taka, a multi-view training protocol that remains accurate with one view, an audit and a print-disjoint test for counterfeit detection, a watermark check with a safe spoken verdict, a tiny physics-based watermark classifier (DMWR), a Bangla OCR benchmark and pipeline, and an evaluation in which every number can be traced to a saved result. We hope that this work will make it easier for others to continue, and that future versions of SARA, tested with and improved by visually impaired users, will help them read, recognise and pay with more independence.

7.6 Analysis of Social Effects

Improving Independence

Reading a label, knowing which note is in the hand and knowing what is in front of oneself without asking another person are small tasks that together decide how independent a person can be. SARA targets exactly these daily tasks.

7.6.1 Reducing Fraud and Financial Risk

Visually impaired people are vulnerable to receiving wrong change or counterfeit notes. Denomination detection can reduce this risk and increase confidence in cash transactions. The counterfeit check, in its present cautious form, tells the user when a note deserves a second look by a trusted person; it does not replace that look.

7.6.2 Supporting Education and Employment

Access to printed text in Bangla and English supports study and work, especially where accessible digital documents are not available.

7.6.3 Changing Perceptions of Assistive Technology

A low-cost device built locally, speaking the user's language, can change the perception that assistive technology is only available as an expensive import.

7.6.4 Possible Social Risks

Over-reliance on an imperfect device, privacy concerns of people around the user, and the stigma of wearing a visible device are possible risks, which must be studied together with users.

7.6.5 Summary of Social Impact

If validated with users, SARA can increase independence, reduce financial risk and improve access to information for visually impaired people in Bangladesh, while keeping their data private.

A Appendix

A.1 Additional Plots

A.1.1 Taka Detector Precision–Recall Curve

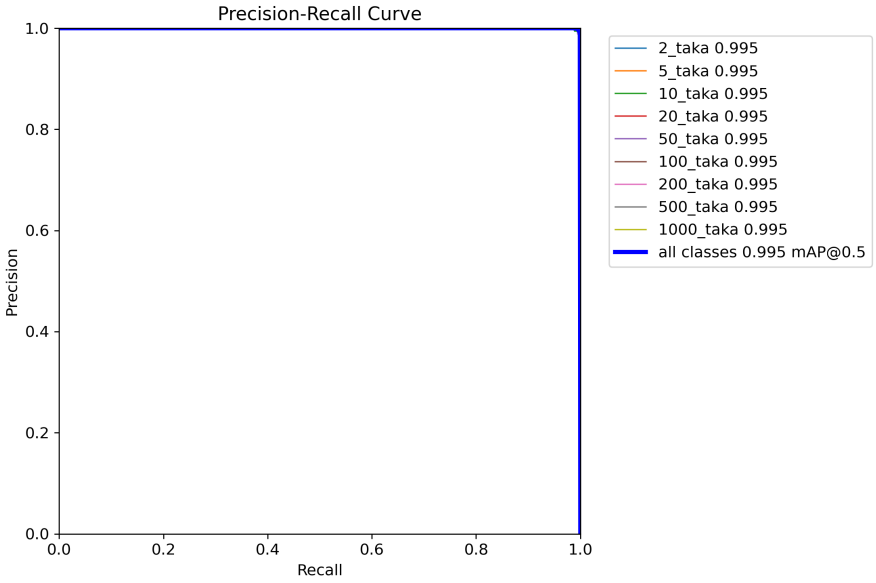


Figure 24: Precision–recall curves of the YOLOv8s Taka detector for all nine classes on the held-out synthetic test set

A.1.2 Sample Detections

Figure 7 — Sample Detections with Bounding Boxes and Confidence Scores

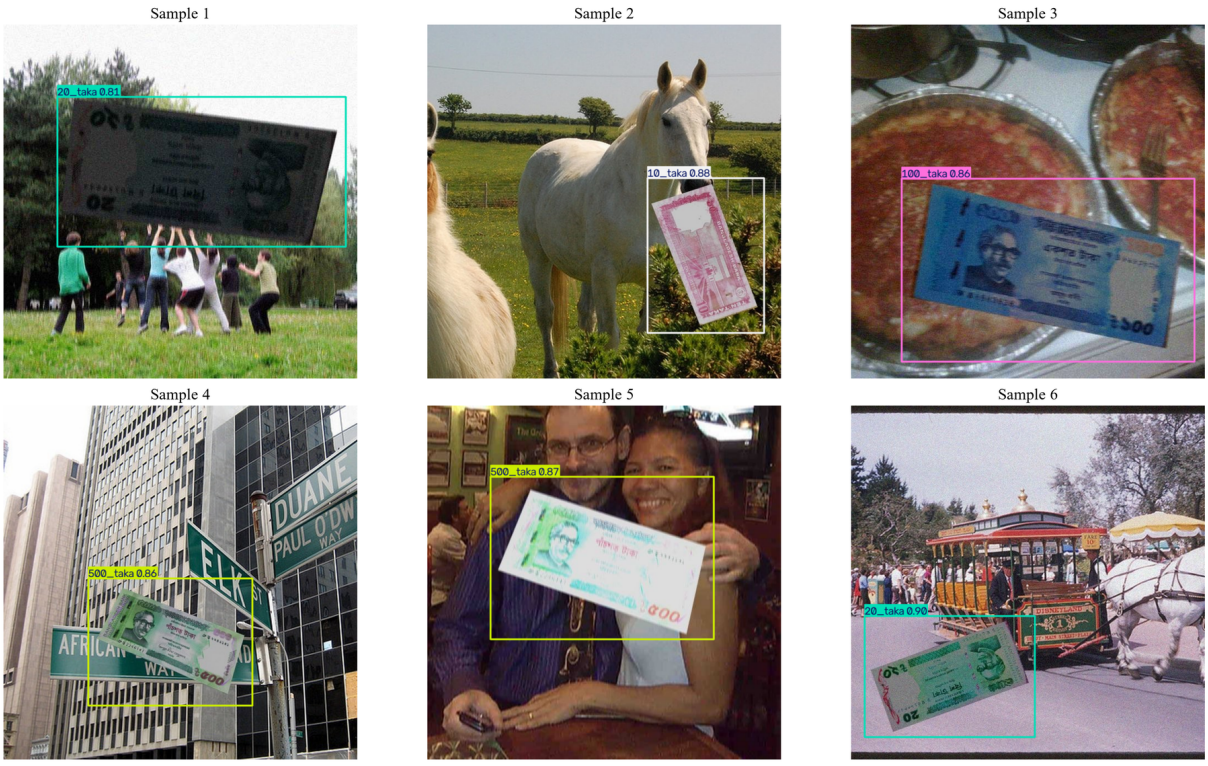


Figure 25: Sample detections of the Taka detector on composited scenes

A.1.3 Authentication Calibration

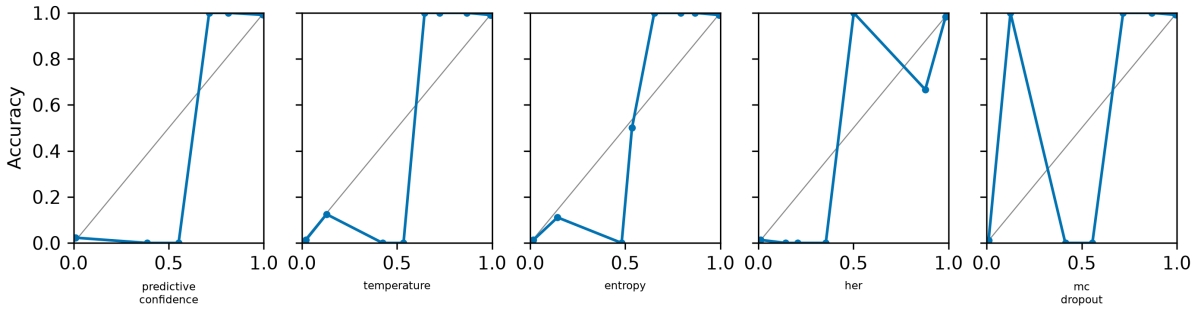


Figure 26: Reliability diagrams of the authenticator at six views for five confidence estimates (raw predictive confidence, temperature scaling, entropy mapping, HER and MC dropout); calibration maps fitted on the validation split and evaluated on the test split

A.1.4 Authentication over Seeds

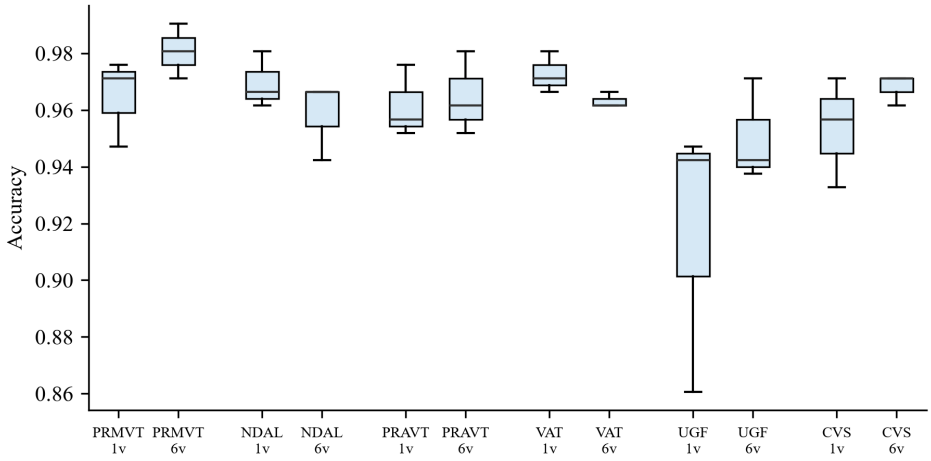


Figure 27: Spread of test accuracy over seeds 42, 43 and 44 at one view (1v) and six views (6v) for the main training variants

A.2 Configuration Parameters

Table A.2.0.1: Main configuration parameters of SARA (`config.py`)

Setting (name in the code)	Value	What it controls
OCR_PIPELINE	v2	Which text-reading pipeline is used (Section 3.4.1)
OCR_V2_MIN_CONF	0.1	Words the reader is less sure about than this are dropped
OCR_CANVAS_SIZE	2560	Picture size used to find text; 640 is faster on the Pi but less accurate
OCR_V2_DECODER	greedy	How the reader turns its output into letters (greedy: the best letter at each step)
OCR_STREAM_PAGES	on, 4 lines	Pages with at least 4 lines are spoken line by line
CURRENCY_ANNOUNCE_CONF	0.60	A note is named only when the detector is at least this sure
JAAL_SAFE_TAU	0.99959	The glass says “likely genuine” only above this score; otherwise “check by hand”
WATERMARK_CLEAR_THRESHOLD	0.5	Score above which the watermark counts as present
CAPTURE_GUIDE_MIN_MEAN	73.87	A note photo held against light must be at least this bright (0–255)
CAPTURE_GUIDE_MIN_LAPVAR	106.75	The photo must be at least this sharp (Laplacian variance)
CAPTURE_GUIDE_TIMEOUT_S	10 s	If no good photo arrives in this time, the glass asks the user to check by hand
FIELD_LOG_SYS_INTERVAL_S	10 s	How often temperature, memory and load are written to the field log
OBJECT_CONFIDENCE	0.50	An object is spoken only when the detector is at least this sure
CURRENCY_CONFIDENCE	0.65	Minimum score of the back-up Taka classifier (MobileNetV3)
OBJECT_SCAN_INTERVAL	1.5 s	How often object mode looks for objects by itself
OBJECT_ANNOUNCE_COOLDOWN	4.0 s	The same object is not repeated within this time
DEFAULT_VOLUME	80%	Loudness when the glass starts
VOLUME_STEP	10%	Loudness change per VOLUME button press
TTS_QUEUE_MAXSIZE	5	At most this many sentences wait to be spoken
TTS_INTER_UTTERANCE_PAUSE	0.15 s	Short pause between spoken sentences
ESPEAK_SPEED / PITCH	135 wpm / 45	Speed (words per minute) and pitch of the espeak-ng voice
Button debounce	250 ms	A second press within this time is ignored (stops accidental double presses)
CAMERA_WIDTH, CAMERA_HEIGHT	640×480	Picture size taken from the camera

Table A.2.0.2: Training hyper-parameters of the YOLOv8s Taka detector

Setting	Value
Model	YOLOv8s (11.1 million parameters, 28.5 GFLOPs), starting from weights already trained on COCO
Training rounds (epochs) / stop early if no progress for	80 / 15 epochs
Picture size / pictures per step (batch)	640 pixels / 16
Learning method (optimiser)	SGD; starting learning rate 0.01, final factor 0.01; momentum 0.937; weight decay 0.0005
Gentle start (warm-up)	3 epochs
Weights of the three training losses	Box position (CIoU) 7.5; note type (BCE) 0.5; box edges (DFL) 1.5
Extra random changes during training	Mosaic of four pictures 1.0; mixing two pictures 0.15; colour shift (hue 0.02, saturation 0.7, brightness 0.4); rotation up to 20°; shear 5°; resize ±50%; shift 10%
Mirror flips	Off (left–right and up–down), because a mirrored note is not a real note
Software	Ultralytics 8.4.64, PyTorch 2.6.0 with CUDA 12.4

A.3 Reproducing the Results

```
# Taka detector (Windows, from realtime_bangla_taka_detection/)
python -m venv venv
.\venv\Scripts\pip.exe install -r requirements.txt
.\venv\Scripts\python.exe scripts\download_backgrounds.py
.\venv\Scripts\python.exe scripts\generate_synthetic_dataset.py
.\venv\Scripts\python.exe scripts\train.py
.\venv\Scripts\python.exe scripts\evaluate.py
.\venv\Scripts\python.exe scripts\realtime_detect.py # live demo

# Smart glass on Raspberry Pi 5 (from savior_glass/)
bash scripts/deploy_pi5.sh # packages, environment, models
python scripts/pi5_preflight.py # check libraries and models
python main.py # start the glass
python scripts/benchmark_pi5.py --iters 100 --sustained 30
python scripts/diagnose_glass.py currency # live frame vs high resolution
python scripts/field_report.py # summary of the field log

# Smart glass on a laptop with a webcam (preview window, keyboard keys)
SHOW_PREVIEW=1 python main.py
python scripts/run_voice_study.py # voice questionnaire session
python scripts/run_voice_study.py --results

# OCR benchmark (from savior_glass/ocr_bench/)
python run_task1.py ... run_task6.py # validation experiments
python run_test.py # test set, once per method

# Counterfeit experiments (from realtime_bangla_taka_detection/)
python scripts/eval/jaaltaka_serial_audit.py
python scripts/train/train_watermark_localizer.py 42
```

```
# Experiment database
python research_results/results_db.py review
```

A.4 Spoken Messages

The glass speaks Bangla. Table A.4.0.1 gives the main messages in English translation.

Table A.4.0.1: Main spoken messages of SARA (translated from Bangla)

When	What the glass says
A note is recognised	“Five hundred Taka note”
The counterfeit check is passed	“Likely genuine”
The counterfeit check is not passed	“Check by hand whether it is genuine”
A note other than 500 or 1,000 Taka (no check available)	“The counterfeit check was not done”
The detector is not sure enough	“Could not confirm the note. Hold it closer.”
No note is found	“No note detected. Hold it in front of the camera.”
The watermark check starts	“Hold the note up to the light”
The picture is too dark	“Hold the note closer to the light”
The picture is blurred	“Keep the note still”
The watermark area cannot be found	“Hold the note straight in front of the light”
No good picture within 10 seconds	“The picture was not clear. Check by hand whether it is genuine.”
Emotion mode finds a face	“The person in front looks happy”
Emotion mode finds no face	“No face found. Point the camera at a person’s face.”

A.5 User Study Questionnaire

The six yes/no questions asked by the voice study tool, translated from Bangla:

1. Was the glass easy to use?
2. Was the speech of the glass clear?
3. Did the glass help you recognise banknotes?
4. Did the glass help you read text?
5. Can you trust the answers of the glass?
6. Would you want to use this glass every day?

Bibliography

[1] World Health Organization. *Blindness and Vision Impairment: Fact Sheet*. https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment. 2023.

[2] GBD 2019 Blindness and Vision Impairment Collaborators. "Trends in Prevalence of Blindness and Distance and Near Vision Impairment over 30 Years: An Analysis for the Global Burden of Disease Study". In: *The Lancet Global Health* 9.2 (2021), e130–e143.

[3] OrCam Technologies. *OrCam MyEye*. https://www.orcam.com.

[4] Envision. *Envision Glasses*. https://www.letsenvision.com.

[5] Microsoft. *Seeing AI*. https://www.seeingai.com.

[6] "Reading Aid and Translator with Raspberry Pi for Blind people". In: *IEEE Conference Proceedings*. 2023. URL: https://ieeexplore.ieee.org/abstract/document/10113042.

[7] "Vision Voice: A Raspberry Pi-Based Text-to-Audio Converter for the Visually Impaired". In: *IEEE Conference Proceedings*. 2024. URL: https://ieeexplore.ieee.org/abstract/document/10664121.

[8] "Camera Detection for Blind People Using OCR". In: *IEEE Conference Proceedings*. 2023. URL: https://ieeexplore.ieee.org/abstract/document/10146654.

[9] Raspberry Pi Ltd. *Raspberry Pi 5 Product Brief*. https://www.raspberrypi.com/products/raspberry-pi-5/. 2023.

[10] "Assistive Technologies for Blind and Visually Impaired Individuals – A Short Review". In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/11012288.

[11] Xiaochen Zhang et al. "Advancements in Smart Wearable Mobility Aids for Visual Impairments: A Bibliometric Narrative Review". In: *Sensors* 24.24 (2024), p. 7986. URL: https://www.mdpi.com/1424-8220/24/24/7986.

[12] "Deep learning techniques for Real time image to voice for the Visually impaired: A review". In: (2024). URL: https://degres.eu/wp-content/uploads/5-DEJ1219.pdf.

[13] John G. Linvill and James C. Bliss. "A Direct Translation Reading Aid for the Blind". In: *Proceedings of the IEEE* 54.1 (1966), pp. 40–51.

[14] "An Assistive Reading System for Visually Impaired using OCR and TTS". In: (2014). URL: https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=b80413fc82693381488d3c8bfc6de368668a7142.

[15] "Implementation of a Speech Based Interface System for Visually Impaired Persons". In: (2013). URL: https://www.researchgate.net/publication/273630605.

[16] Roy Shilkrot et al. “FingerReader: A Wearable Device to Explore Printed Text on the Go”. In: *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI)*. 2014. URL: https://www.researchgate.net/publication/266655994.

[17] “Alternative Product Label Reading and Speech Conversion: An Aid for Blind Person”. In: *IEEE Conference Proceedings*. 2017. URL: https://ieeexplore.ieee.org/abstract/document/8463923.

[18] “Improved Camera-Based Text Reading Assistant System Using Digital Image Processing Techniques”. In: *IEEE Conference Proceedings*. 2022. URL: https://ieeexplore.ieee.org/document/9872447.

[19] “Image processing technology for text recognition”. In: *IEEE Conference Proceedings*. 2019. URL: https://ieeexplore.ieee.org/document/8959877.

[20] “Design and Implementation of Smart Glass with Voice Detection Capability to Help Visually Impaired People”. In: (2017). URL: https://www.researchgate.net/publication/320755584.

[21] Hafeez Ali A. et al. “A Google Glass Based Real-Time Scene Analysis for the Visually Impaired”. In: *IEEE Access* 9 (2021), pp. 166351–166369. DOI: 10.1109/ACCESS.2021.3135024. URL: https://ieeexplore.ieee.org/abstract/document/9648186.

[22] Mukhriddin Mukhiddinov and Jinsoo Cho. “Smart Glass System Using Deep Learning for the Blind and Visually Impaired”. In: *Electronics* 10.22 (2021), p. 2756. URL: https://www.mdpi.com/2079-9292/10/22/2756.

[23] Dionysia Danai Brilli et al. *AIris: An AI-powered Wearable Assistive Device for the Visually Impaired*. arXiv:2405.07606. 2024. URL: https://arxiv.org/abs/2405.07606.

[24] *AI Based Real-Time Blind Assistance System with Voice Feedback*. SSRN. 2025. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5216709.

[25] “Voice Enabled Reading Assistance for Visually Impaired”. In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/11032424.

[26] “Smart Glasses for Blind Assistance: A Lightweight and Ergonomic Approach”. In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/10915049.

[27] Ray Smith. “An Overview of the Tesseract OCR Engine”. In: *International Conference on Document Analysis and Recognition (ICDAR)*. 2007, pp. 629–633.

[28] “Text Extraction and Detection from Images using Machine Learning Techniques: A Research Review”. In: *IEEE Conference Proceedings*. 2022. URL: https://ieeexplore.ieee.org/document/9752274.

[29] Youngmin Baek et al. “Character Region Awareness for Text Detection”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 9365–9374.

[30] Xinyu Zhou et al. “EAST: An Efficient and Accurate Scene Text Detector”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 5551–5560.

[31] Minghui Liao et al. “Real-Time Scene Text Detection with Differentiable Binarization”. In: *AAAI Conference on Artificial Intelligence*. Vol. 34. 2020, pp. 11474–11481.

[32] Baoguang Shi, Xiang Bai, and Cong Yao. “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39.11 (2017), pp. 2298–2304.

[33] Alex Graves et al. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks”. In: *International Conference on Machine Learning (ICML)*. 2006, pp. 369–376.

[34] Minghao Li et al. “TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models”. In: *AAAI Conference on Artificial Intelligence*. Vol. 37. 2023, pp. 13094–13102.

[35] JaidedAI. *EasyOCR: Ready-to-use OCR with 80+ Supported Languages*. https://github.com/JaidedAI/EasyOCR.

[36] “Intelligent Image Text Reader using Easy OCR, NRCLex & NLTK”. In: *IEEE Conference Proceedings*. 2022. URL: https://ieeexplore.ieee.org/document/10047136.

[37] PaddlePaddle Authors. *PaddleOCR: Multilingual OCR Toolkits Based on PaddlePaddle*. https://github.com/PaddlePaddle/PaddleOCR.

[38] Wolf Garbe. *SymSpell: Symmetric Delete Spelling Correction Algorithm*. https://github.com/wolfgarbe/SymSpell.

[39] Kenneth Heafield. “KenLM: Faster and Smaller Language Model Queries”. In: *Proceedings of the Sixth Workshop on Statistical Machine Translation*. 2011, pp. 187–197.

[40] Linting Xue et al. “ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models”. In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 291–306.

[41] “Cloud based Text extraction using Google Cloud Vision for Visually Impaired applications”. In: *IEEE Conference Proceedings*. 2019. URL: https://ieeexplore.ieee.org/abstract/document/9087314.

[42] “Bangla Optical Character Recognition (OCR) Using Deep Learning Based Image Classification Algorithms”. In: *IEEE Conference Proceedings*. 2021. URL: https://ieeexplore.ieee.org/document/9689864.

[43] AKM Shahariar Azad Rabby et al. “Ekush: A Multipurpose and Multitype Comprehensive Database for Online Off-Line Bangla Handwritten Characters”. In: *Recent Trends in Image Processing and Pattern Recognition (RTIP2R)*. 2018.

[44] HarfBuzz Project. *HarfBuzz: A Text Shaping Engine*. https://harfbuzz.github.io.

[45] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.

[46] “Echolight: A Camera-Powered Voice-Assisted Aid for the Visually Impaired”. In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/11019645.

[47] “Assistance for Visually Impaired individuals using Artificial Intelligence and Natural Language Processing”. In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/10941456.

[48] “IV Smart: Empowering the Visually Impaired using AI-Driven Assistive Technology”. In: *IEEE Conference Proceedings*. 2024. URL: https://ieeexplore.ieee.org/abstract/document/10841486.

[49] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. https://github.com/ultralytics/ultralytics. 2023.

[50] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context”. In: *European Conference on Computer Vision (ECCV)*. 2014.

[51] ONNX Runtime Developers. *ONNX Runtime*. https://onnxruntime.ai.

[52] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2704–2713.

[53] “Speech-to-Text and Text-to-Speech Recognition Using Deep Learning”. In: *IEEE Conference Proceedings*. 2023. URL: https://ieeexplore.ieee.org/document/10212222.

[54] *eSpeak NG Text-to-Speech*. https://github.com/espeak-ng/espeak-ng.

[55] Rhasspy. *Piper: A Fast, Local Neural Text to Speech System*. https://github.com/rhasspy/piper.

[56] “Currency recognition system using image processing”. In: *IEEE Conference Proceedings*. 2017. URL: https://ieeexplore.ieee.org/document/8284300.

[57] “Currency Recognition System Using Image Processing”. In: (2023). URL: https://www.researchgate.net/publication/378264217.

[58] “Bangladeshi Banknote Recognition by Neural Network with Axis Symmetrical Masks”. In: *Proceedings of the International Conference on Computer and Information Technology (ICCIT)*. 2007.

[59] *Bangladeshi Banknote Recognition in Real-Time Using Convolutional Neural Network for Visually Impaired People*. Conference paper.

[60] *Robust and Real-Time Bangladeshi Currency Recognition: A Dual-Stream MobileNet and EfficientNet Approach*. arXiv:2602.07015. 2026.

[61] Md. Jubayar Alam Rafi, Mohammad Rony, and Nazia Majadi. “NSTU-BDTAKA: An open dataset for Bangladeshi paper currency detection and recognition”. In: *Data in Brief* 55 (2024), p. 110701. URL: https://www.sciencedirect.com/science/article/pii/S2352340924006681.

[62] *Real-Time Currency Detection and Voice Feedback for Visually Impaired Individuals*. arXiv:2510.20267. 2025.

[63] Debidatta Dwibedi, Ishan Misra, and Martial Hebert. “Cut, Paste and Learn: Surprisingly Easy Synthesis for Instance Detection”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 1301–1310.

[64] Golnaz Ghiasi et al. “Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 2918–2928.

[65] Josh Tobin et al. “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[66] Nino, Gardi, et al. *Synthetic-to-Real Object Detection Using YOLO and Domain Randomization Strategies*. arXiv:2509.15045. 2025.

[67] “BLIND ASSIST: A One Stop Mobile Application for the Visually Impaired”. In: *IEEE Conference Proceedings*. 2021. URL: https://ieeexplore.ieee.org/abstract/document/9686476.

[68] “Divya-Drishti: An Independent Aid for the Visually Impaired”. In: (2021). URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3867707.

[69] "VisioGuide: An AI for Visually Impaired". In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/10932984.

[70] Bibhas Roy Chowdhury Piyas et al. *JaalTaka: A Benchmark Dataset for Detecting Counterfeit Bangladeshi Banknotes*. Mendeley Data, V2, CC BY 4.0. Data article: Data in Brief, https://www.sciencedirect.com/science/article/pii/S235234092501090X. 2025. DOI: 10.17632/2m7wk5cy4c.2. URL: https://data.mendeley.com/datasets/2m7wk5cy4c/2.

[71] Hang Su et al. "Multi-View Convolutional Neural Networks for 3D Shape Recognition". In: *IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 945–953.

[72] Zhirong Wu et al. "3D ShapeNets: A Deep Representation for Volumetric Shapes". In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2015, pp. 1912–1920.

[73] Ren Wang et al. "MVP-N: A Dataset and Benchmark for Real-World Multi-View Object Classification". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. Vol. 35. 2022.

[74] Alexey Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *International Conference on Learning Representations (ICLR)*. 2021.

[75] Natalia Neverova et al. "ModDrop: Adaptive Multi-Modal Gesture Recognition". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38.8 (2016), pp. 1692–1706.

[76] Sayash Kapoor and Arvind Narayanan. "Leakage and the Reproducibility Crisis in Machine-Learning-Based Science". In: *Patterns* 4.9 (2023), p. 100804.

[77] Robert Geirhos et al. "Shortcut Learning in Deep Neural Networks". In: *Nature Machine Intelligence* 2.11 (2020), pp. 665–673.

[78] C. K. Chow. "On Optimum Recognition Error and Reject Tradeoff". In: *IEEE Transactions on Information Theory* 16.1 (1970), pp. 41–46.

[79] Yonatan Geifman and Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017.

[80] Chuan Guo et al. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017.

[81] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

[82] John Brooke. "SUS: A Quick and Dirty Usability Scale". In: *Usability Evaluation in Industry*. Taylor & Francis, 1996, pp. 189–194.

[83] Sandra G. Hart and Lowell E. Staveland. "Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research". In: *Human Mental Workload*. Vol. 52. Advances in Psychology. North-Holland, 1988, pp. 139–183.

[84] Jothi Ganesan et al. "Deep Learning Reader for Visually Impaired". In: *Electronics* 11.20 (2022), p. 3335. URL: https://www.mdpi.com/2079-9292/11/20/3335.

[85] "The Third Eye – An Assistive Technology for the Blind". In: *International Journal of Computer Science Trends and Technology* 7.3 (2019). URL: https://www.ijcstjournal.org/volume-7/issue-3/IJCST-V7I3P13.pdf.

[86] “AI-Based Smart Visual Assistance System for Navigation, Guidance, and Monitoring of Visually Impaired People”. In: *IEEE Conference Proceedings*. 2024. URL: https://ieeexplore.ieee.org/abstract/document/10568710.

[87] “Smart Glass for Visually Impaired Person”. In: *IEEE Conference Proceedings*. 2025. URL: https://ieeexplore.ieee.org/abstract/document/11019089.

[88] “Obstacle Detection and Text Recognition for Visually Impaired Person Based on Raspberry Pi”. In: (2020). URL: https://www.academia.edu/download/66137202/83_Article_Text_147_1_10_20201003.pdf.

[89] eSight Corporation. *eSight Electronic Eyewear*. https://www.esighteyewear.com.

[90] Raspberry Pi Ltd. *The Picamera2 Library*. https://github.com/raspberrypi/picamera2.

[91] Karel Zuiderveld. “Contrast Limited Adaptive Histogram Equalization”. In: *Graphics Gems IV*. Academic Press, 1994, pp. 474–485.

[92] Andrew Howard et al. “Searching for MobileNetV3”. In: *IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019.

[93] Kan Wu et al. “TinyViT: Fast Pretraining Distillation for Small Vision Transformers”. In: *European Conference on Computer Vision (ECCV)*. 2022.

[94] David G. Lowe. “Distinctive Image Features from Scale-Invariant Keypoints”. In: *International Journal of Computer Vision* 60.2 (2004), pp. 91–110.

[95] Martin A. Fischler and Robert C. Bolles. “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography”. In: *Communications of the ACM* 24.6 (1981), pp. 381–395.

[96] José Luis Pech-Pacheco et al. “Diatom Autofocusing in Brightfield Microscopy: A Comparative Study”. In: *International Conference on Pattern Recognition (ICPR)*. Vol. 3. 2000, pp. 314–317.

[97] Mark Sandler et al. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520.

[98] Wei Wu, Hanyang Peng, and Shiqi Yu. “YuNet: A Tiny Millisecond-Level Face Detector”. In: *Machine Intelligence Research* 20.5 (2023), pp. 656–665.

[99] Mingxing Tan and Quoc V. Le. “EfficientNetV2: Smaller Models and Faster Training”. In: *International Conference on Machine Learning (ICML)*. 2021.

[100] Emad Barsoum et al. “Training Deep Networks for Facial Expression Recognition with Crowd-Sourced Label Distribution”. In: *ACM International Conference on Multimodal Interaction (ICMI)*. 2016, pp. 279–283.

[101] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[102] Quinn McNemar. “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages”. In: *Psychometrika* 12.2 (1947), pp. 153–157.

[103] Frank Wilcoxon. “Individual Comparisons by Ranking Methods”. In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83.

[104] Sture Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70.

[105] Edwin B. Wilson. “Probable Inference, the Law of Succession, and Statistical Inference”. In: *Journal of the American Statistical Association* 22.158 (1927), pp. 209–212.

[106] Vladimir I. Levenshtein. “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals”. In: *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.

[107] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 32. 2019.

[108] Gary Bradski. “The OpenCV Library”. In: *Dr. Dobb’s Journal of Software Tools* (2000).

[109] Ramprasaath R. Selvaraju et al. “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017.

[110] *A Diverse Image Dataset for Bangladeshi Currency Recognition (BanglaTaka)*. Mendeley Data. URL: https://data.mendeley.com/datasets/3cv2sypkkh/1.

[111] Shan Li, Weihong Deng, and JunPing Du. “Reliable Crowdsourcing and Deep Locality-Preserving Learning for Expression Recognition in the Wild”. In: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

[112] Tsung-Yi Lin et al. “Focal Loss for Dense Object Detection”. In: *IEEE International Conference on Computer Vision (ICCV)*. 2017.

[113] Jiawei Mao et al. *POSTER V2: A Simpler and Stronger Facial Expression Recognition Network*. arXiv:2301.12149. 2023.

[114] Nibal Nayef et al. “ICDAR2019 Robust Reading Challenge on Multi-lingual Scene Text Detection and Recognition – RRC-MLT-2019”. In: *International Conference on Document Analysis and Recognition (ICDAR)*. 2019.

[115] *BanglaWild: An In-the-Wild Bengali Scene Text Recognition Benchmark for OCR and Vision-Language Models*. arXiv:2608.03884. 2026.